
Compositional Retrieval Systems

A Verified Architecture for Indexing, Querying, and Evidence-Gated Optimization

Architecture study of ragkit, ragopt, rag-ttc, GEC Chat, and the TTC Garden Assistant

August 2026

Table of Contents

Abstract.....	11
Preface and reader's guide.....	12
Principal claims.....	13
Part I. Empirical architecture.....	14
1. Research problem, questions, and method.....	15
1.1 The architectural problem.....	15
1.2 Research questions.....	15
1.3 Method.....	16
1.4 Why category theory appears here.....	16
2. The supplied system as an empirical object.....	17
2.1 Scale.....	17
2.2 Current dependency topology.....	18
2.3 Source overlap between ragkit and rag-ttc.....	18
2.4 Semantic overlap.....	19
3. ragkit: a strong RAG substrate with an overly broad infrastructure floor.....	20
3.1 Contracts-first structure.....	20
3.2 The source-lineage invariant.....	20
3.3 Representations are not evidence.....	20
3.4 Deterministic ordering and its remaining edge case.....	21
3.5 Immutable index bundles.....	21
3.6 Answering as an inspectable staged service.....	21
3.7 Flow and execution.....	22
3.8 The module-weight problem.....	22
4. ragopt: an evidence-gated experiment spine.....	23
4.1 Deliberate scope.....	23
4.2 Exactly-one-mutation candidates.....	23
4.3 Immutable run custody.....	23
4.4 Small product boundary: Arm and Outcome.....	23
4.5 Deterministic paired evaluation.....	24
4.6 Strict comparison and lexicographic gates.....	24
4.7 Correct exclusions.....	24
5. Applied systems as design constraints.....	25

5.1 GEC administrative chat.....	25
5.2 RAG-TTC.....	25
5.3 TTC Garden assistant.....	25
5.4 Admin chat is not a core package.....	26
6. Diagnosis: where the current abstractions fail.....	27
6.1 Duplicate authority is more dangerous than duplicate code.....	27
6.2 Identity is duplicated and under-specified.....	27
6.3 “Evidence” names three different things.....	27
6.4 Flow and optimization operate at different levels.....	28
6.5 Evaluation cannot be unified by type erasure alone.....	28
6.6 Security is currently a serving concern, not a plan property.....	28
6.7 Build, artifact, and release identity are conflated.....	29
6.8 The abstraction test.....	29
Part II. Foundations.....	30
7. Semantic identity, canonical bytes, and immutable artifacts.....	31
7.1 Identity is the architecture’s spine.....	31
7.2 Four identity strata.....	31
Semantic plan identity.....	32
Material artifact identity.....	32
Execution identity.....	32
Release identity.....	32
7.3 Domain-separated hashes.....	32
7.4 Canonical encoding.....	32
7.5 Merkle artifacts.....	33
7.6 Atomic publication.....	33
7.7 Artifacts versus caches.....	33
7.8 Provenance links.....	34
8. A compositional process model.....	35
8.1 Objects and morphisms.....	35
8.2 Parallel composition.....	35
8.3 Effects and the limits of ordinary functions.....	35
8.4 Why arrows fit the core.....	36
8.5 Free plans and interpreters.....	36
8.6 Static specification and dynamic policy.....	37
8.7 Determinism classes.....	37

8.8 Observation algebra.....	37
8.9 Failure as data versus interpreter failure.....	38
9. Trusted kernels and proof-carrying artifacts.....	39
9.1 The trusted-kernel principle.....	39
9.2 Kernel K0: canonical identity.....	39
9.3 Kernel K1: source lineage.....	39
9.4 Kernel K2: total ranking.....	40
9.5 Kernel K3: artifact verification and commit.....	40
9.6 Kernel K4: append-only ledger reducer.....	40
9.7 Kernel K5: exact pairing.....	40
9.8 Kernel K6: lexicographic gate.....	41
9.9 Kernel K7: grounding contract.....	41
9.10 Kernel K8: authorization noninterference boundary.....	41
9.11 Proof strategy.....	42
Part III. Proposed architecture.....	43
10. Module topology and ownership.....	44
10.1 Recommended topology.....	44
evidencekit - verified evidence kernel.....	44
ragkit - RAG semantics and implementations.....	44
ragopt - evidence-gated experiment semantics.....	44
ragbuild - optional build/release coordination.....	45
10.2 Why not put the kernel inside ragkit?.....	45
10.3 Proposed package list.....	45
10.4 Proposed ragkit public surface.....	45
10.5 Proposed ragopt public surface.....	45
10.6 Product packages.....	46
10.7 Import rules.....	46
11. The evidencekit API.....	47
11.1 Design constraints.....	47
11.2 Identity.....	47
11.3 Artifact store.....	48
11.4 Finite scores and total order.....	48
11.5 Outcome as a sum type.....	48
11.6 Observations and usage.....	49
11.7 Operation specifications.....	49

11.8 Typed plan values.....	50
11.9 Interpreters.....	51
11.10 Ledger.....	51
11.11 Law-test package.....	51
12. The ragkit domain API.....	52
12.1 Preserve the domain distinctions.....	52
12.2 Corpus types.....	52
12.3 Representation types.....	52
12.4 Index specifications.....	53
12.5 Build planning.....	53
12.6 Retrieval plan.....	54
12.7 Searcher capabilities.....	54
12.8 Retrieval result and trace.....	55
12.9 Answer plan.....	55
12.10 Metric definitions.....	55
12.11 Provider adapters.....	56
13. The ragopt API after kernel adoption.....	57
13.1 Preserve the current product boundary.....	57
13.2 Metric schema.....	57
13.3 Candidate assets as artifact references.....	57
13.4 Experiment plan identity.....	58
13.5 Cell coordinates.....	58
13.6 Runstore as a reducer.....	58
13.7 Comparison.....	58
13.8 Gate policy.....	59
13.9 Campaigns are a later meta-layer.....	59
14. Indexing and build coordination.....	60
14.1 Indexing is a deterministic plan around possibly nondeterministic producers.....	60
14.2 Build specification.....	60
14.3 Build state machine.....	61
14.4 Inner executor versus outer scheduler.....	61
14.5 Build registry.....	61
14.6 Full release manifests.....	62
14.7 Reproducibility under provider nondeterminism.....	62
15. Querying, answering, and administration.....	63

15.1 Querying is a subject-bound computation.....	63
15.2 Recommended stage order.....	63
15.3 Authorization strategies.....	63
Partitioned index.....	63
Backend prefilter.....	63
Exhaustive local filter.....	63
Bounded overfetch.....	64
15.4 Reranker failure and degradation.....	64
15.5 Context as a verified artifact.....	64
15.6 Query trace schema.....	64
15.7 Admin inspection API.....	64
15.8 Session evidence ledgers.....	65
16. Optimization as a meta-level search.....	66
16.1 Configuration graph.....	66
16.2 Evaluation mapping.....	66
16.3 The loop.....	66
16.4 Diagnostics as artifact inputs.....	66
16.5 Candidate generation.....	67
16.6 Search policy and budget.....	67
16.7 Promotion semantics.....	67
17. Security and trust boundaries.....	68
17.1 Threat model.....	68
17.2 Capability-indexed plans.....	68
17.3 Authorization certificates.....	68
17.4 Path safety.....	68
17.5 Cache and artifact poisoning.....	69
17.6 Evaluation integrity.....	69
17.7 Admin-data minimization.....	69
Part IV. Applied mappings and migration.....	70
18. GEC: from a pragmatic knowledge service to a verified query application.....	71
18.1 What GEC already gets right.....	71
18.2 Current-to-target ownership map.....	71
18.3 A GEC query profile.....	72
18.4 Move authorization before remote disclosure.....	72
18.5 Preserve the GEC reranking policy as policy.....	73

18.6 GEC evidence ledgers.....	73
18.7 Migrate GEC evaluation custody to ragopt.....	74
18.8 Release identity for GEC.....	75
18.9 GEC migration sequence and acceptance criteria.....	75
19. RAG-TTC: eliminate the second RAG core and retain the product system.....	76
19.1 The repository currently contains two architectural layers.....	76
19.2 Package cut line.....	76
19.3 Use behavioral fixtures before the hard cut.....	77
19.4 Resolve the known semantic drift explicitly.....	77
19.5 The RAG-TTC query architecture after cutover.....	78
19.6 The existing ragopt adapter is the reference integration pattern.....	78
19.7 Candidate materialization should become a plan.....	79
19.8 Admin chat for RAG-TTC.....	79
19.9 RAG-TTC migration acceptance criteria.....	80
20. TTC Garden assistant: a product experience over shared evidence.....	81
20.1 The Garden assistant is a distinct application.....	81
20.2 Target dependency topology.....	81
20.3 Product facts require release-level identity.....	81
20.4 Keep evidence presentation separate from evidence admission.....	82
20.5 Choice interactions are product-native evaluation state.....	82
20.6 Garden calibration on ragopt.....	83
20.7 Garden-specific gates.....	83
20.8 Garden admin and support inspection.....	84
20.9 Garden migration sequence and acceptance criteria.....	84
21. Migration program: dependency-ordered hardening.....	85
21.1 Principles.....	85
21.2 Wave 0: freeze semantics and create evidence.....	85
21.3 Wave 1: introduce the verified evidence kernel.....	85
21.4 Wave 2: make ragkit the sole common RAG implementation.....	86
21.5 Wave 3: establish full release identities.....	86
21.6 Wave 4: converge experiment custody on ragopt.....	86
21.7 Wave 5: introduce statically inspectable plans.....	86
21.8 Wave 6: build registry and activation coordination.....	87
21.9 Wave 7: higher assurance.....	87
21.10 Compatibility policy.....	87

21.11 Risk register.....	87
21.12 Decision checkpoints.....	88
Part V. Verification, operation, and use.....	89
22. Verification program.....	90
22.1 Assurance is layered, not binary.....	90
22.2 Verification matrix.....	90
22.3 Canonicalization tests.....	90
22.4 Identity law suite.....	91
22.5 Source-lineage properties.....	91
22.6 Ordering and fusion properties.....	92
22.7 Artifact-store fault model.....	92
22.8 Ledger reducer laws.....	93
22.9 TLA+ model for experiment custody.....	93
22.10 TLA+ model for build and activation.....	93
22.11 Exact pairing and comparison tests.....	94
22.12 Gate decision tables.....	94
22.13 Grounding validation.....	94
22.14 Authorization noninterference tests.....	95
22.15 Differential migration tests.....	95
22.16 Statistical evaluation without loss of custody.....	96
22.17 Performance and resource conformance.....	96
22.18 Proof candidates.....	96
23. Operations, versioning, and governance.....	97
23.1 Version every semantic boundary.....	97
23.2 Read compatibility versus write authority.....	97
23.3 Cache epochs.....	97
23.4 Release activation.....	98
23.5 Rollback.....	98
23.6 Artifact retention and garbage collection.....	98
23.7 Privacy and tenancy.....	98
23.8 Observability.....	99
23.9 Service-level objectives.....	99
23.10 Reproducibility levels.....	99
23.11 Module release policy.....	100
23.12 Ownership map.....	100

23.13 Architecture decision records.....	100
23.14 The two-consumer rule.....	101
23.15 Change review questions.....	101
24. Worked examples.....	102
24.1 Example A: build a hybrid index.....	102
24.2 Example B: authorized GEC query with reranker failure.....	102
24.3 Example C: optimize a TTC search-tool description.....	103
24.4 Example D: optimize a Garden multi-turn interaction.....	104
24.5 Example E: inspect a disputed answer.....	104
24.6 Example F: resume after process failure.....	105
25. Conclusions.....	106
25.1 Answers to the research questions.....	106
25.2 Immediate architectural decisions.....	106
25.3 What should not be unified.....	106
25.4 The deeper result.....	107
25.5 Final recommendation.....	107
Appendix A. Empirical inventory.....	108
A.1 Scope and counting method.....	108
A.2 ragkit package inventory.....	108
A.3 ragopt package inventory.....	109
A.4 ragkit versus copied RAG-TTC substrate.....	109
A.5 Applied-system overlap inventory.....	110
Appendix B. Consolidated API blueprint.....	111
B.1 Evidence kernel.....	111
B.2 Operation plans.....	112
B.3 ragkit semantic surface.....	113
B.4 ragopt semantic surface.....	114
B.5 Release and build surface.....	115
Appendix C. Laws, obligations, and conformance suites.....	116
C.1 Algebraic laws.....	116
C.2 Kernel proof obligations.....	116
C.3 Required conformance suites.....	117
C.4 CI layers.....	118
Appendix D. Architecture decisions and rejected alternatives.....	119
D.1 ADR-001: one common RAG implementation.....	119

D.2 ADR-002: domain-neutral evidence kernel.....	119
D.3 ADR-003: static core plans, dynamic application edges.....	119
D.4 ADR-004: native artifacts are authoritative.....	119
D.5 ADR-005: ordered gates, not weighted utility.....	119
D.6 ADR-006: four identity strata.....	120
D.7 ADR-007: authorization before remote disclosure.....	120
D.8 ADR-008: build scheduling remains external.....	120
D.9 ADR-009: finite scores only.....	120
D.10 ADR-010: full release roots.....	120
D.11 ADR-011: human promotion boundary.....	120
D.12 ADR-012: two-consumer extraction rule.....	121
Appendix E. Glossary.....	122
Appendix F. Selected bibliography.....	124

Abstract

Retrieval-augmented generation systems often begin as a practical sequence of scripts: ingest documents, split them into chunks, generate representations, embed them, build one or more indexes, retrieve candidates, rerank them, assemble context, generate an answer, and evaluate the result. As the system grows, the same sequence is reimplemented in several products, while optimization logic grows alongside it as a second, partially overlapping workflow. The result is not simply duplicated code. It is duplicated *semantic custody*: multiple components decide what identifies an input, what counts as evidence, when a result is replayable, how failures enter evaluation, and what can be promoted.

This study examines five Go codebases supplied as one development snapshot: `ragkit`, `ragopt`, `rag-ttc`, the GEC administrative chat, and the TTC Garden assistant in `ttc-design-system`. The snapshot contains 1,003 Go source files and 1,580 test functions across the five reviewed scopes. The most consequential empirical finding is a split RAG core. `ragkit` was extracted from `rag-ttc` and now contains the reusable contracts and newer correctness fixes, while `rag-ttc` continues to compile and serve from older source copies. GEC uses `ragkit`; the Garden assistant uses `rag-ttc`; `ragopt` is integrated only through a narrow RAG-TTC tool-evaluation adapter. In parallel, GEC and Garden each own evaluation, run, calibration, and evidence-recording mechanisms that overlap with `ragopt` but operate at different semantic levels.

The thesis proposes a layered architecture rather than a single universal framework. A small domain-neutral module, provisionally called `evidencekit`, contains only the correctness-critical substrate shared by `ragkit` and `ragopt`: versioned canonical encoding, typed content identity, immutable artifact references, total ordering, explicit outcomes, observation algebra, statically inspectable typed operations, append-only ledgers, and reusable law tests. `ragkit` remains the sole owner of RAG semantics: source lineage, representations, index artifacts, retrieval, fusion, reranking, context assembly, grounding contracts, and information-retrieval metrics. `ragopt` remains domain-neutral and owns immutable candidates, exact paired evaluation, comparison, lexicographic gates, experiment custody, and promotion evidence. A build and activation coordinator is kept separate and should become a shared module only after two products prove the same operational contract.

The formal model treats immutable schemas as objects and deterministic transformations as morphisms. Sequential composition is categorical composition; independent parallel stages form a monoidal product. Effectful plans are represented by an arrow-like, statically inspectable free process syntax rather than an unrestricted monadic interface, because the system must inspect future structure before execution to derive cache identity, budget requirements, trust-boundary checks, and audit graphs. Observations combine as monoids; provenance can be enriched toward a semiring when explanation algebra requires it. Correctness is concentrated in small validators and reducers, following the logic of trusted kernels and proof-carrying artifacts: complex producers may be untrusted, but their products are accepted only after a small kernel checks lineage, identity, ordering, state transitions, pairing, grounding, and authorization obligations.

The study provides package boundaries, Go API sketches, proof obligations, model-checking targets, property-law suites, product-specific mappings, and a staged migration plan. The immediate recommendation is a hard cut from the copied RAG core in `rag-ttc` to `ragkit`, preceded by behavioral fixtures. The second is to migrate GEC sweeps and Garden calibration custody onto `ragopt` through product-owned arms and native artifacts. The third is to distinguish semantic plan, material, execution, and release identities across indexing and serving. The result is a smaller, formally bounded architecture grounded in the applied systems.

Preface and reader's guide

This document is written in two modes at once. It is a thesis because it states and defends architectural claims from evidence. It is a textbook because it explains the concepts needed to implement those claims, including the cases where a tempting abstraction should be rejected.

Readers responsible for near-term code changes should begin with Chapters 2, 6, 10, 18, 19, 20, and 21. Readers designing the shared kernel should read Chapters 7 through 13 in order. Readers working on evaluation and optimization should focus on Chapters 5, 13, 16, and 22. Readers responsible for production activation should read Chapters 14, 17, and 23.

The package name `evidencekit` is provisional. The architectural role matters more than the name: it is a small module with no dependency on Bleve, SQLite, LLM providers, chat frameworks, CLI frameworks, or product code. The term *verified* in this document does not claim that the current code has been formally proved. It means that the proposed architecture identifies a small trusted computing base, states proof obligations precisely, and gives a realistic path from executable laws and model checking to machine-checked proofs where the return justifies the cost.

Principal claims

1. The highest-risk defect is not missing abstraction but duplicate semantic authority: `ragkit` and `rag-ttc/pkg` currently contain two RAG cores.
2. `ragkit` and `ragopt` should not import each other. They should meet through a third, tiny, domain-neutral evidence kernel.
3. The fundamental shared abstraction is not “RAG pipeline” but “typed, content-identified operation producing verified artifacts and observations.”
4. Plans should be statically inspectable. Arrow-like composition is a better default than unrestricted dynamic bind for the expensive, auditable core.
5. Product-native artifacts remain authoritative. Shared packages should project only the small comparison and inspection contracts they can define honestly.
6. Correctness should be concentrated in validators, reducers, and comparators that are small enough to test exhaustively and, selectively, prove.
7. Authorization must constrain the candidate set before any remote reranker or generator receives source text.
8. Optimization gates are ordered predicates, not a universal weighted score.
9. Build intent, produced bytes, execution attempts, and active releases require different identities.
10. The migration should use hard ownership cutovers, not permanent compatibility layers.

Part I. Empirical architecture

1. Research problem, questions, and method

1.1 The architectural problem

A RAG engine has at least three lifecycles:

- an indexing lifecycle that turns source revisions into verified retrieval artifacts;
- a querying lifecycle that turns a subject-bound request into authorized evidence and a contracted answer;
- an optimization lifecycle that turns diagnostics into isolated candidates, paired evidence, a gate decision, and an externally authorized promotion.

These lifecycles share mechanisms, but they do not share all semantics. Indexing and querying reason about documents, chunks, representations, vectors, ranks, and citations. Optimization must remain capable of comparing systems that do not use those concepts at all. The shared architecture therefore has to solve two opposing problems: remove accidental duplication while preserving legitimate domain boundaries.

The supplied code already contains strong local mechanisms. `ragkit` has narrow interfaces, deterministic retrieval, immutable index bundles, exact chunk lineage, flow-level cache identity, resource admission, and grounded-answer validation. `ragopt` has immutable one-mutation candidates, copied inputs, append-and-sync result custody, exact pairing, explicit missing outcomes, ordered gates, and human-only promotion. The applied systems contain the pressure-tested details that generic packages often omit: access scopes, source roles, synonym expansion, product-fact augmentation, session evidence ledgers, UI projections, polling semantics, native judge artifacts, and fallbacks under provider failure.

The design problem is therefore not to invent a cleaner system in isolation. It is to infer the minimal common algebra from the systems that already have to work.

1.2 Research questions

This study asks five questions.

RQ1. What is the smallest shared kernel that both `ragkit` and `ragopt` need? The answer must not drag RAG dependencies into a domain-neutral optimizer, and it must be strong enough to eliminate duplicate identity, artifact, ordering, outcome, and ledger logic.

RQ2. Which structures make indexing, querying, and optimization compositional without hiding operational facts? The answer must support sequential and parallel composition while keeping cache identity, effects, budgets, and trust boundaries inspectable before execution.

RQ3. Which current applied mechanisms belong in shared packages, and which should remain product-owned? The answer must be grounded in GEC, RAG-TTC, and Garden rather than in a hypothetical universal chatbot.

RQ4. What correctness claims can be made executable? The answer should identify small kernels, algebraic laws, state-machine invariants, property tests, fuzz targets, and model-checking boundaries.

RQ5. How can the current system migrate without invalidating working products? The answer must give concrete package moves, compatibility windows, acceptance criteria, and deletion points.

1.3 Method

The analysis used four complementary methods.

First, a static inventory counted Go files, tests, packages, imports, and broad package fan-in for each codebase. Second, a path-aligned comparison matched `ragkit` against `rag-ttc/pkg`, normalizing module paths to separate extraction noise from semantic drift. Third, key contracts were read at the identity and state boundaries: chunk validation, hit ordering, bundle identity, cache keys, flow steps, candidate validation, run custody, paired cells, comparison, gates, serving authorization, tool evidence, and calibration records. Fourth, the applied implementations were treated as counterexamples against premature generalization. A proposed abstraction was accepted only when at least two mechanisms needed the same semantics or when it belonged to a correctness kernel that every implementation must trust.

The study is limited to the supplied snapshot. It did not execute production traffic, inspect private deployment configuration outside the archive, benchmark providers, or interview operators. The resulting architecture is a defensible design hypothesis with executable migration tests, not a claim that runtime behavior has already been proven equivalent.

1.4 Why category theory appears here

Category theory is useful in this setting for a practical reason: it asks what can be composed while preserving meaning. A category has objects, morphisms, identity morphisms, and associative composition. RAG systems already rely on these ideas informally. A document-to-chunks transformation, chunks-to-representations transformation, and representations-to-vectors transformation are typed arrows. The identity stage should change nothing. Regrouping a sequence of stages should not change its denotation. Independent lexical and vector searches can be composed in parallel and fused later.

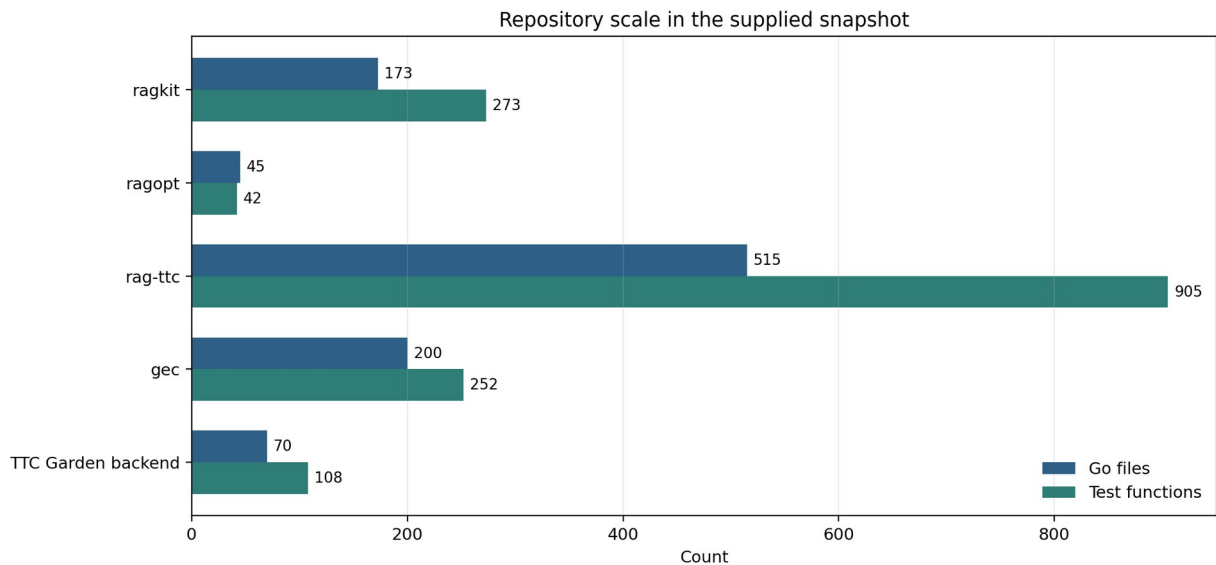
The theory becomes operationally valuable only when it changes APIs and tests. This thesis therefore avoids decorative terminology. Each formal structure is paired with a proposed Go representation, an interpreter, and a law suite.

2. The supplied system as an empirical object

2.1 Scale

The reviewed snapshot contains five relevant scopes. Counts use non-vendored `.go` files and test functions named with Go's `Test...` convention. Nonblank source-line counts are descriptive only; they are not a quality metric.

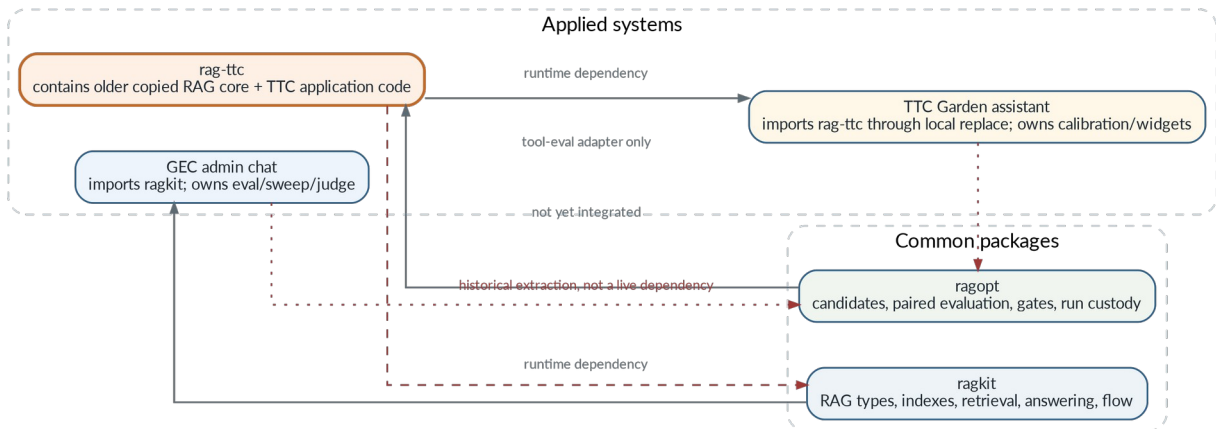
Scope	Go files	Test files	Test functions	Nonblank Go LOC	Packages
ragkit	173	61	273	17,743	23
ragopt	45	9	42	5,925	12
rag-ttc	515	179	905	76,705	77
GEC	200	65	252	28,668	34
TTC Garden backend	70	24	108	8,485	19



Repository scale in the supplied snapshot.

The raw scale matters because it changes the cost of ownership ambiguity. A duplicated utility in a small prototype can be repaired locally. A duplicated RAG core beneath three applications creates divergent cache epochs, bundle formats, tie-breakers, prompts, and operational expectations.

2.2 Current dependency topology



Current dependency topology. The red relationship is historical extraction, not a live source dependency.

The topology has four important properties.

1. GEC imports `ragkit` and therefore receives the extracted package's newer invariants.
2. `rag-ttc` does not import `ragkit`; it continues to compile the source tree from which `ragkit` was extracted.
3. The Garden backend imports `rag-ttc` through a local replacement and therefore inherits the older common core plus TTC-specific packages.
4. `ragopt` is used by RAG-TTC only in the chat tool-evaluation adapter. GEC and Garden have their own parallel evaluation and calibration paths.

This is a split-brain architecture. There is no single component whose source defines “the RAG kernel” for all products.

2.3 Source overlap between `ragkit` and `rag-ttc`

A path-aligned comparison of files under `ragkit` and `rag-ttc/pkg` found 50 byte-exact files, 47 files equal after module-path normalization, 68 changed files, and 11 files without an upstream path match. The unmatched files include the `ragkit` boundary test, module metadata, internal utility copies, and the lexical-only bundle test.

The changed count overstates semantic divergence because log-area names and import paths changed during extraction. The substantive differences are more revealing:

- `ragkit/rag/ordering.go` adds `DocumentID` to the deterministic hit tie-break.
- representation prompts are injectable through a `PromptSet` rather than fixed TTC package constants;
- index bundles can be lexical-only and open without a vector embedder;
- the grounded-answer contract kind is injectable;
- bundle IDs use a package-neutral `rk-` prefix;
- a boundary test forbids chat, CLI, and provider-framework dependencies from leaking into the reusable core.

These are exactly the kinds of changes that should have one authority. They affect determinism, rollback configurations, cache identity, and package boundaries.

2.4 Semantic overlap

Semantic ownership and overlap					
Semantic identity	owns	owns	parallel	uses	uses
Immutable artifacts	owns	owns	parallel	owns	owns
Execution/retry/cache	owns	owns	parallel	owns	owns
Retrieval & ranking	owns	-	parallel	owns	owns
Answer contracts	owns	uses	parallel	owns	owns
Evaluation metrics	owns	uses	parallel	parallel	parallel
Paired comparison & gates	-	owns	owns	parallel	parallel
Session evidence	-	-	owns	owns	owns
Admin/UI composition	-	-	owns	owns	owns
	ragkit	ragopt	rag-ttc	GEC	Garden

Semantic ownership and overlap. "Parallel" indicates independently implemented mechanisms with substantially overlapping responsibility.

The overlap is layered rather than uniform. Retrieval and answer semantics belong naturally in `ragkit`. Paired comparison and gates belong naturally in `ragopt`. Identity, immutable artifacts, outcomes, observations, and event custody appear in both. Evaluation appears everywhere, but at different levels:

- `ragkit/rag/evaluation` computes deterministic information-retrieval metrics;
- GEC evaluates retrieval strata and judged answers, then sweeps serving parameters;
- RAG-TTC evaluates tool-loop behavior and answer quality;
- Garden calibration replays multi-turn sessions and checks UI-visible expectations;
- `ragopt` compares opaque product outcomes and applies generic gate mechanics.

The design must not collapse those meanings into one `Evaluate` interface. It must give them common custody and identity while preserving product-defined cases and metrics.

3. ragkit: a strong RAG substrate with an overly broad infrastructure floor

3.1 Contracts-first structure

ragkit explicitly describes itself as a contracts-first extraction. The root `rag` package defines immutable documents, exact chunks, derived representations, vectors, queries, judgments, hits, fused hits, evidence, provider usage, and narrow component interfaces. Subpackages implement chunking, representation generation, embeddings, lexical and vector indexes, bundle construction, retrieval, reranking, generation, answering, evaluation, and dataset loading. Generic-looking packages named `digest`, `text`, `vector`, `execution`, and `flow` sit beside the RAG domain.

This is a generally sound layering choice. Domain interfaces are small:

```
type Chunker interface {
    Name() string
    Chunk(context.Context, Document) ([]Chunk, error)
}

type Searcher interface {
    Search(context.Context, Query, int) ([]Hit, error)
}

type Reranker interface {
    Rerank(context.Context, RerankRequest) (RerankResult, error)
}
```

The interfaces describe one capability each and do not force a universal engine object. This is preferable to a large service interface with unrelated methods for `build`, `search`, `answer`, `evaluate`, and `administer`.

3.2 The source-lineage invariant

The strongest existing kernel is `ValidateChunk`. A chunk must reference the correct document, use a valid half-open byte range, contain valid UTF-8, and equal the exact source byte slice for that range. This is more important than it first appears. Generated representations may be useful retrieval material, but they cannot become authoritative source evidence. Hydration returns chunks precisely because chunks retain the verified path to source bytes.

The corresponding theorem is simple:

For a valid document `d` and valid chunk `c`, `c.Text = d.Text[c.Start:c.End]`.

Everything that cites a chunk can therefore be traced to immutable source text. The current validator does not independently recompute and compare the content digest, so a strengthened kernel should also require `Hash(c.Text) = c.ContentDigest` and `Hash(d.Text) = d.ContentDigest` under a versioned domain.

3.3 Representations are not evidence

ragkit separates `Representation` from `Chunk`. A representation can be raw chunk text, a breadcrumb expansion, a summary, a generated question, or another retrieval-oriented derivative. A search hit identifies a representation and its source chunk. Collapse maps representation hits to chunk identities. Hydration resolves those identities to source evidence.

This distinction is one of the architecture's most valuable domain decisions. It prevents a generated summary from silently becoming a cited source. The proposed APIs retain the distinction and make it

more explicit by renaming the hydrated type `SourceEvidence` and reserving `ExperimentEvidence` for optimization artifacts.

3.4 Deterministic ordering and its remaining edge case

`HitRanksBefore` orders by descending score and then by document, chunk, and representation identity. This removes dependence on map iteration, database iteration, or unstable sort behavior. Weighted reciprocal-rank fusion also sorts channel names and uses a chunk-ID tie-break.

The remaining correctness gap is floating-point non-finiteness. IEEE NaN is unequal to every value, including itself. A comparator written as `if left.Score != right.Score { return left.Score > right.Score }` does not define a total order over all `float64` values. The kernel should either reject non-finite scores at construction or use a documented total-order policy. Rejection is the safer default for retrieval and metric values:

```
type Score float64

func NewScore(v float64) (Score, error) {
    if math.IsNaN(v) || math.IsInf(v, 0) {
        return 0, ErrNonFinite
    }
    return Score(v), nil
}
```

Once scores are finite, property tests can state transitivity, antisymmetry, and totality directly.

3.5 Immutable index bundles

`rag/indexbundle` builds immutable bundles with a manifest, corpus digest, chunker identity, representation kinds and digest, lexical backend identity, and optional vector identity. Publication is atomic; opening revalidates stored data against the manifest. The extracted package improves the older copy by supporting lexical-only bundles, which are useful for serving, degradation, and rollback.

The current bundle ID is close to a material identity: it depends on the exact documents, representations, and backend specifications. The manifest's `CreatedAt` and `path` are not part of the calculated ID, which is correct. The next design should distinguish three related values:

- a **build plan ID**, identifying the semantic intent before execution;
- an **index artifact ID**, identifying the exact produced manifest and files;
- a **release ID**, identifying the activated set of artifacts and auxiliary product data.

This distinction becomes essential when generation or embedding providers are nondeterministic. The same plan may produce different exact bytes, and the artifact digest rather than the plan ID must be authoritative for replay.

3.6 Answering as an inspectable staged service

The answering service supports lexical, vector, RRF, reranked RRF, multi-query, and HyDE strategies. It retains channel results, query variants, fused hits, evidence, augmentation traces, context, generation, and contract results. The grounded-answer contract parses a structured answer, requires explicit abstention semantics, forbids citations on abstention, requires citations on non-abstention, rejects duplicates, and verifies that every cited chunk appeared in the supplied evidence.

This is already a small proof-carrying design. The generator is not trusted to obey the contract. It emits a candidate answer, and a deterministic validator either accepts it or substitutes a safe abstention. The

proposed architecture generalizes this pattern: expensive or probabilistic components produce candidates and certificates; small deterministic kernels accept or reject them.

3.7 Flow and execution

`flow.Step[I,0]` combines a name, semantic cache identity, execution policy, barrier flag, work function, meter, and result hook. `Identity` names a kind and version and extracts exact key bytes. `Policy` contains workers, admission resources, retry, and a failure mode. The comments correctly insist that worker count, retry, and similar execution choices must not enter semantic cache identity.

The implementation also has bounded ordered mapping, budgets, rates, atomic cache entries, corruption detection, and fail-fast, quarantine, or skip behavior. These are strong mechanisms. The abstraction issue is that `Step` is simultaneously a domain-neutral operation description, an execution closure, a cache contract, and a reporting hook. Its pipeline composition is inspectable only through internal flattened stage state, and its identity encoding is coupled to `encoding/json` and a stale `rag-ttc-execution-cache/v1` schema name.

The proposed `op` package keeps the good separation between semantic specification and execution policy but makes the plan a first-class static syntax with multiple interpreters. Existing `flow` can first become an interpreter over that syntax and then be retired or retained as a convenience layer.

3.8 The module-weight problem

Although `digest`, `execution`, and `flow` are domain-neutral in concept, they live in the same Go module as `Bleve` and `SQLite` implementations. Making `ragopt` import them would violate its deliberate independence and pull a large RAG dependency graph into experiment-only programs. This is the principal reason for a third kernel module rather than `ragopt -> ragkit`.

4. ragopt: an evidence-gated experiment spine

4.1 Deliberate scope

ragopt is explicitly designed as a separate repository. Its design states the intended dependency direction:

```
product runtime      -> may depend on ragkit
product experiment command -> depends on ragopt
ragopt               -> does not depend on product runtime or ragkit
```

This is the right boundary. An optimization harness should be able to compare a RAG engine, a SQL policy, a routing policy, a tool description, or a prompt-only system without importing document or vector-index types.

The current lifecycle is:

```
diagnose -> propose -> evaluate -> compare -> gate -> review -> promote/reject
```

ragopt owns the middle evidence spine. Diagnosis and proposal remain product-owned or human. Promotion remains a human or product release action.

4.2 Exactly-one-mutation candidates

A candidate bundle contains a parent snapshot, a child snapshot, locked assets, mutable assets, dimensions, a mutation declaration, proposer identity, expected improvement, regression risks, and diagnostic evidence links. Loading is strict. Paths remain inside the bundle, bytes and digests are verified, locked assets and dimensions must remain equal, the mutable asset set cannot change, and exactly one mutable asset must differ.

This is not merely an experiment convenience. It is an attribution theorem:

Given a valid candidate and fixed execution/evaluation identities, any observed behavioral difference is attributable to the one changed asset plus declared nondeterminism.

The theorem is conditional, but the candidate validator makes its structural assumptions executable. Multi-asset optimization can be represented as a sequence or graph of single-asset edges rather than weakening the core invariant.

4.3 Immutable run custody

runstore creates a self-describing directory with a manifest, canonicalized configuration digest, host and Go metadata, copied inputs, result and native-artifact directories, atomic JSON writes, append-and-sync JSONL, and terminal complete or failed state. A run is writable while active and rejects writes after terminal transition. Resume requires exact configuration identity.

This design correctly separates a semantic configuration digest from a random, time-based run ID. The run ID identifies an execution attempt. It should never become a cache key or candidate identity.

The remaining shared opportunity is to implement the run directory as one interpreter of a generic artifact store and append-only ledger. The experiment semantics remain in ragopt; the durability and hash-chain semantics move to the small kernel.

4.4 Small product boundary: Arm and Outcome

The central adapter is intentionally narrow:

```
type Arm interface {
    Name() string
```

```
Run(ctx context.Context, request Request) (Outcome, error)
}
```

A case carries stable identity, groups, and opaque JSON input. An outcome carries completion, contract validity, abstention, an attributable failure, product-defined metrics, coarse cost counters, duration, and a verified native artifact. The native artifact remains product-owned and is copied or written inside the assigned cell directory.

This is a strong example of an honest abstraction. `ragopt` does not pretend that a GEC judge transcript, RAG-TTC tool trace, and Garden session snapshot are the same structure. It compares the small shared projection and preserves exact native evidence by digest.

A useful refinement is to make `Outcome` impossible to construct in contradictory states. Today booleans permit combinations such as incomplete, contract-valid, and no failure. A constructor-based sum type can enforce success, abstention, attributable failure, and infrastructure failure as distinct cases.

4.5 Deterministic paired evaluation

The runner schedules the incumbent and challenger for every `(case ID, repeat index)` coordinate. The cell identity includes the suite digest, policy digest, candidate ID, snapshot digest, case, repeat, and arm. Ordinary product arm errors become explicit failed outcomes and retain native artifacts; cancellation and custody failures stop the run.

This distinction is essential:

- a provider timeout for one arm is experiment data;
- inability to durably write the cell means the experiment record is not trustworthy.

The same distinction should be used in `evidencekit/outcome`: attributable failures are values, while interpreter integrity errors remain Go errors.

4.6 Strict comparison and lexicographic gates

`compare` pairs cells exactly by case and repeat, exposes missing pairs, computes per-pair metric and cost deltas, and aggregates by group and metric. `gate` evaluates ordered phases: hard invariants, target improvement, regressions, and then cost tie-breakers. A failed earlier phase stops later quality claims from authorizing promotion.

This ordered structure is preferable to a single weighted score. A weighted score allows an inexpensive but catastrophically wrong system to compensate quality failure with cost savings. In the proposed API, metric definitions also declare direction and unit so that “higher is better” is not assumed globally.

4.7 Correct exclusions

The current design explicitly excludes retrieval, indexing, chat loops, universal judges, transcript warehouses, search algorithms, schedulers, deployment, and automatic promotion. Those exclusions should remain. `ragopt` may later add a campaign package for proposing or selecting candidates, but only after two product integrations show a common interface. It should not become a generic workflow engine.

5. Applied systems as design constraints

5.1 GEC administrative chat

GEC is the cleanest existing consumer of `ragkit`. Its knowledge service opens and verifies an index bundle, reconstructs a query embedder from the vector manifest when necessary, loads verified source documents, and keeps the service immutable across sessions. Per-session evidence lives in a separate bounded ledger.

Its `SearchRequest` adds server-owned access scopes, source roles, and route selection. The serving path supports lexical-only retrieval, hybrid fusion, curated lexical synonym expansion, optional reranking, and a fallback to fused rank when the reranker fails. It then applies another rank blend between fused and reranked order. These are product-serving policies, not generic index semantics.

The most important security detail is explicit in the code: access filtering occurs after ranking with fixed overfetch. Unauthorized documents are not returned, but they consume rank positions and can starve eligible results. This is acceptable as a documented transitional behavior only if filtering occurs before any remote reranker or generator receives text. The target design supports backend prefilters or scope-partitioned indexes and makes authorization a named plan stage.

GEC also owns a retrieval evaluation set, answer judge, statement judge, reranker providers, parameter sweeps, best-cell selection, tool DTOs, and evidence ledger. These mechanisms should not be copied into `ragkit`. The retrieval metrics can use `ragkit/eval`; the full evaluation execution, pairing, custody, and gates should use `ragopt`; the judge and native artifacts remain GEC-owned.

5.2 RAG-TTC

RAG-TTC is both the research origin of `ragkit` and a pragmatic product/research application. It contains the copied common RAG packages plus TTC-specific components such as connected runtime assembly, tool-based answering, tool configuration, product catalog integration, provider adapters, application sessions, admin UI, diagnostics, review, and judgments.

The existing `cmd/rag-ttc/cmds/chat/tooleval/ragopt.go` is the reference integration pattern. It constructs a `ragopt` arm, materializes candidate-owned tool configuration, executes the real product loop, projects a small outcome, and writes a native artifact atomically inside the cell. This adapter demonstrates that `ragopt` does not need to understand TTC chat internals.

The major architectural action is not another abstraction. It is deletion: replace imports of `github.com/the-tree-center/rag-ttc/pkg/rag...`, `pkg/flow`, `pkg/execution`, `pkg/digest`, `pkg/text`, and `pkg/vector` with the `ragkit` authority, then remove the copied packages after behavior fixtures pass. TTC-specific prompts, provider adapters, product facts, tools, and connected runtime remain in TTC packages.

5.3 TTC Garden assistant

The Garden backend imports RAG-TTC through a local module replacement. Its RAG search service opens a bundle, configures routes and sessions, wraps the TTC search tool, adds intent and product-fact augmentation, records structured facts, and maps evidence into grounded widgets. Its real runtime composes provider, middleware, chat, and UI concerns.

Garden calibration is a separate replay engine. It creates a session, submits deterministic idempotency keys, polls until a terminal answer settles, records turn snapshots, and checks expectations such as

answer presence, choice count, source kinds, and maximum answer words. This is valuable product logic. Its run identity, cell custody, repeated evaluation, and comparison logic overlap with `ragopt`.

The correct migration is not to make `ragopt` understand chat choices. Garden should expose each calibration case as an opaque suite case, execute the current multi-turn runner inside an arm, emit its `TurnRecord` collection as the native artifact, and project product metrics such as expectation success, source-kind coverage, and latency. The existing expectation checker remains product-owned.

5.4 Admin chat is not a core package

GEC, RAG-TTC, and Garden all have administrative or inspection needs, but their UI and session semantics differ. A shared admin-chat framework would likely couple the core to sessionstream, widgets, provider middleware, or CLI libraries. The reusable boundary should be smaller:

- a typed query request and result;
- a sequence of named stage observations;
- source-evidence and citation views;
- artifact references for deep inspection;
- capability and plan-description endpoints.

Each application owns authentication, session state, commands, projections, and widgets.

6. Diagnosis: where the current abstractions fail

6.1 Duplicate authority is more dangerous than duplicate code

Code duplication is usually discussed as a maintenance cost. In this system, the more serious problem is that duplicated packages decide semantic facts independently. The copied cores can disagree about:

- what fields enter a cache key;
- what ties define final rank order;
- whether a lexical-only artifact is valid;
- which prompt text identifies a representation;
- what contract kind a generator must emit;
- which schema string appears in persisted entries;
- which dependencies are permitted inside the reusable layer.

When those decisions diverge, the same apparent query configuration can produce different evidence in GEC and Garden. A bug fix applied to one core is not a system fix. The first governing rule should therefore be:

Every persisted semantic format and every correctness-critical comparator has exactly one source owner.

6.2 Identity is duplicated and under-specified

Both `ragkit` and `ragopt` hash JSON. `ragkit/digest.JSON` relies on Go's `encoding/json`; `ragopt` separately canonicalizes JSON input and then hashes it. The current behavior is deterministic for many Go values, including sorted string map keys, but the contract is tacit, Go-specific, and vulnerable to schema evolution. Identity-critical structures also include maps and, in run summaries, `map[string]any`.

A long-lived identity contract should define:

1. the domain and schema version;
2. the canonical byte encoding;
3. whether absent and zero values are distinct;
4. ordering rules for sets and maps;
5. float admissibility and normalization;
6. string normalization, if any;
7. domain separation before hashing.

RFC 8785 JSON Canonicalization Scheme or deterministic CBOR under RFC 8949 are suitable standards. The choice matters less than making it explicit, versioned, and testable across implementations.

6.3 “Evidence” names three different things

The code uses evidence to mean at least three concepts:

- source evidence: a verified source chunk used for grounding;
- derivational evidence: a trace explaining how a result was produced;
- experimental evidence: cells, metrics, native artifacts, and gate decisions used for promotion.

Overloading the term encourages invalid substitutions. A generated representation is not source evidence. A retrieval score is not source evidence. A promotion report is not a citation. The proposed vocabulary is:

- `SourceEvidence` for immutable, hydrated source content;
- `Derivation` or `Trace` for operation provenance;
- `ExperimentRecord` or `PromotionEvidence` for optimization custody.

The kernel may offer common artifact and observation primitives, but domain packages should use precise names.

6.4 Flow and optimization operate at different levels

`ragkit/flow` manages item-level or stage-level execution inside a build or query: cache, retry, admission, failure mode, batch, and reporting. `ragopt` manages experiment-level work: candidate, suite, case, repeat, arm, pair, gate, and promotion report.

The overlap is real - identity, outcomes, artifacts, events, and resume - but the levels should not be collapsed. A flow step may be replayed inside one arm cell. A build job may contain a flow plan. A `ragopt` run should not schedule individual embeddings, and a flow engine should not decide whether a prompt candidate is promotable.

This distinction can be formalized as object level versus meta level:

- object-level operations transform RAG data;
- meta-level operations compare immutable configurations of the object-level system.

The shared kernel supports both without owning either domain.

6.5 Evaluation cannot be unified by type erasure alone

A generic interface that accepts `any` and returns `map[string]float64` appears reusable, but it discards the contracts that make an evaluation trustworthy. Retrieval evaluation requires judgments, target levels, cutoffs, and deterministic rank order. Answer evaluation may require a judge prompt, statement decomposition, citation checks, and failure accounting. Garden calibration requires multi-turn state and UI-visible choices.

The reusable contract is not the evaluation function. It is the experiment envelope:

- immutable suite identity;
- stable case coordinates;
- product-owned execution;
- explicit outcome or failure;
- native artifact custody;
- exact pairing;
- product-declared metric semantics;
- ordered gate policy.

`ragopt` already points in this direction. The proposed architecture strengthens the envelope rather than generalizing the evaluator.

6.6 Security is currently a serving concern, not a plan property

GEC carries access scopes and source roles in its request and filters results. The shared RAG contracts do not describe whether a searcher supports a prefilter, which stages may cross a network boundary, or what evidence is safe to send to a provider. As a result, authorization is invisible to cache and plan inspection unless each product remembers to include it.

Authorization should be a first-class operation and a declared effect constraint. A static plan analyzer should be able to reject a plan in which an operation marked `RemoteDisclosure` precedes the operation that establishes an authorized candidate set.

This is not a complete information-flow proof, but it turns a critical sequencing rule into an executable plan invariant.

6.7 Build, artifact, and release identity are conflated

An index bundle is immutable, but production systems also need build attempts, evaluations, activation generations, and rollback. RAGOPT's build design correctly proposes a state machine and distinguishes build identity from worker counts, retries, timestamps, job IDs, and logs. The current package structure does not give that distinction a shared typed representation.

The proposed design introduces four identity strata, shown in Figure 9 later in this document. A build plan can be identical while two execution attempts differ. Two attempts may produce the same exact artifact. A release can pair an index artifact with a product-fact database or policy artifact. Activation changes release state without changing the underlying bytes.

6.8 The abstraction test

A candidate shared abstraction should pass five tests:

1. **Semantic necessity:** at least two packages need the same meaning, not merely similar syntax.
2. **Stable ownership:** the abstraction can have one owner without importing product behavior.
3. **Lawfulness:** useful invariants can be stated and tested independently of an implementation.
4. **Dependency economy:** consumers do not inherit unrelated heavy dependencies.
5. **Deletion power:** adopting it removes existing code or prevents a known duplicate.

`identity.Ref`, a total score comparator, an artifact store, and an event reducer pass. A universal chat transcript, provider interface, or optimizer does not yet pass.

Part II. Foundations

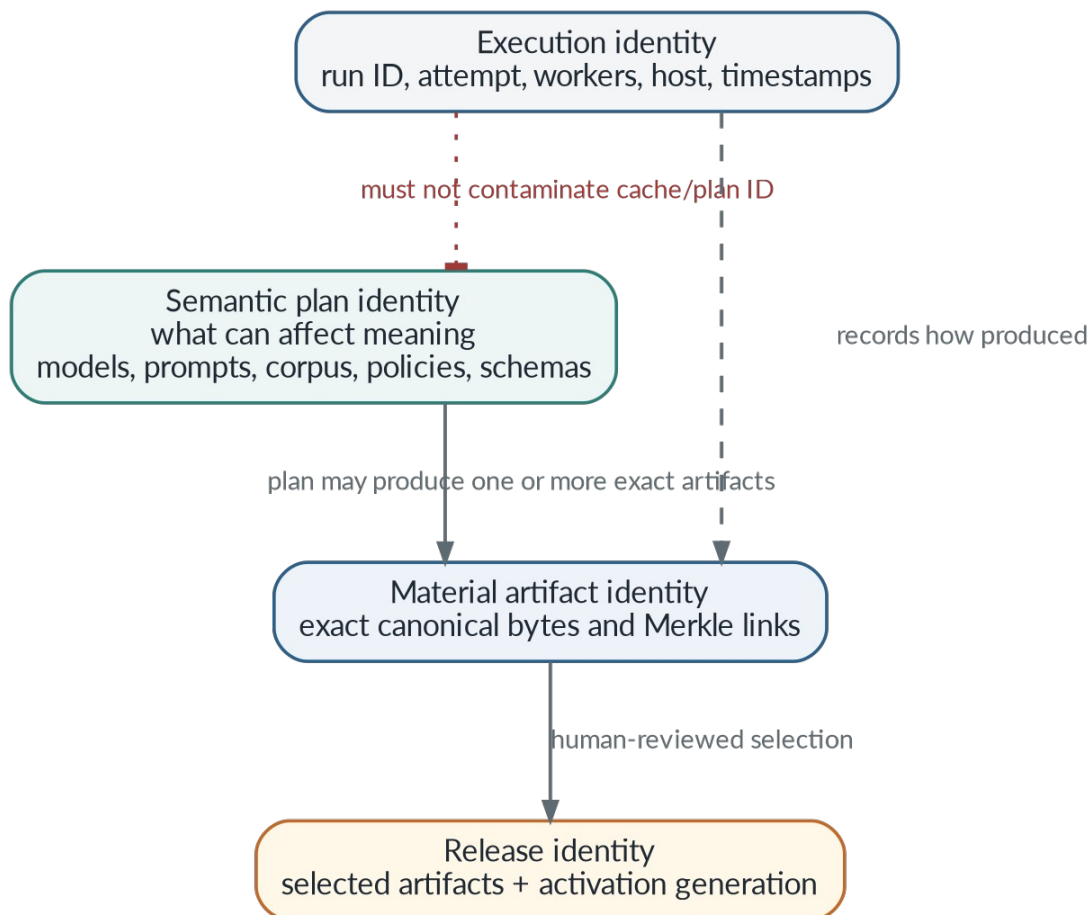
7. Semantic identity, canonical bytes, and immutable artifacts

7.1 Identity is the architecture's spine

Indexing, caching, replay, comparison, and promotion are all claims about sameness. A cache hit claims that two requests have the same denotation. A bundle ID claims that stored files represent one exact retrieval artifact. A paired experiment claims that incumbent and challenger differ only in a declared mutation. A release manifest claims that the activated bytes are the reviewed bytes.

A system that does not define sameness explicitly cannot define reproducibility. This is why identity belongs in the smallest shared kernel.

7.2 Four identity strata



Four identity strata. Execution details record how work occurred but must not contaminate semantic plan identity.

The strata are:

Semantic plan identity

This identity covers every input that can affect intended meaning: corpus revision, chunker algorithm and parameters, normalization, prompt, model, provider semantics, representation set, embedding model and dimensions, retrieval strategy, fusion weights, reranker, authorization policy, context policy, answer contract, suite, judge, and gate policy.

It excludes worker counts, retry delays, output directories, hostnames, start times, and job IDs.

Material artifact identity

This is the digest of exact canonical bytes plus typed links to dependent artifacts. It answers, “Which bytes were actually produced?” A material ID may differ across executions of the same plan when a provider is nondeterministic.

Execution identity

This identifies one attempt: run ID, attempt number, host, process version, worker policy, start and finish time, retry history, provider request IDs, and logs. It is essential for operations and audit, but it is not semantic sameness.

Release identity

This identifies the reviewed set of material artifacts selected for serving, together with an activation generation and rollback relation. A Garden release may need both an index artifact and a product-fact database artifact. A GEC release may include an index plus serving configuration and authorization-policy references.

7.3 Domain-separated hashes

Hashing canonical bytes directly is insufficient when the same byte sequence can represent different schemas. The kernel should hash a domain prefix, schema identity, and length-delimited bytes:

```
H(
  "evidencekit-id-v1" ||
  len(domain) || domain ||
  len(schema) || schema ||
  len(payload) || canonical_payload
)
```

This prevents accidental cross-type identity collisions even when the cryptographic hash itself does not collide. The schema should be explicit, such as `ragkit.chunk/v2` or `ragopt.suite/v1`.

7.4 Canonical encoding

A canonical encoder must be deterministic over its admitted value domain. For JSON, the kernel should adopt RFC 8785 or a deliberately narrower profile. For CBOR, it can adopt deterministic encoding under RFC 8949. A narrower profile is often better:

- UTF-8 strings with no implicit Unicode normalization;
- object keys sorted by encoded key bytes;
- no non-finite floats;
- integral counts encoded as integers, not floats;
- explicit omission rules per schema;
- sets sorted by a schema-defined total key before encoding;

- time encoded in UTC with fixed precision when time is semantic;
- no `map[string]any` in identity-critical values.

The codec itself should have golden vectors checked in Go and at least one independent implementation. Cross-language fixtures are more convincing than comments about determinism.

7.5 Merkle artifacts

Git, IPFS, and Merkle's original tree construction illustrate the practical power of content-addressed data: a parent's identity commits to the identities of its children. The proposed artifact model is a typed Merkle DAG.

An artifact envelope contains:

- a schema identity;
- media type;
- payload byte digest and size;
- ordered or named links to other artifact IDs;
- optional non-semantic annotations such as creation time and producer run.

An index artifact links to corpus, chunks, representations, lexical files, vector files, and manifest. A `ragopt` cell links to the suite, candidate snapshot, policy, native artifact, and small outcome projection. A release links to one or more serving artifacts.

7.6 Atomic publication

Content identity is useful only when partial data cannot appear under a valid reference. A filesystem store should implement:

1. write payload to a unique temporary file in the target filesystem;
2. flush payload;
3. calculate and verify the expected digest;
4. atomically rename into the content-addressed path;
5. flush the containing directory;
6. publish the manifest or root reference last.

The operation is idempotent: publishing the same bytes under the same ID succeeds without mutation. Existing `ragkit` cache and index-bundle code already implements much of this discipline. Existing `ragopt` run writes and synced JSONL provide the other half.

7.7 Artifacts versus caches

A cache is an index from semantic operation identity and input identity to a material output reference. The cached output is not trusted merely because it exists. Load verifies the envelope, key, schema, size, and output digest. Corruption fails closed.

This suggests a unified shape:

```
type CacheKey struct {
    Operation identity.ID
    Input      identity.ID
}

type Cache interface {
    Lookup(context.Context, CacheKey) (artifact.Ref, bool, error)
    Bind(context.Context, CacheKey, artifact.Ref) error
}
```

The artifact store owns bytes; the cache owns the denotational lookup. This separation allows a cache entry to be deleted without deleting an authoritative artifact and allows several caches to point to the same content.

7.8 Provenance links

At minimum, every derived artifact should link to the exact artifacts that affected it. A representation links to a chunk, prompt, model specification, and native provider output when retained. An index links to representations and backend specifications. An answer trace links to query, authorization policy, evidence set, context, generation output, and contract result.

Database provenance semirings show how annotations can support richer explanation algebra. The initial kernel does not need a full symbolic polynomial for every RAG stage. It should, however, avoid a provenance format that prevents later enrichment. Named typed edges with stable identities are the appropriate base.

8. A compositional process model

8.1 Objects and morphisms

Let $\mathcal{C}$ be a category whose objects are versioned immutable value domains, such as `DocumentRevision`, `ChunkSet`, `RepresentationSet`, `IndexArtifact`, `Query`, `AuthorizedCandidates`, `SourceEvidenceSet`, and `GroundedAnswer`. A pure deterministic operation is a morphism $f : A \rightarrow B$.

The category laws are operational requirements:

- identity: $\text{id} ; f = f ; \text{id}$;
- associativity: $(f ; g) ; h = f ; (g ; h)$.

Associativity means that introducing or removing grouping adapters does not change the denotation, artifact identity, or combined observations, modulo explicitly declared execution scheduling.

8.2 Parallel composition

Indexing and querying contain independent work. Lexical and vector search can run in parallel. Several representation generators can run in parallel. A suite contains independent cases even if the initial evaluator runs them sequentially.

A symmetric monoidal structure adds a tensor product, written here as $\times$:

```
f : A -> B
g : C -> D
f x g : (A, C) -> (B, D)
```

The product does not imply unconstrained concurrency. It states independence at the semantic level. An execution interpreter may run the branches sequentially, in bounded parallelism, or on separate workers while preserving the same paired output order.

8.3 Effects and the limits of ordinary functions

Real stages call providers, read artifacts, consume budgets, write traces, retry, and fail. Moggi's monadic account of computation gives a principled way to model effectful results. Plotkin and Power's algebraic effects separate operations from handlers. These ideas motivate explicit effects in the plan.

An unrestricted monadic `Bind`, however, allows the next operation graph to depend on the full runtime value. That is useful at application boundaries but problematic for the auditable core. Before execution, the system needs to know:

- which resources may be consumed;
- which stages may disclose data remotely;
- which cache identities exist;
- which branches can run in parallel;
- which artifacts will be produced;
- which stage versions form the build or query plan.

If future structure is hidden in arbitrary runtime code, those questions cannot be answered statically.

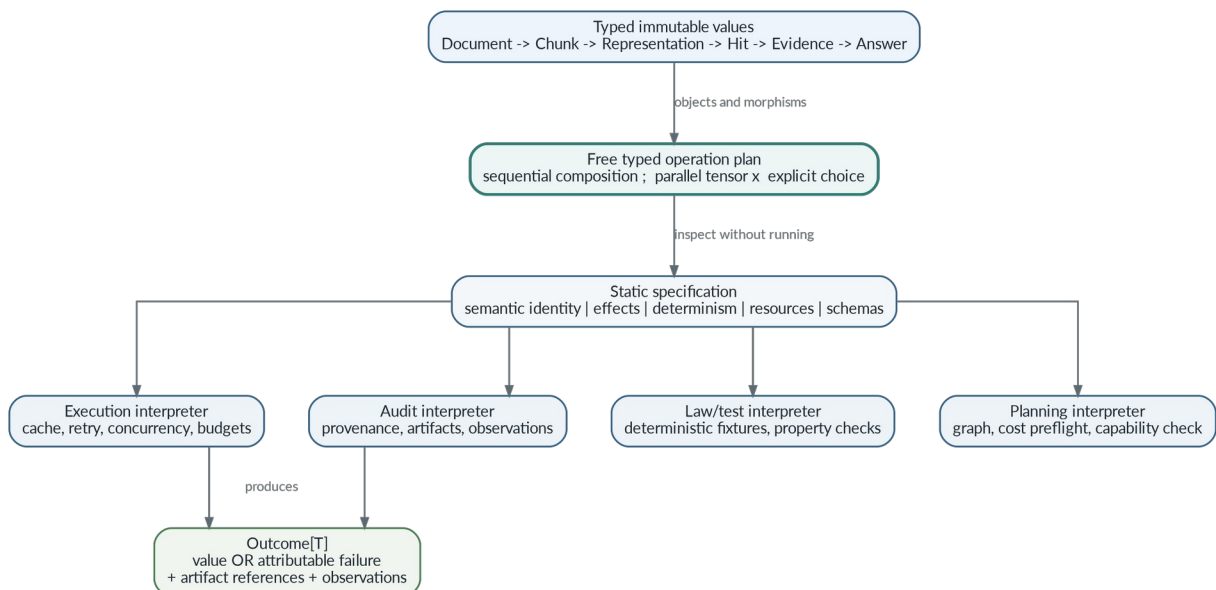
8.4 Why arrows fit the core

Hughes introduced arrows as a generalization of monads, and Paterson developed practical notation for them. The relevant property here is not Haskell syntax. It is that an arrow describes a computation from input to output while retaining more static structure than an unrestricted monadic bind.

The proposed core uses an arrow-like API with these combinators:

- `Then(f, g)` for sequential composition;
- `Fanout(f, g)` for applying two operations to one input;
- `Zip(f, g)` for independent operations on a pair;
- `Choose(f, g)` for explicit sum-type choice;
- `Map(op)` for ordered collection traversal;
- `Batch(op, policy)` for a semantically equivalent batch interpreter;
- `Lift(pure)` for deterministic pure functions.

Dynamic Bind is allowed only in an application orchestration layer and marks the plan as incompletely inspectable. It should not be the default for build and evaluation kernels.



Typed operation plans and their interpreters.

8.5 Free plans and interpreters

The operation API should build a free typed syntax rather than execute immediately. “Free” means the plan records composition without committing to one interpretation. The same plan can then be interpreted as:

- an executor with caching, retry, concurrency, and budgets;
- an identity analyzer that computes the semantic plan ID;
- a capability checker that verifies required backends and secrets;
- a trust-boundary analyzer;
- a cost and resource preflight;
- a graph renderer for admin inspection;
- a deterministic fixture interpreter for tests;
- an audit interpreter that records artifacts and observations.

This is the central compositional abstraction proposed by the thesis. It replaces multiple ad hoc pipeline descriptions without requiring a universal workflow scheduler.

8.6 Static specification and dynamic policy

Each primitive operation has a `Spec` and one or more runners.

The specification contains semantic facts:

- operation kind and version;
- input and output schemas;
- semantic configuration identity;
- declared effects;
- determinism class;
- required capabilities;
- resource units and disclosure class.

The execution policy contains operational choices:

- worker count;
- queue or local execution;
- retry and backoff;
- rate limits;
- timeout;
- cache location;
- logging verbosity.

Only facts that can affect the value belong in semantic identity. If a timeout changes the returned domain result rather than merely ending an attempt, it must be represented as a semantic policy and not hidden as execution policy. The distinction is made by behavior, not by field name.

8.7 Determinism classes

The binary labels deterministic and nondeterministic are insufficient. The plan should declare one of four classes:

1. **Pure:** same admitted input always yields the same value without effects.
2. **Deterministic effect:** effects occur, but exact referenced artifacts determine the result, such as reading an immutable object.
3. **Replayable effect:** an initial call may vary, but the exact native response is captured and later replay is deterministic.
4. **Observed nondeterministic:** repeated execution may vary and only the observed result and provider metadata are authoritative.

Caching is valid by construction for the first three when the necessary artifacts are present. The fourth may still be memoized as an experiment choice, but the cache entry represents reuse of one observation rather than a theorem that every execution would match.

8.8 Observation algebra

Usage, counters, warnings, artifacts, and trace records must combine when operations compose. The combination should form a monoid:

- an empty observation set $\emptyset$;
- an associative combine operation $+$;

- $0 + x = x = x + 0$.

Counters add. Ordered trace events concatenate in plan-defined order. Artifact references union by identity while preserving first observation order. Warnings concatenate with stable deduplication if the schema declares it.

The monoid law means that regrouping a pipeline does not change aggregate usage or trace meaning. Existing `rag.Usage.Add` is already an instance of this idea, though a shared implementation should use typed integer cost units rather than floating-point currency.

8.9 Failure as data versus interpreter failure

A provider refusal, timeout, malformed candidate answer, or failed product expectation can be an attributable outcome. A corrupt cache entry, invalid artifact digest, impossible state transition, or inability to sync a cell means the interpreter cannot guarantee custody.

The API should enforce the distinction:

```
type Runner[I, O any] func(
    context.Context,
    I,
) (outcome.Result[O], error)
```

`Result[O]` contains success, abstention, or attributable failure plus observations. The Go `error` return is reserved for cancellation and integrity failure. This mirrors the best existing `ragopt` behavior and gives `ragkit/flow` a clearer failure model.

9. Trusted kernels and proof-carrying artifacts

9.1 The trusted-kernel principle

LCF-style theorem provers achieve trust by placing a small kernel beneath a much larger untrusted tactic layer. Proof-carrying code similarly allows a producer to supply a certificate checked by a smaller consumer. The same architecture is appropriate here.

LLM calls, index builders, rerankers, chat loops, and optimization proposers are too complex and variable to prove as whole systems. They should be treated as producers of candidates, artifacts, and certificates. Small deterministic kernels decide whether those products may enter the trusted state.

The trusted computing base should contain only code that:

- canonicalizes and identifies values;
- verifies artifact bytes and links;
- validates source lineage;
- compares finite scores totally;
- applies event-ledger transitions;
- validates exact experiment pairing;
- checks answer grounding contracts;
- checks authorization-before-disclosure plan obligations.

9.2 Kernel K0: canonical identity

Input: a schema, canonical codec, and value.

Output: canonical bytes and a typed digest.

Obligations:

- repeated encoding is byte-identical;
- decode-encode round trips preserve the schema value;
- domain and schema changes change the preimage;
- prohibited values, including non-finite floats, fail closed;
- golden vectors match independent implementations.

This kernel replaces duplicated digest and canonicalization code.

9.3 Kernel K1: source lineage

Input: a document revision and a chunk certificate.

Output: an accepted `Chunk` or an error.

Obligations:

- valid byte bounds;
- exact byte-slice equality;
- correct parent identity;
- document and chunk digest verification;
- stable ordinal and chunker identity rules where required.

Generated representations can then carry a typed link to an accepted chunk without acquiring source status.

9.4 Kernel K2: total ranking

Input: ranked keys with finite scores and complete identity tie-breakers.

Output: a deterministic total order.

Obligations:

- totality: exactly one of $<$, $=$, or $>$ for any two keys;
- antisymmetry;
- transitivity;
- equality only when every order field is equal;
- stable rank numbering after filtering or fusion.

Property-based tests should generate random finite scores, identity strings, and permutations. Sorting every permutation of a small set should yield the same result.

9.5 Kernel K3: artifact verification and commit

Input: an artifact envelope and payload.

Output: a published typed reference.

Obligations:

- payload digest and size match;
- linked artifact schemas are admissible;
- a reference never resolves to partial bytes;
- duplicate publication is idempotent;
- a corrupt existing object fails closed;
- root publication occurs after all children.

The filesystem implementation can be tested under injected crashes at every write boundary.

9.6 Kernel K4: append-only ledger reducer

Input: current state and next event.

Output: next state.

Obligations:

- sequence numbers are contiguous;
- event hash links to the prior event;
- only declared transitions are accepted;
- terminal states reject mutation;
- replay from the same prefix yields the same state;
- a valid durable prefix remains recoverable after truncation of an incomplete tail.

A TLA+ model is suitable because the core difficulty is temporal: interleavings, retries, crashes, and terminality.

9.7 Kernel K5: exact pairing

Input: a suite, repeat count, arm set, and cell records.

Output: a paired matrix or a validation failure.

Obligations:

- exactly one cell for every expected coordinate;
- no duplicate coordinate;
- every cell carries the expected suite, policy, candidate, and snapshot identity;
- missing or failed cells remain visible;
- aggregate denominators equal declared expected counts.

The current `ragopt/compare` behavior is close to this kernel and should be preserved.

9.8 Kernel K6: lexicographic gate

Input: paired comparison and versioned policy.

Output: pass or fail with ordered check evidence.

Obligations:

- hard checks execute first;
- target checks cannot override hard failure;
- regression checks cannot be hidden by target mean;
- cost is advisory or a tie-break only after quality passes;
- missing metrics follow explicit policy;
- metric direction is honored.

The gate is a pure function and should have exhaustive golden tests over representative decision paths.

9.9 Kernel K7: grounding contract

Input: candidate answer and exact context evidence set.

Output: accepted grounded answer or safe abstention.

Obligations:

- every citation resolves to a context evidence identity;
- no duplicate citations;
- abstention and non-abstention invariants are disjoint;
- source evidence has already passed lineage validation;
- generated representations cannot satisfy citations unless mapped to source chunks.

`ragkit/answering` already implements most of this kernel.

9.10 Kernel K8: authorization noninterference boundary

A full noninterference proof over the entire service is beyond the initial scope. A useful kernel can still enforce a concrete property:

No operation declared as remotely disclosing source text may receive a candidate whose authorization certificate does not match the request subject and policy identity.

The certificate is produced by a trusted local authorization stage. A plan analyzer rejects any path from hydration to remote disclosure that bypasses it. Runtime validation checks the certificate before the remote adapter executes.

9.11 Proof strategy

The proof ladder should be incremental:

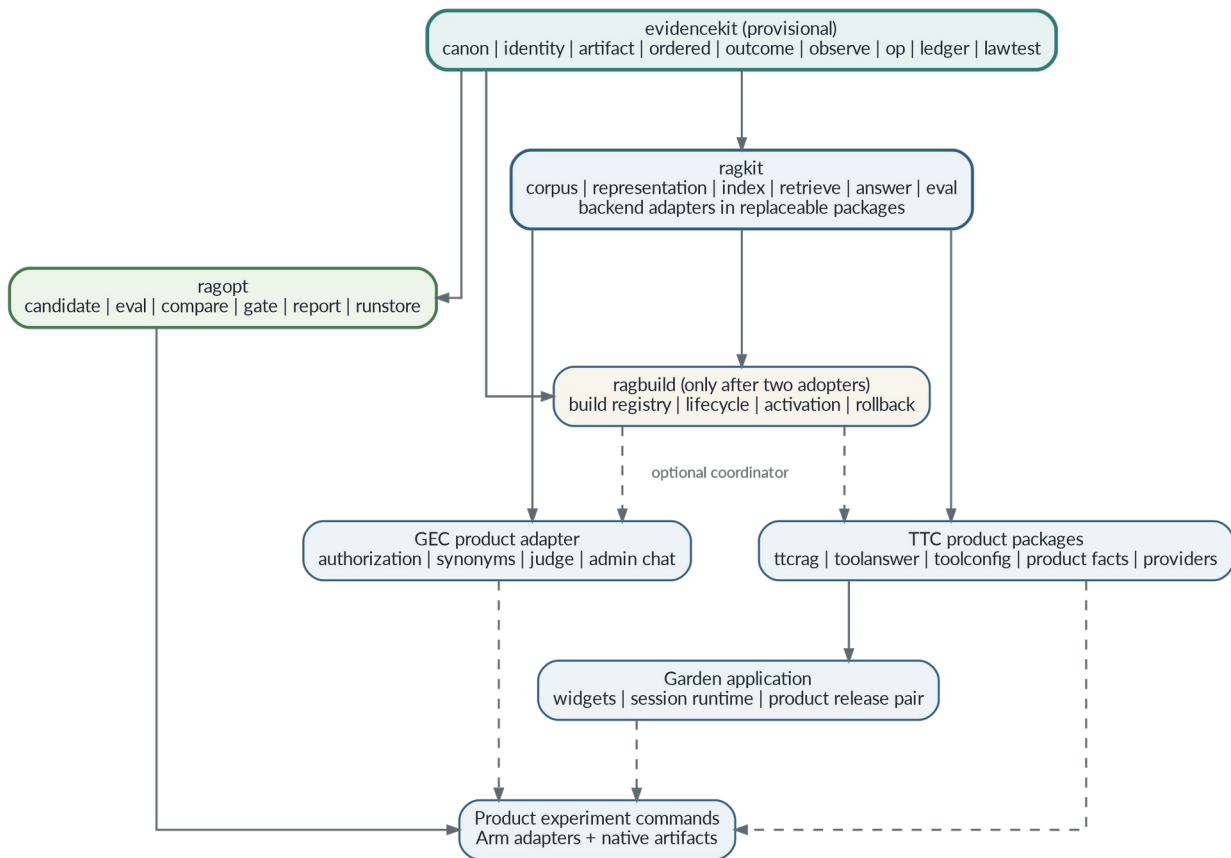
1. ordinary unit tests for examples and errors;
2. fuzz tests for decoders and path boundaries;
3. property tests for algebraic laws and comparators, following the QuickCheck tradition;
4. model checking for ledger and build state machines;
5. cross-implementation golden vectors for canonical identity;
6. optional Lean, Coq, or Isabelle proofs for the pure comparator, reducer, or codec model when maintenance value exceeds proof cost.

The system gains meaningful assurance before every component is machine-proved because the trusted kernels are small and the rest of the architecture cannot bypass them.

Part III. Proposed architecture

10. Module topology and ownership

10.1 Recommended topology



Proposed module topology. *ragbuild* is conditional on demonstrated reuse.

The architecture has four reusable layers.

evidencekit - verified evidence kernel

A small domain-neutral module containing canonical identity, immutable artifacts, total ordering, outcomes, observations, operation plans, ledgers, and law-test helpers. It has no RAG, database, provider, chat, CLI, or product dependency.

ragkit - RAG semantics and implementations

The sole owner of document lineage, chunking, representations, embeddings, index specifications and artifacts, retrieval, fusion, reranking, context, generation contracts, and IR evaluation. Heavy backends remain replaceable packages and may eventually move to separate modules if dependency weight becomes material.

ragopt - evidence-gated experiment semantics

The owner of snapshots, one-mutation candidates, suites, cells, paired evaluation, comparison, gates, reports, and experiment run custody. It depends on evidencekit, not on ragkit.

ragbuild - optional build/release coordination

A later module, introduced only after both GEC and TTC require the same build lifecycle and registry. It depends on `ragkit` and `evidencekit` and owns build records, activation, and rollback. It does not own an infrastructure scheduler, provider implementation, or optimizer.

10.2 Why not put the kernel inside ragkit?

There are three plausible options.

Option	Advantages	Costs	Decision
ragopt -> ragkit/kernel	Fewer repositories; immediate reuse	Reverses the stated independence; Go module still brings RAG dependency metadata; kernel ownership appears RAG-specific	Reject
Duplicate small helpers	No new module	Identity and custody continue to diverge; no single law suite; no deletion power	Reject
New domain-neutral kernel	Preserves dependency direction; smallest trusted base; reusable beyond RAG	One more module and release stream	Recommend

The new module is justified only if it remains small. It should not become a dumping ground for generic utilities.

10.3 Proposed package list

evidencekit/	
canon/	versioned deterministic codecs and golden vectors
identity/	domains, schemas, digests, typed references
artifact/	immutable objects, Merkle links, stores, verification
ordered/	finite scores and total comparators
outcome/	success, abstention, attributable failure
observe/	usage, trace, artifact and warning monoids
op/	typed static plans, combinators, analyzers, interpreters
ledger/	append-only events, hash chain, pure reducers
lawtest/	reusable property and conformance suites

The package boundaries are based on different laws, not on file count. `canon` and `identity` may be one package if an import cycle or API friction appears. `lawtest` should remain separate so production binaries do not import test generators.

10.4 Proposed ragkit public surface

ragkit/	
corpus/	document revisions, chunks, spans, lineage
representation/	retrieval material and generation specifications
index/	index plan, artifact, open/verify contracts
retrieve/	channels, collapse, fusion, hydration, filters
answer/	context, generation, citations, grounding contract
eval/	deterministic IR metrics and labeled sets
inspect/	plan descriptions and stage observations
backend/bleve/	
backend/sqliteexact/	
adapter/...	optional provider adapters in separate dependency zones

A compatibility `rag` package can re-export existing types during migration, but new code should use semantically named packages. Re-exports must have a deletion date and should not create a second implementation.

10.5 Proposed ragopt public surface

ragopt/	
candidate/	retain current responsibility; use typed artifact refs

eval/	retain Arm boundary and exact cell coordinates
compare/	retain strict pairing; add metric direction metadata
gate/	retain ordered phases; validate finite values
report/	retain human-reviewable and machine-readable outputs
runstore/	implement experiment reducer over evidencekit ledger/store
experimental/campaign/	absent until two product proofs justify it

The existing package structure is already close to the target. Most change is infrastructural hardening, not conceptual redesign.

10.6 Product packages

TTC-specific code should live outside `ragkit` in packages such as:

```
ttcrag/search
ttcrag/toolanswer
ttcrag/toolconfig
ttcrag/productcatalog
ttcrag/connected
ttcrag/providers
```

GEC keeps its authorization, source-role semantics, curated synonyms, judge, SQL/tool policy, admin chat, and UI projections. Garden keeps widgets, choice semantics, product-fact presentation, and session calibration expectations.

10.7 Import rules

The following rules should be executable boundary tests:

- `evidencekit` imports only the standard library and narrowly justified codec/test dependencies.
- `ragkit` may import `evidencekit`; `evidencekit` never imports `ragkit`.
- `ragopt` may import `evidencekit`; it never imports `ragkit` or a product.
- product runtime packages may import `ragkit` and product adapters, but not `ragopt` unless they are experiment commands.
- UI/session packages never appear below `ragkit`, `ragopt`, or `evidencekit`.
- provider frameworks appear only in adapter packages.
- Garden does not import the RAG-TTC application module for common RAG semantics after cutover.

These rules prevent the next extraction from recreating the current problem.

11. The evidencekit API

11.1 Design constraints

The kernel API should satisfy six constraints.

1. It must be usable by `ragkit` and `ragopt` without either importing the other.
2. Identity-critical values must have explicit schemas and codecs.
3. Invalid semantic states should be difficult to construct.
4. Plans must be inspectable before execution.
5. Product-native payloads must remain opaque but content-addressed.
6. The implementation must be small enough for exhaustive law tests and code review.

The following APIs are design sketches. Names may change, but the semantic separations should not.

11.2 Identity

```
package identity

type Domain string

type Schema struct {
    Domain Domain `json:"domain"`
    Name    string  `json:"name"`
    Version uint32 `json:"version"`
}

type Digest [32]byte

type ID struct {
    Schema Schema `json:"schema"`
    Digest Digest `json:"digest"`
}

// Ref is a typed reference. T is phantom at runtime but prevents accidental
// cross-domain wiring in Go code.
type Ref[T any] struct {
    ID          ID      `json:"id"`
    SizeBytes   int64   `json:"size_bytes"`
    MediaType   string  `json:"media_type"`
}
```

The `Schema` belongs to the preimage and to the serialized reference. `Digest` should implement strict text and JSON encoding, including an algorithm prefix at external boundaries, for example `sha256:...`. The kernel should reject unknown algorithms rather than silently treating strings as opaque.

Identification accepts canonical bytes rather than arbitrary values. This keeps codec policy explicit:

```
func FromCanonical(
    schema Schema,
    canonical []byte,
) ID
```

Convenience functions can combine a schema and typed codec:

```
package canon

type Codec[T any] interface {
    Marshal(T) ([]byte, error)
    Unmarshal([]byte) (T, error)
}

func Identify[T any](
    schema identity.Schema,
    codec Codec[T],
    value T,
) (identity.ID, []byte, error)
```

11.3 Artifact store

```
package artifact

type Link struct {
    Role string    `json:"role"`
    ID    identity.ID `json:"id"`
}

type Object struct {
    Schema    identity.Schema `json:"schema"`
    MediaType string          `json:"media_type"`
    Payload   []byte          `json:"- "`
    Links     []Link          `json:"links,omitempty"`
}

type Metadata struct {
    ID        identity.ID `json:"id"`
    SizeBytes int64      `json:"size_bytes"`
    MediaType string    `json:"media_type"`
    Links     []Link    `json:"links,omitempty"`
}

type Store interface {
    Put(context.Context, Object) (Metadata, error)
    Open(context.Context, identity.ID) (io.ReadCloser, Metadata, error)
    Stat(context.Context, identity.ID) (Metadata, error)
    Verify(context.Context, identity.ID) error
}
```

Put computes identity from schema and payload, verifies links according to policy, and publishes atomically. An FSStore is sufficient initially. Cloud/object-store adapters should preserve the same commit semantics and should not be part of the kernel module until required.

Typed helpers wrap metadata in `identity.Ref[T]` at domain boundaries. The untyped store does not need reflection or a registry of every domain type.

11.4 Finite scores and total order

```
package ordered

type Score struct{ value float64 }

func NewScore(v float64) (Score, error)
func (s Score) Float64() float64

type RetrievalKey struct {
    Score           Score
    DocumentID      string
    ChunkID         string
    RepresentationID string
}

// Compare returns -1 when left ranks before right, 0 when equal, and +1 after.
func CompareRetrieval(left, right RetrievalKey) int
```

The unexported score field prevents direct construction of NaN or infinity. Similar types can represent metric values and normalized costs. A generic comparator package is unnecessary; the kernel should provide only comparators whose laws and semantics are stable.

11.5 Outcome as a sum type

Go does not have native algebraic data types, but unexported fields and constructors can enforce a valid state space.

```
package outcome

type Kind uint8

const (
    KindSuccess Kind = iota + 1
```



```

    KindAbstained
    KindFailure
)

type Failure struct {
    Class      string      `json:"class"`
    Message    string      `json:"message"`
    Retryable  bool        `json:"retryable,omitempty"`
    Detail     json.RawMessage `json:"detail,omitempty"`
}

type Result[T any] struct {
    kind Kind
    value T
    fail Failure
    obs  observe.Set
}

func Success[T any](value T, obs observe.Set) Result[T]
func Abstained[T any](value T, obs observe.Set) Result[T]
func Failed[T any](failure Failure, obs observe.Set) Result[T]

func (r Result[T]) Kind() Kind
func (r Result[T]) Value() (T, bool)
func (r Result[T]) Failure() (Failure, bool)
func (r Result[T]) Observations() observe.Set

```

Custom JSON marshaling serializes a tagged union. The constructor set makes contradictory booleans impossible. Domain packages can wrap this with stronger contracts, such as a grounded answer in which abstention has an empty citation set.

11.6 Observations and usage

```

package observe

type Counter struct {
    Name string `json:"name"`
    Unit string `json:"unit"`
    Value int64 `json:"value"`
}

type Money struct {
    Currency string `json:"currency"`
    Micros int64 `json:"micros"`
}

type Event struct {
    Kind      string      `json:"kind"`
    Schema    identity.Schema `json:"schema"`
    Payload   json.RawMessage `json:"payload,omitempty"`
    ArtifactID *identity.ID   `json:"artifact_id,omitempty"`
}

type Set struct {
    Counters []Counter      `json:"counters,omitempty"`
    Costs    []Money         `json:"costs,omitempty"`
    Events   []Event         `json:"events,omitempty"`
    Artifacts []identity.ID   `json:"artifacts,omitempty"`
    Warnings []string        `json:"warnings,omitempty"`
}

func Empty() Set
func Combine(left, right Set) Set

```

Combine is deterministic and associative. Counters are normalized by (name, unit), costs by currency, and artifact references by identity. Events preserve sequence. Floating-point dollars are excluded from the shared kernel.

11.7 Operation specifications

```

package op

type Determinism uint8

```

```

const (
    Pure Determinism = iota + 1
    DeterministicEffect
    ReplayableEffect
    ObservedNondeterministic
)

type Effect string

const (
    ReadArtifact      Effect = "read_artifact"
    WriteArtifact     Effect = "write_artifact"
    Network           Effect = "network"
    ModelCall         Effect = "model_call"
    RemoteDisclosure  Effect = "remote_disclosure"
)

type Resource struct {
    Name      string `json:"name"`
    Unit      string `json:"unit"`
    Ceiling   int64  `json:"ceiling"`
}

type Spec struct {
    ID          identity.ID      `json:"id"`
    Name        string        `json:"name"`
    InputSchema identity.Schema `json:"input_schema"`
    OutputSchema identity.Schema `json:"output_schema"`
    Determinism Determinism    `json:"determinism"`
    Effects     []Effect      `json:"effects,omitempty"`
    Resources   []Resource    `json:"resources,omitempty"`
    Capabilities []string     `json:"capabilities,omitempty"`
    Disclosure  string        `json:"disclosure,omitempty"`
}

```

`Spec.ID` identifies the primitive operation semantics, including its versioned configuration. Input identity is combined with `Spec.ID` by the cache interpreter.

11.8 Typed plan values

A practical Go implementation can hide an erased plan node behind typed wrappers:

```

package op

type Plan[I, O any] struct {
    node *node // unexported typed-erased syntax
}

func Primitive[I, O any](
    spec Spec,
    run Runner[I, O],
) Plan[I, O]

func Lift[I, O any](
    name string,
    pure func(I) (O, error),
) Plan[I, O]

func Then[A, B, C any](
    first Plan[A, B],
    second Plan[B, C],
) Plan[A, C]

func Fanout[A, B, C any](
    left Plan[A, B],
    right Plan[A, C],
) Plan[A, Pair[B, C]]

func Zip[A, B, C, D any](
    left Plan[A, B],
    right Plan[C, D],
) Plan[Pair[A, C], Pair[B, D]]

func Map[A, B any](item Plan[A, B]) Plan[[]A, []B]

```

Go's generic type checker verifies the wires. The private node records enough structure for analyzers. `Primitive` validates that the runtime types match the declared schemas through domain-level registration or explicit constructors; the kernel does not rely on `reflect.Type` as semantic identity.

11.9 Interpreters

```
type ExecPolicy struct {
    Workers int
    Retry    RetryPolicy
    Timeout time.Duration
}

type Executor interface {
    Run[I, O any](
        context.Context,
        Plan[I, O],
        I,
        ExecPolicy,
    ) (outcome.Result[O], error)
}
```

Go interfaces cannot currently declare independently generic methods in all desired forms without constraints, so a real implementation may use a generic free function over a non-generic executor core. The semantic point is that execution is an interpretation of the plan, not the definition of the plan.

Analyzers traverse the same node:

```
func Describe[I, O any](Plan[I, O]) Description
func SemanticID[I, O any](Plan[I, O]) (identity.ID, error)
func RequiredResources[I, O any](Plan[I, O]) []Resource
func CheckDisclosure[I, O any](Plan[I, O], Policy) error
```

11.10 Ledger

```
package ledger

type Event struct {
    LedgerID    string      `json:"ledger_id"`
    Sequence    uint64      `json:"sequence"`
    Kind        string      `json:"kind"`
    PayloadID   identity.ID `json:"payload_id"`
    Previous    identity.Digest `json:"previous"`
    RecordHash  identity.Digest `json:"record_hash"`
    RecordedAt  time.Time    `json:"recorded_at"`
}

type Reducer[S any] interface {
    Initial() S
    Apply(S, Event) (S, error)
}
```

The event's record hash commits to sequence, kind, payload ID, previous hash, and optionally the recorded timestamp. The semantic state reducer should depend only on event kind and payload, not wall-clock time unless time is explicitly part of the domain transition.

An active run uses a single writer. If multi-process writers become necessary, locking or compare-and-swap belongs in the store adapter rather than in the reducer.

11.11 Law-test package

`lawtest` provides reusable suites:

```
func CanonicalCodec[T comparable](t *testing.T, codec canon.Codec[T], values []T)
func Monoid[T any](t *testing.T, zero T, combine func(T, T) T, gen Generator[T])
func TotalOrder[T any](t *testing.T, compare func(T, T) int, gen Generator[T])
func PlanCategory(t *testing.T, primitives FixturePrimitives)
func ArtifactStore(t *testing.T, newStore func(t *testing.T) artifact.Store)
func LedgerReducer[S any](t *testing.T, reducer ledger.Reducer[S], traces []Trace)
```

12. The ragkit domain API

12.1 Preserve the domain distinctions

The next `ragkit` API should preserve the best current distinctions while removing ambiguous names and infrastructure coupling:

- a document is one immutable source revision;
- a chunk is an exact source span with a lineage certificate;
- a representation is retrieval material derived from a chunk;
- a hit is a channel-local observation;
- a fused candidate is a rank-combination result;
- source evidence is a hydrated, authorized chunk;
- an answer is accepted only through a contract validator.

12.2 Corpus types

```
package corpus

type DocumentRevision struct {
    ID          identity.ID
    Logical     string           // stable logical document name
    URI         string
    Title       string
    Text        string
    Metadata   map[string]string // not identity-critical unless schema says so
}

type Span struct {
    ByteStart int
    ByteEnd   int
}

type Chunk struct {
    ID          identity.ID
    Document    identity.Ref[DocumentRevision]
    Ordinal     int
    Span        Span
    Text        string
    Chunker     identity.ID
}

type LineageCertificate struct {
    DocumentID identity.ID
    Span        Span
    TextDigest  identity.Digest
}

func VerifyChunk(
    document DocumentRevision,
    chunk Chunk,
) (LineageCertificate, error)
```

A logical document name can remain stable across revisions, while the material ID changes with text. This avoids using a mutable string ID as both logical identity and content identity.

12.3 Representation types

```
package representation

type Kind string

type Material struct {
    ID          identity.ID
    Chunk       identity.Ref[corpus.Chunk]
    Kind        Kind
}
```

```

    Text      string
    Producer  identity.ID // prompt/model/algorithm specification
    Native    *identity.ID
}

```

The Producer commits to all semantic inputs. A raw representation uses a pure producer specification. A generated summary can link to a native provider response artifact. The representation's ID is calculated from exact text and links; it is not inferred from the producer plan alone.

12.4 Index specifications

```

package index

type ChannelSpec struct {
    Name      string
    Backend   identity.ID
    SourceKind []representation.Kind
    Parameters json.RawMessage
}

type VectorSpec struct {
    Channel      string
    Provider     identity.ID
    Model        string
    Dimensions   int
    Metric       string
}

type Spec struct {
    Schema        identity.Schema
    Corpus        identity.Ref[corpus.Corporus]
    Chunker       identity.ID
    Representations []identity.Ref[representation.Material]
    Channels      []ChannelSpec
    Vector        *VectorSpec
}

type Artifact struct {
    Ref      identity.Ref[Artifact]
    Manifest Manifest
}

```

Channel order in the specification is either semantically ordered or canonicalized by name. The schema must say which. Backend versions, tokenizers, normalizers, title/body boosts, vector metric, dimensions, and representation selection enter semantic identity.

12.5 Build planning

```

type BuildInput struct {
    Spec index.Spec
}

type BuildOutput struct {
    Artifact identity.Ref[index.Artifact]
    Report   BuildReport
}

func BuildPlan(
    backends BackendRegistry,
) op.Plan[BuildInput, BuildOutput]

```

The plan expands into snapshot verification, chunk verification, representation generation or reuse, embedding, backend assembly, manifest construction, artifact verification, and publication. The exact structure remains statically visible for preflight and admin inspection.

The current `indexbundle.Build` can be wrapped as one primitive during migration. It does not need to be rewritten before the plan API proves useful.

12.6 Retrieval plan

```
package retrieve

type Query struct {
    ID    identity.ID
    Text  string
}

type Subject struct {
    Principal string
    Scopes    []string
    Roles     []string
}

type Request struct {
    Query    Query
    Subject  Subject
    Limit    int
}

type Channel struct {
    Name      string
    Searcher  Searcher
    Depth     int
    Weight    float64
}

type Plan struct {
    Index          identity.Ref[index.Artifact]
    Authorization  AuthorizationPlan
    Channels       []Channel
    Collapse       CollapsePlan
    Fusion         FusionPlan
    Rerank         *RerankPlan
    Hydration      HydrationPlan
}
```

The domain-level plan is a configuration value. `Compile(plan)` returns an `op.Plan[Request, Result]`. This separates serializable policy from executable adapters.

12.7 Searcher capabilities

The current `Searcher.Search(query, topK)` is intentionally narrow, but authorization and representation filters need capabilities. Avoid expanding one interface with many optional parameters. Use capability-specific interfaces:

```
type Searcher interface {
    Search(context.Context, Query, int) ([]Hit, error)
}

type FilteredSearcher interface {
    SearchFiltered(context.Context, Query, Filter, int) ([]Hit, error)
}

type ExhaustiveSearcher interface {
    MaximumDepth() int
}
```

The compiler selects the strongest available strategy:

1. backend prefilter;
2. authorized partition selection;
3. exhaustive local search then filter;
4. bounded overfetch with an explicit starvation warning.

No remote stage executes before authorization is established.

12.8 Retrieval result and trace

```

type SourceEvidence struct {
    Chunk      corpus.Chunk
    Rank       int
    Retrieval   ordered.Score
    Rerank      *ordered.Score
    Certificate AuthorizationCertificate
}

type Result struct {
    Query      Query
    Channels    []ChannelResult
    Fused      []FusedCandidate
    Evidence    []SourceEvidence
    Derivation identity.Ref[QueryTrace]
}

```

Scores are observations and are excluded from source evidence identity. The ordered evidence identity contains chunk IDs and content digests. The trace links score observations, routes, query variants, fusion contributions, filters, reranker results, and artifact references.

12.9 Answer plan

```

package answer

type Plan struct {
    Retrieval   retrieve.Plan
    Context     ContextPlan
    Generator    GeneratorPlan
    Contract    ContractPlan
    MaxEvidence int
    MaxContextByte int
}

type Request struct {
    Query   retrieve.Query
    Subject retrieve.Subject
}

type Grounded struct {
    Text      string
    Citations []identity.Ref[corpus.Chunk]
    Abstained bool
}

type Result struct {
    Answer      Grounded
    Evidence    []retrieve.SourceEvidence
    Contract    ContractResult
    Derivation  identity.Ref[AnswerTrace]
}

```

The accepted `Grounded` value is constructed only by the contract kernel. Raw model output is a different type, such as `CandidateAnswer`.

12.10 Metric definitions

`ragkit/eval` should continue to own precision, recall, hit rate, MRR, and nDCG. It should introduce typed metric metadata:

```

type Direction uint8
const (
    HigherIsBetter Direction = iota + 1
    LowerIsBetter
)

type MetricSpec struct {
    Name      string
    Unit      string
    Direction Direction
}

```

```
| Range      *Interval  
| }
```

This metadata can be projected into `ragopt` policies without making `ragopt` import RAG evaluation code.

12.11 Provider adapters

Provider interfaces are stable only at the semantic operation boundary. An embedding adapter must report model identity, dimensions, native usage, and response artifact. A generation adapter must report the exact request, native response, finish reason, and usage. Rerankers must receive only authorized evidence.

Adapters may live in separate modules or product repositories because provider frameworks carry large dependency trees and change faster than the RAG contracts. The existing `ragkit` boundary test should be retained and generalized.

13. The ragopt API after kernel adoption

13.1 Preserve the current product boundary

The most important decision is to keep `Arm` small and product-owned. The following refinement adds typed identities and a valid outcome state without making an arm understand `ragkit`.

```
type Arm interface {
    Name() string
    Run(
        context.Context,
        Request,
    ) (outcome.Result[Projection], error)
}

type Projection struct {
    Metrics    []MetricValue
    Counters   []observe.Counter
    Duration   time.Duration
    Native     identity.Ref[NativeArtifact]
}
```

The product writes its native artifact first and returns a typed reference. `ragopt` verifies that the artifact belongs to the assigned cell root or artifact namespace.

13.2 Metric schema

```
type MetricDirection uint8
const (
    HigherBetter MetricDirection = iota + 1
    LowerBetter
)

type MetricDefinition struct {
    Name      string
    Unit      string
    Direction MetricDirection
    MissingPolicy string
}

type MetricValue struct {
    Name string
    Value ordered.Score // finite numeric wrapper; name may be "metric value"
}
```

The numeric wrapper can be a separate `Finite` type rather than retrieval `Score`; the principle is rejection of `NaN` and infinity. The suite or policy binds definitions by name. Gate code normalizes deltas according to direction so positive normalized delta always means improvement.

13.3 Candidate assets as artifact references

Current candidates identify files by path, SHA-256, media type, and size. After kernel adoption, paths become bundle-local transport metadata and typed artifact references become semantic identity:

```
type AssetRef struct {
    Name      string
    Artifact  identity.Ref[Asset]
    LocalPath string // copied/materialized path, excluded from identity
}
```

The exactly-one-mutation validator compares artifact identities and media types, not merely file metadata. Locked dimensions become a canonical versioned value artifact.

13.4 Experiment plan identity

A run plan includes:

- candidate and parent snapshot references;
- suite reference;
- policy reference;
- arm implementation identities;
- repeat count;
- metric definitions;
- declared nondeterminism/replay policy.

It excludes run root, start time, host, worker count, and logging. The plan receives a semantic ID before a run begins. Each run attempt has its own run ID and ledger.

13.5 Cell coordinates

The exact cell key remains:

```
| (plan ID, case ID, repeat index, arm role, snapshot ID)
```

A cell payload includes start/finish observations, outcome, native artifact, and execution metadata. The payload is immutable. Resume checks the ledger for an accepted cell with the exact coordinate and artifact verification; it never resumes by filename convention alone.

13.6 Runstore as a reducer

ragopt/runstore defines experiment-specific events:

```
RunCreated
InputBound
CellStarted
CellCompleted
CellFailed
ComparisonWritten
GateDecided
ReportWritten
RunCompleted
RunFailed
```

An experiment reducer enforces transitions. The shared ledger provides sequence, hash chaining, durability, and replay. `RunCompleted` is accepted only when every expected coordinate has a terminal cell and required reports verify.

This structure makes `status.json` a projection, not the sole authority. It can be regenerated from the ledger.

13.7 Comparison

Comparison remains exact and paired. For each metric with direction *d*, define normalized improvement:

```
improvement = candidate - incumbent    when higher is better
improvement = incumbent - candidate    when lower is better
```

Group aggregates must report:

- expected pairs;
- pairs present;
- attributable failures by arm and class;
- mean, median, minimum, and maximum improvement;

- repeat-level aggregates;
- optional confidence interval;
- cost and latency deltas as separately typed values.

No statistic should omit failed pairs without an explicit policy that also reports the changed denominator.

13.8 Gate policy

A gate policy is a canonical artifact with ordered phases:

```
type Policy struct {
    APIVersion string
    Name       string
    Hard       HardChecks
    Target     TargetChecks
    Regressions RegressionChecks
    TieBreakers []TieBreaker
}
```

A decision records every check in execution order. The policy should support:

- complete pairing;
- completion and contract-valid floors;
- metric presence;
- absolute candidate floors;
- minimum target improvement by group;
- maximum per-case and group regression;
- repeat consistency;
- failure-class ceilings;
- lower-cost tie-breaks after quality.

Statistical tests may be added as checks, but a small p-value should not replace effect size, case-level visibility, or hard safety invariants.

13.9 Campaigns are a later meta-layer

A future campaign package may define:

```
type Proposer interface {
    Propose(context.Context, Diagnostic, Snapshot) ([]CandidateDraft, error)
}

type Selector interface {
    Select(Frontier, Budget) ([]CandidateDraft, error)
}
```

It should treat `ragopt` evaluation as an oracle over immutable candidates. It must not mutate the evaluator, judge, suite, safety policy, or gate inside the same campaign. Search algorithms, reflectors, and population management remain outside the trusted experiment kernel.

14. Indexing and build coordination

14.1 Indexing is a deterministic plan around possibly nondeterministic producers

A typical build is:

```
snapshot sources
-> normalize/extract
-> chunk
-> derive representations
-> embed
-> assemble lexical/vector backends
-> verify
-> evaluate
-> await activation
```

Every arrow except provider-backed generation and embedding can usually be pure or deterministic over immutable artifacts. Provider outputs become replayable when exact native responses and resulting bytes are stored.

14.2 Build specification

A build specification must include every semantic input:

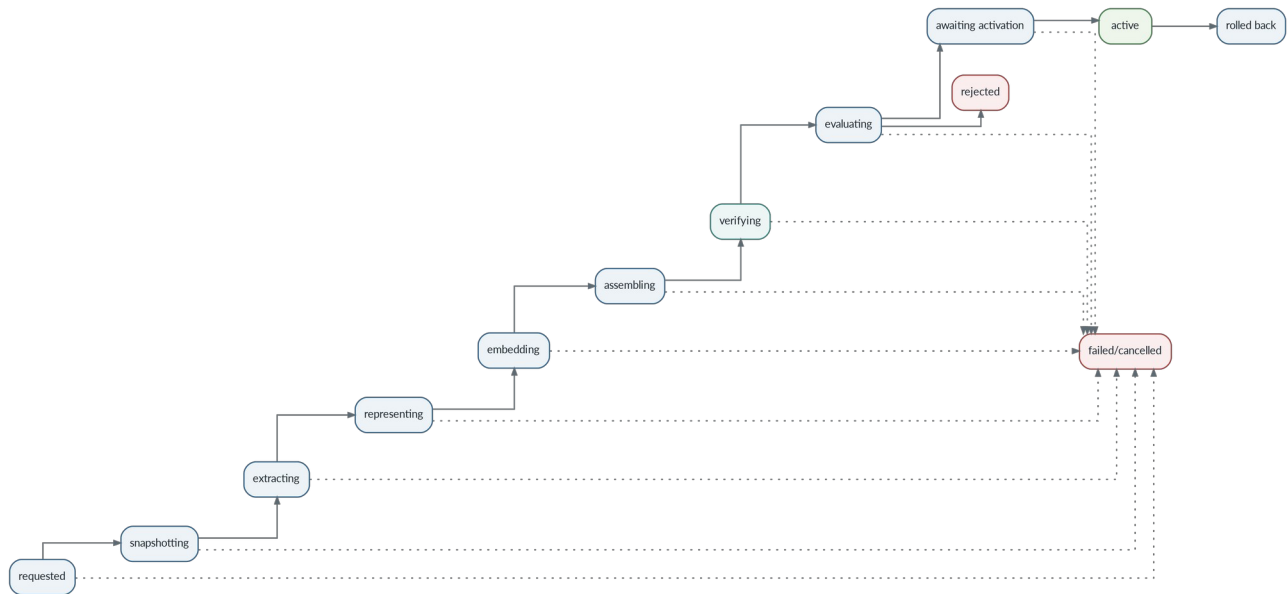
- source snapshot and corpus schema;
- document ordering rule;
- normalization and extraction versions;
- chunker kind and parameters;
- representation kinds, prompts, models, and batching semantics;
- embedding provider, model, dimensions, normalization, and metric;
- lexical backend, tokenizer, fields, boosts, and version;
- vector backend, distance metric, quantization, and version;
- index manifest schema;
- validation and evaluation policy references.

It excludes:

- output directory;
- worker count;
- queue/job ID;
- retry count and backoff;
- hostname;
- timestamps;
- log destination.

The plan ID identifies intent. Produced artifacts identify exact results.

14.3 Build state machine



Recommended build and activation state machine.

The state machine from the RAGOPT build design is appropriate:

```
requested -> snapshotting -> extracting -> representing -> embedding
           -> assembling -> verifying -> evaluating
           -> rejected | awaiting_activation -> activating -> active
active     -> rolled_back
any nonterminal state -> failed | cancelled
```

The figure elides activating for space; the reducer should include it. Each transition is an event with artifact references. `verifying` accepts only an artifact whose children and manifest pass the kernel. `evaluating` links a ragopt run or product-native evaluation. `active` identifies a release generation, not a mutation of the index artifact.

14.4 Inner executor versus outer scheduler

The system needs two levels of retry and resume:

- the **inner executor** replays deterministic or cached operations inside one build attempt;
- the **outer scheduler** retries or resumes a coarse build job after process, machine, or queue failure.

ragkit/flow or its op interpreter is the inner executor. It should not become a distributed scheduler. A product job runner or later ragbuild coordinator owns the outer lifecycle.

The recommended job granularity is one coarse build per artifact or release, not one queue job per representation or embedding. Fine-grained distributed scheduling multiplies custody states before evidence shows it is necessary.

14.5 Build registry

A build registry records:

- build plan ID;
- execution run IDs;
- current state and state history;
- input and output artifact references;

- linked evaluation run;
- rejection reasons;
- activation generation;
- rollback target;
- operator or automation principal.

It is separate from `ragopt` runstore. The two systems link by artifact and run IDs. An optimization run can compare two index artifacts without owning their build lifecycle.

14.6 Full release manifests

A production assistant rarely depends on only an index. Garden also uses product facts and widget metadata. GEC may use source-role policy, curated synonyms, a reranker specification, or tool descriptions. A release manifest should identify the exact compatible set:

```
type Release struct {
  Schema      identity.Schema
  Index        identity.Ref[index.Artifact]
  ProductFacts *identity.Ref[ProductFacts]
  QueryPlan    identity.Ref[retrieve.Plan]
  AnswerPlan   identity.Ref[answer.Plan]
  Policy       identity.Ref[AuthorizationPolicy]
}
```

Activation changes the pointer from one immutable release to another. Rollback selects a prior release; it does not rewrite artifacts.

14.7 Reproducibility under provider nondeterminism

Two notions of reproduction must be reported separately:

- **plan reproduction:** execute the same semantic plan again;
- **artifact replay:** serve or evaluate the exact previously produced artifacts.

Only the second promises byte-exact behavior. The first may generate new representations or vectors unless provider responses are replayed from captured native artifacts. Build reports should state the determinism class of every stage and whether the final artifact is independently reproducible from retained inputs.

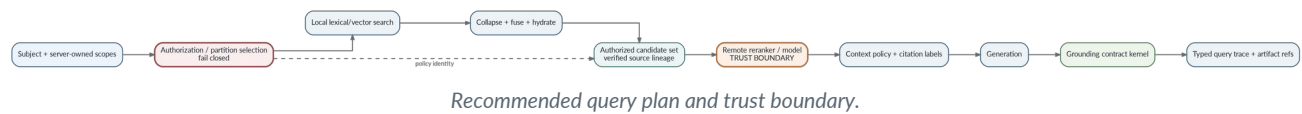
15. Querying, answering, and administration

15.1 Querying is a subject-bound computation

A query is not only text. It includes the requesting subject, server-owned scopes and roles, selected release, route policy, and limits. Model-generated fields must never expand authority. The query semantic identity includes the normalized query text, release, route, authorization-policy identity, and any deterministic augmentation that affects retrieval.

Interactive request IDs and timestamps are execution metadata.

15.2 Recommended stage order



The stage order is:

1. resolve the active immutable release;
2. derive authorization constraints from the authenticated subject;
3. select authorized index partitions or backend filters;
4. execute lexical and vector channels;
5. collapse and fuse candidates;
6. hydrate and verify source lineage;
7. establish the authorized evidence certificate;
8. cross the remote boundary for reranking, if configured;
9. assemble context under byte/token and diversity policies;
10. call generation;
11. validate the grounding contract;
12. publish the query trace and session evidence view.

Synonym expansion, multi-query, HyDE, product-fact augmentation, and route selection fit as explicit operations with their own identities and disclosure classes.

15.3 Authorization strategies

The compiler chooses among four strategies.

Partitioned index

Each scope or tenant has an immutable partition, and the subject selects a set of partitions. This offers the strongest isolation and avoids rank starvation, at the cost of more artifacts and fusion complexity.

Backend prefilter

The index backend accepts an authorization predicate or filter key and guarantees that ineligible representations do not enter the candidate set.

Exhaustive local filter

For small exact indexes, search all candidates locally, filter, then truncate. This is deterministic and safe before remote disclosure, though it may be expensive.

Bounded overfetch

Search limit * factor, filter locally, and report possible starvation. This is the current GEC approach. It should be represented as an explicit degraded capability so evaluation and admin inspection can see it.

15.4 Reranker failure and degradation

GEC's fallback to fused rank on reranker failure is a legitimate product policy. The shared plan should make it explicit:

```
type FailurePolicy uint8
const (
    FailQuery FailurePolicy = iota + 1
    UsePreviousRanking
    Abstain
)
```

The fallback is part of semantic query behavior and therefore belongs in the plan identity. Retry count and backoff remain execution policy unless they change the externally visible failure semantics.

15.5 Context as a verified artifact

Context assembly should produce an artifact containing:

- ordered source-evidence identities;
- exact rendered context bytes;
- citation labels and their chunk mapping;
- truncation, diversity, and deduplication decisions;
- policy identity;
- token or byte counts.

The generation request links to this artifact. The answer contract validates citations against it. This makes “what the model saw” an exact replayable object rather than a reconstructed log string.

15.6 Query trace schema

A shared trace should be stage-oriented and artifact-linked, not a universal chat transcript:

```
type StageRecord struct {
    Sequence    int
    Stage       string
    SpecID      identity.ID
    InputIDs    []identity.ID
    OutputIDs   []identity.ID
    Outcome     string
    Observations observe.Set
}

type QueryTrace struct {
    RequestID string // execution identity
    PlanID    identity.ID
    Records   []StageRecord
}
```

Sensitive payloads can be omitted or stored in access-controlled artifacts. The trace remains useful because it preserves identities, outcomes, and summaries.

15.7 Admin inspection API

A common inspect package can expose:

```
type CapabilityReport struct {
    ReleaseID    identity.ID
    Channels     []string
    SupportsPrefilter bool
}
```



```

    Reranker      *identity.ID
    DegradationModes []string
}

type ExplainResult struct {
    Plan      Description
    Trace     *QueryTrace
    Evidence  []EvidenceView
    Artifacts []identity.ID
}

```

GEC can render this through its admin chat. RAG-TTC can render it through its work-in-progress admin chat. Garden can map it to widgets. The common package does not own sessions or UI components.

15.8 Session evidence ledgers

Per-session evidence should remain separate from the immutable service. A bounded ledger may retain:

- query and answer IDs;
- cited source-evidence references;
- display metadata safe for the current subject;
- tool-call and route summaries;
- trace artifact references.

The ledger is subject-scoped. Reusing a session after authorization changes should either revalidate every entry or start a new ledger generation. A shared ledger interface may emerge later, but current product display semantics differ enough that only the event and artifact primitives should be shared now.

16. Optimization as a meta-level search

16.1 Configuration graph

Let cfg be a graph whose nodes are immutable system snapshots and whose directed edges are valid exactly-one-asset mutations. A mutation edge has a hypothesis, expected target, and regression risks. A sequence of edges represents a multi-step optimization history without weakening attribution within one experiment.

Under suitable identity edges, cfg can be treated as a category: identity is the no-change snapshot, and composition is a sequence of valid mutations. In practice, composed edges should retain the intermediate snapshots rather than pretending several changes were one atomic mutation.

16.2 Evaluation mapping

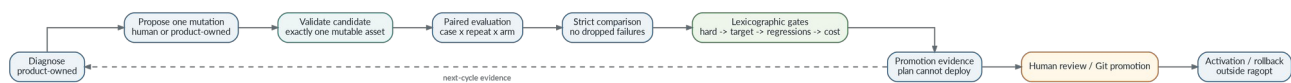
For a fixed suite S , policy P , and execution protocol, evaluation maps a snapshot to an ordered vector of case-repeat outcomes:

```
Eval(Snapshot) = [Outcome(case_1, repeat_1), ...]
```

Because providers may be nondeterministic, the codomain is better understood as observed traces or distributions rather than a pure metric vector. Explicit repeats and captured native artifacts make the observations comparable.

The paired comparison maps a mutation edge ($parent \rightarrow child$) to pointwise deltas over the shared coordinates. This is why unpaired sweeps are insufficient for promotion evidence.

16.3 The loop



Evidence-gated optimization loop.

The loop has three ownership zones:

- product or human: diagnose and propose;
- ragot: validate, evaluate, compare, gate, and render evidence;
- human/release system: promote, activate, or reject.

This separation is a safety property. The component that proposes a change does not get to redefine the evaluator or gate in the same run. The component that reports a pass cannot deploy it.

16.4 Diagnostics as artifact inputs

A candidate can link to diagnostic artifacts such as:

- low-recall query sets;
- failure packets;
- retrieval traces;
- judge disagreements;
- latency or cost outliers;
- source-role starvation reports;
- calibration expectation failures.

`ragopt` does not interpret the diagnostic. It verifies the reference and records selected case IDs. Future proposers can consume the same typed artifacts.

16.5 Candidate generation

Human-authored candidates are the reference implementation. A reflector or optimizer should produce a complete candidate bundle under the same validator. It cannot write directly into a live configuration or bypass the one-mutation rule.

Candidate proposals should be deterministic when possible: include the diagnostic input identity, proposer model and prompt, temperature/seed semantics, native response, and exact resulting asset. The proposal itself may be nondeterministic; the candidate bytes are not.

16.6 Search policy and budget

A campaign can manage a frontier of candidates and a budget of evaluation cells or provider cost. It should select candidates based on prior evidence, not mutate completed runs. Search state is another append-only ledger linked to immutable candidates and evaluation run IDs.

Potential strategies include sequential manual selection, bandit allocation, Bayesian optimization over numeric parameters, evolutionary search, or GEPA-style reflection. None belongs in `ragopt` core because their state and objective assumptions differ. They consume the same evaluation oracle.

16.7 Promotion semantics

A passing gate produces a *reviewable promotion plan*, not a deployment command. The plan contains:

- parent and child snapshot IDs;
- exact changed asset IDs;
- suite, policy, judge, and arm identities;
- comparison and decision artifact IDs;
- linked native artifacts;
- release target description;
- fixed `human_apply_required` state.

A separate release workflow verifies that the child asset and all locked references still match before creating a new immutable release.

17. Security and trust boundaries

17.1 Threat model

The relevant threats include:

- unauthorized source text entering remote provider requests;
- model-supplied scopes or routes expanding authority;
- path traversal in candidate or native artifact handling;
- corrupt or partial cache and run entries being accepted;
- stale corpus expectations producing misleading evaluations;
- missing failures improving aggregate metrics;
- mutable configuration changing between evaluation and promotion;
- prompt or provider identity omitted from caches;
- admin traces exposing sensitive source content.

The proposed kernel does not replace application authentication, encryption, secret management, or network controls. It provides identities and validators that those controls can bind to.

17.2 Capability-indexed plans

An operation declares capabilities such as:

```
artifact.read
artifact.write
index.search.prefilter
provider.embed
authz.evaluate
remote.disclose.source
release.activate
```

A plan compiler checks that the selected adapters satisfy required capabilities. A high-assurance policy can require `index.search.prefilter` and reject bounded overfetch. A development policy can allow overfetch but mark the trace and evaluation dimension.

17.3 Authorization certificates

An authorization certificate should bind:

- subject identity or session generation;
- authorization policy identity;
- release identity;
- source-evidence identities;
- decision time or policy epoch when required.

The certificate is not a bearer token exposed to the model. It is a local typed value checked by remote-disclosure adapters. If the subject or release changes, the certificate no longer matches.

17.4 Path safety

`ragopt` already verifies that candidate assets and native artifacts remain inside assigned roots. The shared artifact layer should reduce reliance on arbitrary paths by using content IDs. When paths are unavoidable:

- resolve symlinks and absolute paths;
- require containment under a canonical root;

- create files with restrictive permissions;
- reject special files;
- verify size and digest after close;
- never trust a product-returned path without containment validation.

The archive itself demonstrated why path handling matters: some entries used traversal-like prefixes to encode an original workspace location. Analysis safely remapped only known repository roots.

17.5 Cache and artifact poisoning

A cache entry must commit to the full semantic key, output schema, output digest, and artifact reference. A mismatched or oversized existing entry fails closed. Provider responses used for replay should be immutable native artifacts, not mutable files at a remembered path.

Domain-separated IDs prevent a valid digest from one schema being interpreted as another. Manifest links prevent replacing one child artifact without changing the root.

17.6 Evaluation integrity

The evaluator must treat timeouts, judge failures, contract failures, and missing cells as explicit outcomes. It must not count a case as success because no statements were produced or because a failing judge result was omitted. Locked assets include suite labels, judge prompt, evaluator implementation identity, safety ceilings, and gate policy.

An optimization candidate must not modify authorization or safety policy as a side effect of an answer-quality experiment. Such a change requires a separate experiment class with different hard gates and review.

17.7 Admin-data minimization

Stage traces should default to identities and summaries. Exact query text, source text, prompts, and model responses should be stored in access-controlled artifacts with retention policy. UI projections should receive only the source metadata and snippets authorized for the current subject.

Because artifact IDs can themselves reveal equality across contexts, highly sensitive deployments may need tenant-scoped hash domains or access-controlled resolution. The kernel should not assume that a content digest is non-sensitive.

Part IV. Applied mappings and migration

18. GEC: from a pragmatic knowledge service to a verified query application

18.1 What GEC already gets right

GEC is the most direct consumer of `ragkit` in the supplied snapshot. Its `internal/knowledge` package opens an immutable index bundle, reconstructs a query-side embedder from the vector manifest when necessary, loads verified source documents, and separates the shared immutable service from per-session evidence state. Its internal `SearchRequest` makes `AccessScopes`, `SourceRoles`, and channel selection server-owned values. This is a strong application boundary: the model can ask to search, but it does not manufacture its own authority.

GEC also contains several mechanisms that should inform, but not be moved wholesale into, the shared RAG layer:

- curated lexical synonym expansion applies only to the BM25 channel;
- hybrid retrieval fuses lexical and vector rankings before optional reranking;
- reranker failure degrades to the fused ranking rather than failing the serving request;
- reranker order is blended with the prior fused order by a second reciprocal-rank fusion;
- the reranker receives title or heading context in addition to raw chunk text;
- source-role and access-scope filters are explicit;
- a per-session evidence ledger records what the model has actually seen;
- product-specific tools and UI projections determine how evidence appears to administrators.

These are not signs that `ragkit` is too weak. They are examples of application policy assembled from shared mechanisms. The architecture should make this composition more explicit and more inspectable without absorbing GEC policy into a universal search service.

18.2 Current-to-target ownership map

Current GEC mechanism	Current location	Target owner	Reason
Bundle open and verification	<code>internal/knowledge/service.go</code> plus <code>ragkit/indexbundle</code>	<code>ragkit/index</code> with thin GEC adapter	RAG artifact semantics are shared; configuration resolution is product-owned
Lexical, vector, collapse, RRF	<code>internal/knowledge/service.go</code> plus <code>ragkit/retrieval</code>	<code>ragkit/retrieve</code>	These are domain-level retrieval operations
Curated synonym groups	<code>internal/knowledge/synonyms.go</code>	GEC	Vocabulary is corpus and product policy
Access scopes and source roles	<code>internal/knowledge</code>	GEC policy compiled into a <code>ragkit</code> filter capability	Authorization semantics are application-owned
Reranker endpoint and document composition	<code>internal/knowledge/rerank*.go</code>	GEC adapter; composition declared in query-plan identity	Provider and presentation text are deployment policy
Reranker fallback	<code>internal/knowledge/service.go</code>	GEC query policy using shared explicit outcomes	Degradation is a serving decision, not a universal rule
Session evidence ledger	<code>internal/knowledge/evidence.go</code> and web runtime	GEC, implemented over <code>evidencekit/ledger</code>	Session meaning and retention are product-owned; reducer mechanics are shared
Eval cases, judge, and sweep	<code>internal/knowledge/eval.go</code> , <code>judge.go</code> , <code>sweep.go</code>	GEC evaluator plus <code>ragopt</code> custody	Native labels and judging remain GEC-specific; run pairing and gates are shared

Current GEC mechanism	Current location	Target owner	Reason
Tool result schema	internal/knowledge/tool.go	GEC	It is an application protocol
Admin chat and projections	internal/webchat, projection packages	GEC	UI, authentication, and conversational state are not RAG-core concerns

The target does not eliminate GEC's `internal/knowledge` package. It changes that package from a second retrieval orchestrator into a compiler and adapter around explicit `ragkit` plans.

18.3 A GEC query profile

A useful application-level configuration is a value that can be compiled into a subject-bound query plan:

```

type QueryProfile struct {
    Release      identity.ID
    LexicalTopK  int
    VectorTopK   int
    RankConstant ordered.Score
    VectorWeight ordered.Score
    SynonymAsset artifact.Ref
    Reranker     *RerankerProfile
    RerankPool   int
    ContextPolicy ragkit.ContextPolicy
    Degradation  DegradationPolicy
}

type RerankerProfile struct {
    Adapter      identity.ID
    Model        identity.ID
    TextComposition identity.ID
    CachePolicy  identity.ID
}

```

The profile is not itself authority. It is combined with a server-derived subject and authorization policy:

```

type SubjectQuery struct {
    Subject      auth.Subject
    Query        ragkit.Query
    AccessScopes []Scope
    SourceRoles  []SourceRole
}

func Compile(
    profile QueryProfile,
    request SubjectQuery,
    opened ragkit.Release,
) (op.Plan[struct{}], GECAnswer, error)

```

Compilation checks that the release named by the profile is the release actually opened by the process, the synonym asset resolves to the exact recorded bytes, requested roles are subsets of server policy, and all source-disclosing stages occur after authorization. The compiled plan can then be rendered for an administrator and identified independently of execution concurrency or retry settings.

18.4 Move authorization before remote disclosure

The current fixed over-fetch strategy ranks a bounded global candidate set and filters it afterward. It prevents unauthorized chunks from being returned to the model so long as no remote component receives source text before filtering. It does not prove completeness for the authorized subset. A relevant authorized result can be absent because unauthorized candidates consumed the bounded prefix.

The migration should make the retrieval capability explicit:

```

type SearchCapabilities struct {
    MetadataPrefilter bool
    Partitioned        bool
    ExhaustiveLocal    bool
}

```


GEC should prefer, in order:

1. backend metadata prefiltering by scope and source role;
2. physically or logically partitioned indexes selected by server authority;
3. exhaustive local retrieval followed by local authorization;
4. bounded over-fetch only as a documented degraded capability.

The query compiler rejects a plan that sends source text to a remote reranker before an `AuthorizationCertificate` has been produced for every candidate. This makes a security property structural rather than dependent on service-method ordering.

The certificate can remain local and compact:

```
type AuthorizationCertificate struct {
    SubjectID    identity.ID
    PolicyID     identity.ID
    ReleaseID    identity.ID
    EvidenceIDs  []identity.ID
    Epoch        uint64
    MAC          []byte
}
```

A remote-disclosure adapter verifies that the certificate binds exactly the evidence being serialized. It need not understand GEC scope strings; it verifies a typed decision made by the GEC policy engine.

18.5 Preserve the GEC reranking policy as policy

GEC's current reranker behavior embodies three distinct decisions:

1. prepare a richer reranker document from heading context and chunk body;
2. treat endpoint failure as a recoverable stage outcome;
3. blend reranker rank with fused retrieval rank rather than replacing it.

These should be represented separately. The shared RAG package can offer `PrepareRerankCandidates`, `Rerank`, and `FuseRankings`. GEC's profile selects the composition and fallback:

```
reranked := op.Then(authorized,
    op.Recover(
        ragkit.Rerank(rerankerSpec),
        GECFallbackUseFused,
    ),
)
final := ragkit.Fuse(
    map[string]ragkit.Ranking{
        "retrieval": fused,
        "reranker":  reranked,
    },
    blendSpec,
)
```

`Recover` should not erase failure. The output is a success-with-warning or a typed degraded result whose observation contains the failed operation ID, failure class, attempted provider, and selected fallback. Evaluation can then gate degradation rate even when serving remains available.

The text-composition identity must be part of the reranker operation specification. Changing title-prefix behavior changes the semantic input to the model and therefore creates a new cache epoch. A source-code comment is not a sufficient identity mechanism.

18.6 GEC evidence ledgers

The per-session evidence ledger has a different purpose from an experiment ledger. It records what evidence labels are available to the model and may enforce bounded context or citation validity. It should remain GEC-owned, but its storage and reducer can use `evidencekit/ledger`.

A minimal event vocabulary is:

```
type EvidenceEvent interface{ isEvidenceEvent() }

type SearchAdmitted struct {
    QueryID      identity.ID
    Evidence      []ragkit.SourceEvidenceRef
    Authorization identity.ID
}

type EvidenceDisclosed struct {
    TurnID      identity.ID
    EvidenceID   identity.ID
    Label        string
}

type EvidenceCited struct {
    TurnID      identity.ID
    EvidenceID   identity.ID
    Label        string
}
```

The pure reducer enforces label uniqueness within a session generation, prevents citation of undisclosed evidence, and computes a bounded view for the next model turn. Storage appends immutable events; retention and access control remain GEC concerns.

This design also clarifies admin inspection. An administrator can ask why a citation was accepted and receive the event chain linking search admission, authorization, disclosure, and citation. The UI need not infer this chain from model transcript text.

18.7 Migrate GEC evaluation custody to ragopt

GEC's retrieval evaluation, judge, and parameter sweep are mature product assets. Replacing their semantics with a generic score would discard useful information. The correct migration is to make one GEC evaluator implement `ragopt/eval.Arm` while retaining a native artifact.

A GEC case remains opaque to `ragopt`:

```
{
  "query": "How should the restricted vault feed be reconciled?",
  "expected_document_ids": ["ops/vault-reconciliation"],
  "access_scopes": ["finance-admin"],
  "source_roles": ["procedure"],
  "required_claims": ["dual-control review"],
  "tags": ["authorization", "hybrid"]
}
```

The GEC arm performs the real search and optional answer/judge loop. It writes a native artifact containing, at minimum:

- exact release and query-profile identities;
- case and repeat coordinates;
- authorized and rejected candidate identities;
- channel rankings and fusion trace;
- reranker request identity and outcome;
- final context identity;
- answer contract result;
- native judge rubric, per-dimension result, and explanation;
- usage, latency, warnings, and degradation events.

It projects only comparable fields into `ragopt.Outcome`, for example recall at fixed cutoffs, reciprocal rank, nDCG, groundedness, answer-quality dimensions, contract validity, unauthorized-disclosure count, provider calls, tokens, and latency. The native artifact remains the source for diagnosis.

A gate should be phased:

1. **hard safety:** no unauthorized disclosure, no path or artifact violation, no invalid contract above the allowed ceiling;
2. **coverage:** every exact arm/case/repeat cell exists or carries an explicit failure outcome;
3. **target improvement:** the candidate improves the named primary metric under the configured paired statistic;
4. **regression limits:** no protected tag slice or secondary metric exceeds its tolerance;
5. **cost and operational tie-break:** use lower provider calls, token cost, or latency only after the earlier phases pass.

This directly replaces ad hoc sweep selection without replacing the GEC evaluator.

18.8 Release identity for GEC

GEC serving currently centers on an index bundle, but a reproducible knowledge behavior also depends on configuration that is not part of the bundle: synonyms, reranker adapter and model, query profile, grounded-answer contract, judge-independent safety policy, and possibly application profile data.

A GEC release manifest should therefore include:

```
type ReleaseManifest struct {
  Schema      identity.Schema
  Index       artifact.Ref
  Synonyms    *artifact.Ref
  QueryProfile artifact.Ref
  RerankerAdapter *artifact.Ref
  Contract     artifact.Ref
  AuthzPolicy  artifact.Ref
  ToolSchema   artifact.Ref
}
```

The active release ID is the content identity of this manifest. Query traces record the release ID, not merely the index bundle ID. Activation verifies every child artifact and publishes one atomic pointer to the release manifest. Rollback changes the pointer to a previously verified release; it does not reconstruct an old configuration from mutable files.

18.9 GEC migration sequence and acceptance criteria

The GEC migration can proceed without changing user-visible behavior:

1. serialize current query profiles and reranker text composition as immutable artifacts;
2. add a release manifest around the already used `ragkit` bundle;
3. emit a structured query trace alongside current logs;
4. replace internal ranking orchestration with a compiled `ragkit` plan while retaining golden output fixtures;
5. add prefilter-capable search or explicitly mark over-fetch as degraded;
6. implement the GEC `ragopt` arm and run it in shadow against existing sweeps;
7. switch experiment custody and reporting to `ragopt`; delete duplicate run/pairing/gate code only after equivalence fixtures pass;
8. move the session evidence ledger storage to the shared reducer infrastructure without changing its product event vocabulary.

Acceptance requires more than compilation. For a fixed local fixture, the new path must produce identical authorized hit identities and ranks, identical fallback behavior, equivalent tool output, and a trace that proves no unauthorized text crossed a remote boundary. Resume tests must demonstrate exact cell custody, and promotion reports must resolve every referenced native artifact.

19. RAG-TTC: eliminate the second RAG core and retain the product system

19.1 The repository currently contains two architectural layers

`rag-ttc` is not merely a consumer of reusable RAG code. It contains both a common substrate copied into `pkg/digest`, `pkg/execution`, `pkg/flow`, `pkg/rag`, `pkg/text`, and `pkg/vector`, and a large TTC-specific product and research layer. The latter includes connected retrieval, product catalogs, tool answer schemas, tool configuration, providers, diagnostics, review, evaluation, chat commands, datasets, and UI components.

The correct migration is not to replace `rag-ttc` with `ragkit`. It is to remove the common substrate from `rag-ttc` so that the repository becomes an honest product layer over `ragkit`.

19.2 Package cut line

The immediate package map is:

RAG-TTC current package	Target	Action
<code>pkg/digest</code>	<code>evidencekit/identity</code> or temporary <code>ragkit/digest</code>	Replace imports, then delete
<code>pkg/execution</code>	<code>evidencekit/op</code> interpreters or retained <code>ragkit/execution</code> during transition	Replace imports; no fork
<code>pkg/flow</code>	<code>evidencekit/op</code> / compatibility layer	Replace imports, then delete
<code>pkg/text</code> , <code>pkg/vector</code>	<code>ragkit/text</code> , <code>ragkit/vector</code> or future focused packages	Replace imports, then delete
<code>pkg/rag</code> root domain types	<code>ragkit/rag</code> compatibility or new semantic packages	Hard cut, then delete copied files
<code>pkg/rag/chunking</code>	<code>ragkit/rag/chunking</code>	Replace
<code>pkg/rag/representations</code>	<code>ragkit/rag/representations</code>	Replace; supply TTC prompt set explicitly
<code>pkg/rag/embedding</code>	<code>ragkit/rag/embedding</code>	Replace
<code>pkg/rag/lexical</code> and backends	<code>ragkit</code> backends	Replace
<code>pkg/rag/vector</code> and backends	<code>ragkit</code> backends	Replace or retain only TTC-specific experimental backend adapters
<code>pkg/rag/indexbundle</code>	<code>ragkit/rag/indexbundle</code>	Replace; accept lexical-only and new identity behavior
<code>pkg/rag/retrieval</code> , reranking, answering, evaluation, dataset, generation	corresponding <code>ragkit</code> packages	Replace
<code>pkg/rag/connected</code> , <code>connectedconfig</code>	TTC product layer	Retain and refactor to depend on <code>ragkit</code>
<code>pkg/rag/productcatalog</code>	TTC product layer	Retain
<code>pkg/rag/toolanswer</code> , <code>toolconfig</code> , <code>knowledgetools</code>	TTC product layer	Retain
<code>pkg/rag/providers/geppetto</code>	TTC adapter layer	Retain outside shared core
<code>pkg/rag/diagnostic</code> , review, <code>tooleval</code> , <code>agenttrace</code>	TTC experiment and admin layer	Retain; integrate with <code>ragopt</code> where applicable
<code>pkg/ttcrag</code>	TTC application facade	Retain, but make its dependencies explicit

The package name does not determine ownership. A package under `pkg/rag` can remain TTC-specific, but it should not pretend to be part of the common RAG kernel. Moving product packages under `pkg/ttcrag` or a top-level `ttcrag` tree would make the boundary legible, but import replacement and implementation deletion matter more than directory aesthetics.

19.3 Use behavioral fixtures before the hard cut

Because many copied files are identical or nearly identical, a gradual adapter mesh would create more risk than it removes. The recommended method is a fixture-driven hard cut:

1. identify every public symbol imported by TTC-specific packages;
2. create golden fixtures for chunk IDs, representation text, representation IDs, bundle manifests, hit ordering, fusion results, hydrated evidence, answer context, grounded contract validation, and usage aggregation;
3. record expected cache-key and bundle-ID changes where the new package intentionally creates an epoch;
4. add `ragkit` as a real module dependency;
5. change all common imports in one branch;
6. delete the copied common packages in the same change;
7. run repository-wide build, tests, fixture comparisons, and selected end-to-end commands;
8. add a boundary test that forbids reintroduction of the deleted package paths.

A compatibility re-export package is acceptable only when an external consumer cannot move in the same change. It must contain aliases, not copied implementation, and have a deletion issue. Internal TTC packages should not use it.

19.4 Resolve the known semantic drift explicitly

The overlap analysis found changes that must not be treated as accidental compile failures:

- `ragkit` includes document identity in the deterministic hit tie-break;
- generated representation prompts are injectable through a `PromptSet` while the default preserves upstream texts;
- lexical-only index bundles are valid;
- grounded-answer contract kind is configurable;
- bundle identity uses a different prefix;
- the extracted module enforces provider and UI dependency boundaries.

The cutover should codify intended behavior:

Ordering. TTC adopts the strengthened total tie-break and adds finite-score validation. Existing fixtures that depended on an unstable tie are defects, not compatibility requirements.

Prompts. TTC constructs an explicit `PromptSet` artifact. The initial bytes equal the old package constants so generated representation and cache behavior remain stable. Future prompt changes become candidates or release changes rather than source edits.

Lexical-only bundles. TTC accepts them in generic open and inspect commands. Product profiles may still require a vector channel and should validate that requirement at profile compilation.

Contract kind. TTC supplies the exact product contract identifier. The shared answer package should not default to a TTC name after the API transition.

Identity prefix. IDs are typed by schema rather than interpreted by string prefix. Where external scripts currently parse a prefix, migrate them to manifest fields. Treat the extraction as an explicit cache and artifact epoch.

Dependency boundary. Geppetto, Pinocchio, Glazed, Cobra, Bubble Tea, and product UI dependencies remain in TTC adapter or command packages, never in `ragkit` core.

19.5 The RAG-TTC query architecture after cutover

The product query path should compile TTC configuration into a `ragkit` plan and then add TTC operations:

```
subject/query
-> TTC intent and route selection
-> ragkit authorized retrieval plan
-> TTC connected or structured-data augmentation
-> ragkit context assembly
-> TTC tool loop and answer schema
-> ragkit grounding validation
-> TTC response and admin projections
```

Connected retrieval and product catalog facts are not representations of source chunks. They are distinct evidence providers with their own provenance. The context type should therefore be a sum of evidence kinds rather than coercing everything into a chunk:

```
type TTCTextItem interface{ isTTCTextItem() }

type SourceChunk struct {
    Evidence ragkit.SourceEvidence
}

type StructuredFact struct {
    QuerySpecID identity.ID
    DatabaseID   identity.ID
    EntityID     string
    Field        string
    Value        json.RawMessage
    Provenance   artifact.Ref
}

type ConnectedResult struct {
    ConnectorID identity.ID
    RequestID   identity.ID
    Payload     artifact.Ref
    Disclosure   identity.ID
}
```

The answer contract can then state grounding rules per evidence kind. A citation to a source chunk proves verbatim lineage; a structured fact proves the query specification, database digest, entity key, and field. These are different proof obligations and should not be collapsed into one string label.

19.6 The existing ragopt adapter is the reference integration pattern

The supplied `cmd/rag-ttc/cmds/chat/tooleval/ragopt.go` already demonstrates the correct dependency direction. It loads a `ragopt` suite, exposes incumbent and challenger arms, materializes the candidate's assets into a product configuration, executes the actual TTC tool loop, invokes the product judge, writes a TTC-native artifact, and projects a narrow `ragopt.Outcome`.

The adapter should be generalized, not moved into `ragopt`. The following pieces remain TTC-owned:

- decoding the TTC case input;
- resolving tool configuration and product prompts;
- executing the TTC chat/tool loop;
- parsing the grounded answer;

- invoking the TTC answer-quality judge;
- deciding which native dimensions are meaningful;
- writing a native artifact that links the session transcript and product trace.

ragopt should own:

- copied and validated candidate inputs;
- exact case/repeat/arm coordinates;
- run creation and resume;
- append-only cell custody;
- strict paired comparison;
- ordered gate evaluation;
- machine and human reports.

The adapter currently writes a run-relative native artifact and projects contract validity, abstention, metrics, provider calls, tool calls, and tokens. That is already a strong model. The kernel migration should replace path-only artifact references with verified content references while preserving product details.

19.7 Candidate materialization should become a plan

The current adapter materializes files into a generated tool configuration. This is pragmatic but can be made auditable. Candidate application should be a pure transformation:

```
type TTCSnapshot struct {
    OrchestrationPrompt artifact.Ref
    AnswerSchema         artifact.Ref
    SearchDescription     artifact.Ref
    ToolProfile          artifact.Ref
    JudgePolicy          artifact.Ref
    Release              identity.ID
}

func ApplyCandidate(
    base TTCSnapshot,
    candidate ragopt.Candidate,
) (TTCSnapshot, error)
```

The function validates exactly one mutable asset, verifies all locked assets, returns a new snapshot value, and does not write files. A separate materializer interprets the snapshot for the legacy runtime. Its output directory and generated config become a derived artifact linked to the snapshot ID. This removes string-built configuration from the semantic boundary and permits differential tests between direct and materialized runtimes.

19.8 Admin chat for RAG-TTC

The planned RAG-TTC admin chat should not introduce another query engine. It should consume the same plan descriptions and trace artifacts as production evaluation. Its capabilities can include:

- resolve a release, plan, build, run, candidate, or cell by typed ID;
- display a redacted stage graph and operation identities;
- compare channel rankings and show collapse/fusion provenance;
- inspect context inclusion and omission reasons;
- resolve native artifacts under administrator authorization;
- replay from exact provider-response artifacts where policy permits;
- create an optimization candidate bundle without mutating live configuration;
- launch an externally authorized experiment command;
- render gate decisions and linked evidence.

It must not own authentication, release activation, direct file mutation, or hidden evaluator changes. Administrative actions should invoke application services that validate typed requests and append auditable events.

19.9 RAG-TTC migration acceptance criteria

The common-core deletion is complete when:

- no TTC source file imports the deleted common package paths;
- `go list -deps` shows `ragkit` as the sole common RAG implementation;
- a boundary test rejects local packages named for the deleted substrate;
- canonical fixtures document all intentional identity epochs;
- end-to-end indexing and query tests run against a `ragkit` bundle;
- the existing `ragopt` proof cycle produces equivalent projected metrics and resolvable native artifacts;
- Garden integration tests pass through the refactored TTC product facade;
- no product provider, CLI, or UI dependency appears in `ragkit`.

Only after this cut should deeper API renaming occur. Removing duplicate authority is higher priority than achieving the ideal package vocabulary.

20. TTC Garden assistant: a product experience over shared evidence

20.1 The Garden assistant is a distinct application

The TTC Garden assistant combines a chat runtime, intent-aware search, product-catalog resolution, structured facts, evidence widgets, source cards, choice interactions, persistence, and calibration. It currently imports `rag-ttc` through a local module replacement and reaches both common RAG packages and TTC-specific packages.

Its architecture should not be reduced to a thin skin over a generic chat package. Garden owns customer experience and interaction semantics. The shared design should instead give it stable evidence and experiment boundaries.

20.2 Target dependency topology

The target dependency path is:

```
TTC Garden backend
-> TTC product facade (`ttcrag` packages)
  -> ragkit
  -> evidencekit
  -> ragopt only from calibration/experiment commands
```

Garden should no longer import common types from `github.com/the-tree-center/rag-ttc/pkg/rag`. Display metadata and tests that need `Document`, `Chunk`, or evidence types should import `ragkit` directly or, preferably, consume TTC facade types. Garden may continue importing TTC product-catalog and tool-configuration packages from the RAG-TTC module until a separate TTC product library is justified.

A later repository split can create a stable `ttcrag` module and leave research commands in `rag-ttc`, but it is not required for semantic cleanup. The immediate rule is that only the TTC facade may compose common RAG and product behavior.

20.3 Product facts require release-level identity

Garden's runtime can combine index evidence with a product-facts database. The backend already exposes a fact-database SHA-256 and records structured-fact provenance. An index bundle ID alone therefore cannot identify the behavior of a Garden release.

A Garden release manifest should bind:

```
type GardenRelease struct {
  TTCQueryRelease  identity.ID
  ProductFacts     artifact.Ref
  ProductQuerySpec artifact.Ref
  IntentPolicy     artifact.Ref
  RuntimeProfile   artifact.Ref
  SystemPrompt     artifact.Ref
  WidgetSchemas   []artifact.Ref
  SourceViewPolicy artifact.Ref
}
```

The product-facts reference should include the exact database bytes or a verified snapshot manifest, not merely a path. `ProductQuerySpec` identifies the fixed queries and field interpretations used to turn the database into facts. A value is not sufficiently grounded by naming a database digest if the query semantics are mutable.

The runtime identity attached to chat turns should include the Garden release ID. Existing profile registry, profile version, fingerprint, and system-prompt fields can remain useful projections, but one root release ID simplifies reproducibility and rollback.

20.4 Keep evidence presentation separate from evidence admission

Garden's source-results tool validates that requested citation labels were admitted in the current conversation, resolves display metadata, optionally augments presentation with product facts, groups evidence, and publishes customer-facing widgets. This is a good separation from retrieval, but the architecture should distinguish three stages more sharply:

1. **evidence production:** source chunks and structured facts are produced with provenance;
2. **evidence admission:** application policy decides which items may enter the current conversation and assigns stable local labels;
3. **evidence presentation:** a widget projection groups and redacts admitted evidence for a customer.

Presentation-only enrichment must not silently become answer evidence. For example, facts fetched solely to decorate a source card should be marked `presentation` and excluded from the answer-grounding set unless separately admitted. A typed role avoids relying on comments:

```
type EvidenceUse uint8

const (
    UseAnswer EvidenceUse = iota + 1
    UsePresentation
    UseDiagnostic
)
```

A widget payload should link each displayed fact either to admitted answer evidence or to an explicitly presentation-only provenance record. This preserves the useful Garden behavior while making the grounding boundary auditable.

20.5 Choice interactions are product-native evaluation state

Garden calibration cases may contain a user turn or a selection by choice identifier or index. The runner verifies that a selected choice was actually offered by the preceding snapshot, derives a deterministic idempotency key, polls until a terminal answer has settled, and records normalized snapshots. This is richer than a single request-response evaluation cell.

`ragopt` should not redefine a case as one prompt. The Garden arm can interpret one opaque case as a multi-turn scenario and produce one cell outcome after the scenario completes. The native artifact retains every `TurnRecord`:

- session and runtime identity;
- turn number and prompt or selected choice;
- idempotency key;
- start and duration;
- normalized terminal snapshot;
- expectation result;
- source kinds and answer/choice counts;
- any create, submit, polling, or expectation failure.

The projected `ragopt.Outcome` summarizes scenario-level metrics such as all-turn pass, intent match, expected source-kind coverage, invalid-choice count, terminal completion, answer-length compliance, provider calls, latency, and cost. It must not discard the first failing turn or convert an absent later turn into success.

20.6 Garden calibration on ragopt

A candidate bundle for Garden should isolate one mutable asset, for example:

- system prompt;
- intent-routing policy;
- search tool description;
- source-card instruction;
- answer schema;
- retrieval profile;
- product-resolution policy.

The suite, release, product database, model profile, widget schemas, expectation interpreter, and judge policy remain locked unless the campaign explicitly studies one of them.

A Garden *Arm* can be defined as:

```
type GardenArm struct {
    RuntimeFactory RuntimeFactory
    BaseRelease     GardenRelease
    Candidate       ragopt.CandidateView
    ArtifactStore   artifact.Store
}

func (a *GardenArm) Run(
    ctx context.Context,
    req ragopt.Request,
) (ragopt.Outcome, error)
```

For each cell it creates an isolated session, executes the scenario with deterministic idempotency coordinates, stores the full normalized transcript and widget/evidence artifacts, evaluates native expectations, and returns the projection. Ordinary session or expectation failures become completed cell outcomes with explicit failure class. Loss of run custody, artifact-store corruption, or cancellation remains an interpreter error and stops safely.

Paired evaluation should use the same scenario order and repeat coordinate for incumbent and challenger. To reduce temporal provider drift, execution can alternate arms by deterministic cell schedule while preserving exact coordinates. The schedule is execution identity, not experiment semantic identity.

20.7 Garden-specific gates

Garden needs protected dimensions that a generic RAG metric suite would miss. A representative ordered gate is:

1. zero evidence-provenance violations;
2. zero invalid-choice continuation and zero cross-session citation reuse;
3. no increase in failed or nonterminal scenarios;
4. required source kinds present for every protected case;
5. no regression in product-fact accuracy or named-product resolution;
6. target improvement in intent routing, recommendation quality, or scenario pass rate;
7. answer-length and interaction-shape tolerances;
8. latency, provider calls, and token cost tie-breaks.

The gate policy names metrics and tag slices; the native evaluator defines them. A weighted average across all these dimensions would allow a serious interaction or provenance regression to be hidden by answer-style gains.

20.8 Garden admin and support inspection

Garden support tools should project the same release and evidence trace into a customer-support-appropriate view. Useful questions include:

- which release and profile generated this turn;
- which intent was selected and why;
- which source chunks and structured facts were admitted;
- which facts were presentation-only;
- what choice messages were offered;
- which widget publication corresponds to which admitted evidence set;
- whether a fallback or provider error occurred;
- whether the turn matches a calibration failure pattern.

Raw reasoning, sensitive provider payloads, and unrestricted corpus text should not be exposed merely because the interface is called administrative. Trace resolution is capability-controlled and redacted by default.

20.9 Garden migration sequence and acceptance criteria

1. introduce a Garden release manifest binding TTC query release and product facts;
2. record its ID in runtime identity and turn snapshots;
3. refactor Garden imports so common RAG types come from `ragkit` or the TTC facade;
4. make evidence use—answer, presentation, diagnostic—explicit;
5. wrap the existing calibration runner as a `ragopt` arm while preserving native `TurnRecord` artifacts;
6. shadow-run existing calibration output and compare every scenario and turn outcome;
7. move run custody, resume, pairing, comparison, and reports to `ragopt`;
8. delete only the duplicated experiment infrastructure, not the Garden expectation language or normalized snapshot model.

Acceptance requires deterministic choice validation, resolvable exact runtime and release identities, equivalent widget/evidence behavior, exact paired cell coverage, and a demonstrated rollback to a prior full release including product-fact data.

21. Migration program: dependency-ordered hardening

21.1 Principles

The migration should optimize for deletion of duplicate authority, not for the number of packages created. Five principles govern the sequence:

1. preserve working application behavior with golden and differential fixtures;
2. establish one owner for each semantic decision before generalizing APIs;
3. make intentional identity epochs explicit rather than pretending cache compatibility;
4. adopt shared custody around product-native artifacts, not instead of them;
5. avoid permanent dual writes, bidirectional adapters, and compatibility layers without deletion conditions.



Dependency-ordered migration roadmap.

21.2 Wave 0: freeze semantics and create evidence

Before moving packages, record the behavior that matters:

- source document, chunk, representation, and evidence identities;
- fixed chunk byte ranges and source-slice validation;
- lexical and vector rankings on deterministic fixtures;
- collapse, fusion, and reranker-blend results;
- index manifests and open verification;
- answer context and contract results;
- GEC authorization and fallback behavior;
- TTC tool-loop native outcomes;
- Garden multi-turn snapshots, choice resolution, evidence groups, and widgets;
- current experiment run, resume, and missing-cell behavior.

Fixtures should include invalid and adversarial cases: duplicate IDs, non-finite scores, corrupt cache entries, path escapes, missing cells, unauthorized high-ranked candidates, stale documents, provider failure, and partial terminal sessions.

This wave produces a compatibility matrix that states which outputs must remain byte-identical, semantically equivalent, or intentionally epoch-changing.

21.3 Wave 1: introduce the verified evidence kernel

Extract only mechanisms already duplicated or clearly required by both `ragkit` and `ragopt`:

- canonical codec with versioned golden vectors;
- typed identity and domain separation;
- finite ordered numeric values;
- immutable artifact references and verification;
- explicit outcomes and observations;
- append-only ledger primitives and pure reducers;
- law-test helpers.

Do not begin with the full typed plan DSL. Identity, artifacts, outcomes, and reducers provide immediate value and constrain later design. `ragkit` and `ragopt` adopt them behind existing APIs. Existing serialized schemas remain readable where necessary, but new writes use explicit new versions.

Exit criteria are cross-module identity golden tests, no dependency cycle, and a small dependency graph that passes boundary tests.

21.4 Wave 2: make ragkit the sole common RAG implementation

Perform the RAG-TTC hard cut described in Chapter 19. This is the highest-value architectural change because it removes a live fork. Garden should continue to pass through the TTC facade during the cut.

Exit criteria are deletion of copied packages, repository-wide build and tests, documented cache epochs, and import-boundary enforcement. No advanced optimization redesign should block this wave.

21.5 Wave 3: establish full release identities

Add immutable release manifests to GEC and Garden/TTC. A release binds all artifacts that can alter serving behavior, not just the index. Activation uses an atomic verified pointer. Query traces record the release root.

This wave should also classify all provider-facing caches by semantic key and schema. Unknown old entries either migrate through a verifier or become an explicit cache epoch. Do not infer release identity from a directory name or deployment timestamp.

Exit criteria include exact replay from retained artifacts where permitted, rollback tests, and a release diff that identifies every changed child artifact.

21.6 Wave 4: converge experiment custody on ragopt

Implement product-owned arms for:

- the existing RAG-TTC proof cycle;
- GEC retrieval and answer evaluation;
- Garden multi-turn calibration.

Run old and new custody paths in shadow on fixed local suites. Compare cell coordinates, failure classification, native artifacts, projected metrics, pairing, and gate results. Once equivalent, delete duplicate run, resume, pair, and report mechanisms while retaining native evaluators.

Exit criteria are exact resume after interruption, no silently missing cells, immutable terminal runs, and promotion reports whose references all verify.

21.7 Wave 5: introduce statically inspectable plans

With identity and artifact semantics stable, replace orchestration hidden in service methods with typed operation specifications and static composition. Start with one index path and one query path, not a universal engine.

Required interpreters are:

- local execution;
- semantic identity analysis;
- graph/description rendering;
- resource and remote-disclosure analysis;

- deterministic test interpretation.

Caching and concurrency can initially delegate to existing `ragkit/execution` and `flow` code. The plan model earns adoption only if it makes the current system easier to inspect and test.

Exit criteria include plan IDs independent of worker count, pre-execution rejection of an invalid disclosure order, and equivalence with the legacy execution path.

21.8 Wave 6: build registry and activation coordination

Implement the build state machine inside product code first. When both GEC and TTC demonstrate the same event vocabulary and operational needs, extract `ragbuild`. The outer scheduler remains external; a scheduler starts or resumes one coarse build job, while the inner plan interpreter handles stage-level concurrency and caching.

Exit criteria for extraction are two independent adopters, shared state-transition laws, and a clear deletion of product-local duplicated coordination. Without two adopters, keep the code product-local.

21.9 Wave 7: higher assurance

Add model checking for run and build reducers, cross-language canonicalization vectors, adversarial authorization tests, and selective formal proofs. These activities refine a small established kernel; they should not delay removal of the copied RAG core.

21.10 Compatibility policy

Every compatibility mechanism must state:

- old and new schema or import surface;
- which direction conversion is allowed;
- whether conversion preserves semantic or only informational equality;
- read deadline and write deadline;
- owner and deletion condition;
- behavior for unknown fields and invalid legacy values.

New systems should never write both old and new authoritative records. During migration, one is authoritative and the other is a derived projection. Dual authority recreates the original problem at the storage layer.

21.11 Risk register

Risk	Likelihood	Impact	Mitigation
Hidden TTC dependency on copied implementation detail	High	High	symbol inventory, compile-time hard cut, golden behavior fixtures
Cache or bundle IDs change unexpectedly	High	Medium	explicit epoch matrix, typed schemas, no cross-epoch cache sharing
Generic kernel grows into a framework	Medium	High	dependency ceiling, package law, two-consumer rule, boundary tests
Product-native diagnostic detail is lost	Medium	High	native artifact remains authoritative; projection tests
Authorization filtering	High	Medium	protected authorized-subset

Risk	Likelihood	Impact	Mitigation
changes recall			fixtures, capability-labeled strategy
Provider nondeterminism makes equivalence noisy	High	Medium	artifact replay, deterministic local fakes, paired repeats
Old and new experiment stores diverge	Medium	High	shadow one-way projection, exact coordinate comparison, single authority
Build coordinator duplicates scheduler	Medium	Medium	one coarse external job; inner interpreter only
Formal work becomes detached from code	Medium	Medium	proofs target executable kernel specifications and golden vectors
More repositories increase release friction	Medium	Low	keep kernel small, automated compatibility matrix, independent versioning

21.12 Decision checkpoints

Three decisions should be revisited with evidence rather than preference:

Kernel module boundary. If `evidencekit` cannot remain narrow after two adoption waves, merge it with a more appropriate domain-neutral infrastructure module. Do not put it into `ragkit` merely to reduce repository count.

Plan DSL depth. If static plans do not improve identity analysis, trust checks, or testing in two real paths, retain ordinary Go composition with explicit operation specifications. The category-theoretical model is a guide, not a requirement to construct syntax trees everywhere.

Build module extraction. Create `ragbuild` only after two product-local implementations converge. Premature extraction would confuse a state-machine pattern with an actual shared operational contract.

Part V. Verification, operation, and use

22. Verification program

22.1 Assurance is layered, not binary

A production RAG system is not made correct by adding a proof assistant, nor is it unverified merely because providers are nondeterministic. Assurance comes from matching each claim to an appropriate mechanism. Pure codecs and comparators admit strong algebraic tests and possible formal proof. File publication and run custody admit crash tests and state-machine model checking. Retrieval quality requires labeled empirical evaluation. Generative behavior requires repeated paired evidence and explicit uncertainty. Authorization requires both structural plan checks and adversarial integration tests.

The useful question is not “is the RAG engine proven correct?” It is “which property is claimed, what is the trusted boundary, and what evidence would falsify the claim?”

22.2 Verification matrix

Property	Kernel or owner	Primary method	Secondary method
Canonical bytes are deterministic	evidencekit/canon	golden vectors, property tests	cross-language implementation
IDs are domain-separated	evidencekit/identity	type and schema tests	collision-domain adversarial tests
Chunks are exact source spans	ragkit/corpus	unit and property tests	fuzzing invalid ranges/UTF-8 boundaries
Scores induce a total order	evidencekit/ordered	exhaustive small-domain and property tests	optional formal proof
Fusion is permutation-invariant over channel-map order	ragkit/retrieve	property tests	differential implementation
Artifacts are accepted only after complete verification	evidencekit/artifact	fault injection and crash tests	filesystem model/model checking
Run state is append-only and terminally immutable	ragopt/runstore	reducer property tests	TLA+ model
Cells are paired exactly	ragopt/compare	exhaustive coordinate tests	optional formal proof
Gates are lexicographic	ragopt/gate	decision-table tests	reference evaluator
Grounded claims resolve to admitted evidence	ragkit/answer plus product policy	contract tests	adversarial generated outputs
Unauthorized text does not cross remote boundary	product query compiler	static capability analysis	instrumented egress tests
Release activation is atomic and rollback-safe	product or ragbuild	crash/fault tests	TLA+ model
Product behavior improves	product evaluator	paired empirical evaluation	confidence intervals and slice analysis

No single column substitutes for the others. A proof of comparator totality says nothing about corpus relevance, and an excellent evaluation score says nothing about crash-safe publication.

22.3 Canonicalization tests

The canonical codec is small enough to deserve an unusually strict test suite. It should include:

- field-order independence for map-like input before canonical encoding;
- stable Unicode treatment under the selected specification;
- exact integer and finite-float representation rules;

- rejection or normalization policy for negative zero;
- explicit rejection of NaN and infinities where the codec permits floating values;
- no dependence on process locale, timezone, architecture, or map iteration;
- versioned handling of unknown fields;
- golden vectors checked by every supported language implementation;
- domain-prefix tests showing that identical payload bytes under different schemas produce different IDs.

The encoder and decoder need not accept every convenient Go value. A narrow identity schema is safer than attempting to canonically encode arbitrary interfaces, functions, filesystem handles, provider clients, or maps with non-string keys.

A useful conformance record is:

```
{
  "schema": "evidencekit.canon-vector.v1",
  "name": "nested-map-and-finite-score",
  "semantic_value": {
    "query": "spruce for wet soil",
    "weights": {"lexical": 1, "vector": 0.8},
    "top_k": 20
  },
  "canonical_hex": "...",
  "domain": "ragkit.retrieval-plan.v1",
  "sha256": "..."
}
```

The canonical bytes, not the pretty JSON, are authoritative. Pretty JSON is a review projection.

22.4 Identity law suite

For any identity domain D , codec C , and values x and y , the law suite should check:

1. **determinism:** $\text{Identify}(D, C, x)$ is stable across repeated calls;
2. **canonical equivalence:** values defined as semantically equal by C have the same ID;
3. **domain separation:** identifying the same canonical bytes under distinct domains yields distinct IDs;
4. **schema version separation:** changing the schema version changes the ID even when payload bytes are otherwise equal;
5. **round-trip verification:** resolving an artifact and re-identifying its canonical semantic payload matches the declared ID;
6. **no implicit projection:** adding an identity-relevant field changes the ID unless the schema explicitly excludes it;
7. **execution independence:** worker count, retry count, timestamps, temporary paths, and log verbosity do not alter semantic plan identity.

The law suite should expose generated and hand-written values. Hand-written boundary vectors remain necessary because generators often under-sample empty strings, maximum sizes, Unicode edge cases, and values near numeric limits.

22.5 Source-lineage properties

`ValidateChunk` already enforces the central lineage property: the chunk text equals the exact source byte slice. The strengthened corpus package should state additional laws:

- document IDs are unique within a corpus revision;
- chunk IDs are unique and commit to document revision plus span and chunker semantics;
- $0 \leq \text{StartByte} \leq \text{EndByte} \leq \text{len}(\text{Document.Bytes})$;
- the source slice is exactly equal to chunk bytes;
- chunk order is deterministic for one document and chunker plan;

- overlapping chunks are allowed only when declared by the chunker specification;
- generated retrieval representations point to a valid source chunk and cannot be hydrated directly as source evidence;
- a source evidence reference resolves to one exact document revision and chunk span.

Fuzzing should generate arbitrary byte documents, including invalid UTF-8 when the corpus contract permits bytes, and random spans. Text-oriented chunkers should define whether they reject invalid UTF-8 or preserve byte offsets through a documented normalization step. Silent normalization would invalidate source-slice proofs.

22.6 Ordering and fusion properties

Once scores are finite, a comparator can be tested as a total order. For generated values *a*, *b*, and *c*:

- `compare(a, a) == 0`;
- `sign(compare(a, b)) == -sign(compare(b, a))`;
- if *a* ≤ *b* and *b* ≤ *c*, then *a* ≤ *c*;
- either *a* ≤ *b* or *b* ≤ *a*;
- equal scores resolve by the documented stable identity tuple;
- sorting twice yields the same sequence;
- sorting any permutation of a set yields the same sequence.

Weighted reciprocal-rank fusion has additional laws under a valid configuration:

- channel iteration order does not change output;
- adding an empty channel with zero or absent weight does not change output;
- duplicating a channel under a different name generally changes output and must not be silently deduplicated;
- all output scores are finite;
- every fused item originates in at least one input ranking;
- contribution traces sum to the fused score within the declared numeric tolerance;
- ties use the shared total identity order.

These properties are more valuable than a large number of example-only tests because fusion bugs frequently hide in map order, duplicate IDs, malformed ranks, and floating edge cases.

22.7 Artifact-store fault model

Artifact publication should be tested under faults at every boundary:

1. before temporary creation;
2. during child-object write;
3. after write but before synchronization;
4. after file synchronization but before directory synchronization;
5. during manifest write;
6. after manifest verification but before rename;
7. after rename but before parent-directory synchronization;
8. while another process attempts the same publication;
9. when an existing path contains mismatched bytes;
10. when a symlink or special file appears under the target root.

The accepted postcondition is binary: either no committed artifact is visible, or a complete artifact whose root and every child verify is visible. Temporary debris may be garbage-collected, but it is never interpreted as committed state.

Platform semantics differ. The implementation should define the filesystems and operating systems for which crash durability is claimed. On unsupported stores, publication may require an object-store adapter with conditional put semantics rather than pretending that a local rename proof applies.

22.8 Ledger reducer laws

For an event sequence E and reducer R , the run and build ledgers should satisfy:

- **prefix determinism:** reducing the same valid prefix yields the same state;
- **append locality:** reducing $E ++ [e]$ equals applying e to $\text{reduce}(E)$;
- **invalid transition rejection:** no event can create a state outside the transition relation;
- **terminal immutability:** no ordinary event changes an immutable terminal state;
- **coordinate uniqueness:** at most one authoritative terminal cell result exists per coordinate;
- **resume equivalence:** uninterrupted reduction and reduction after any valid persisted prefix produce the same terminal semantic state when the same remaining events are appended;
- **hash-chain integrity:** changing, deleting, or reordering a committed event invalidates the chain;
- **projection purity:** human reports and indexes can be rebuilt from authoritative events and artifacts.

The reducer should not read the wall clock, filesystem, provider, or global configuration. Timestamps are event data. This makes reducer behavior reproducible and model-checkable.

22.9 TLA+ model for experiment custody

A small TLA+ specification can model experiment state without modeling LLM behavior. State variables include:

```
phase      ∈ {Absent, Active, Completed, Failed, Cancelled}
expected   ⊆ Case × Repeat × Arm
started    ⊆ expected
terminal   ⊆ expected
results    : terminal -> OutcomeClass
configDigest : Digest u {None}
```

Transitions include `Create`, `StartCell`, `RecordCell`, `Cancel`, `FailCustody`, and `Complete`. Invariants include:

```
terminal ⊆ started ⊆ expected
phase = Completed => terminal = expected
phase ∈ TerminalPhases => UNCHANGED terminal
no coordinate receives two distinct authoritative outcomes
resume requires the same configDigest and expected set
```

Model checking should explore cancellation between any two events, duplicate delivery, attempted completion with missing cells, and resume under a mismatched configuration. The model need not contain JSONL or file descriptors. Those are implementation refinements tested separately.

22.10 TLA+ model for build and activation

The build model contains a registry of builds, artifact verification state, one active release pointer per target, and previous active releases. Key invariants are:

- only a verified `awaiting_activation` build can be activated;
- activation changes one target pointer atomically;
- the active pointer always resolves to a verified release artifact;
- a rejected or failed build cannot become active;
- rollback selects a previously verified release;
- cancellation never deletes an already committed artifact;
- build retries do not mutate a terminal build record.

A separate liveness property can state that a build with available resources and no permanent failure eventually reaches a terminal state under a fair scheduler. Liveness assumptions must be explicit; no model can prove provider availability.

22.11 Exact pairing and comparison tests

Comparison should be tested over the complete power set of small coordinate grids. For two cases, two repeats, and two arms, generate every subset of present outcomes and assert:

- comparison succeeds only when every required coordinate has one terminal outcome for each arm;
- ordinary failed outcomes are paired and counted rather than dropped;
- duplicate outcomes are rejected;
- arm labels cannot be swapped by input ordering;
- metrics with missing values follow declared policy rather than language defaults;
- non-finite metric values are invalid;
- lower-is-better and higher-is-better directions are applied consistently;
- slices use the same paired coordinate set as the overall comparison unless explicitly defined otherwise.

The comparator should emit a pairing certificate listing the ordered coordinates and the two artifact references used for each pair. A report is then auditable without re-running the evaluator.

22.12 Gate decision tables

Lexicographic gates are best tested as decision tables. Each test specifies phase results and the expected first decisive reason. For example:

Hard safety	Coverage	Target	Regressions	Cost	Decision
fail	pass	pass	pass	pass	reject: hard safety
pass	fail	pass	pass	pass	reject: incomplete evidence
pass	pass	fail	pass	pass	reject: target
pass	pass	pass	fail	pass	reject: regression
pass	pass	pass	pass	fail	reject or tie by configured cost rule
pass	pass	pass	pass	pass	eligible for review

Tests should permute the input metric map to prove that map order cannot affect the phase order. A gate policy is invalid when it names an undefined metric, uses contradictory bounds, or attempts to use a non-finite threshold.

22.13 Grounding validation

Grounding is a contract between answer syntax, admitted context, and evidence provenance. Tests should cover:

- every cited label resolves in the current context generation;
- duplicate labels do not alias different evidence;
- source quotations are exact or satisfy a declared normalized-match policy;
- generated representations cannot be cited as sources;
- structured facts carry database and query-spec provenance;
- an abstention has no unsupported positive claims under the selected contract;

- empty statements are not automatically counted as success unless abstention is explicit;
- evidence from another session, subject, release, or turn generation is rejected;
- malformed model output fails closed or enters a typed repair path whose identity is recorded;
- answer contract version is part of the plan and release identity.

Property-based testing can generate valid contexts and then mutate labels, evidence IDs, release IDs, spans, or fact provenance one field at a time. Each mutation should invalidate the corresponding certificate.

22.14 Authorization noninterference tests

A structural query-plan analyzer should reject any path in which a value containing source text can reach a capability labeled `remote.disclose.source` without passing through an authorization operation. Integration tests should then instrument all remote adapters and assert that observed payload evidence IDs are a subset of the certificate.

Adversarial fixtures should place unauthorized documents at the top of lexical and vector rankings, create equal-score ties, request more results than the authorized partition contains, induce reranker fallback, and attempt to inject scope names through model tool arguments. The server-owned subject and policy must remain the only source of authority.

The strongest practical invariant is:

```
For every remote disclosure event d,
there exists an authorization certificate c such that
subject(d) = subject(c), release(d) = release(c),
and evidence(d)  $\subseteq$  evidence(c).
```

This is a trace property. The kernel can verify it from events even when it cannot prove the external provider deleted the received text.

22.15 Differential migration tests

During migration, old and new implementations should run against identical deterministic fixtures. A differential harness compares semantically meaningful projections:

- IDs and manifests where byte identity is required;
- ordered evidence identities and scores within declared tolerance;
- failure classes and fallback choices;
- context membership and omission reasons;
- contract validity and citation mapping;
- exact evaluation coordinates and projected metrics;
- release child-artifact sets.

Differences require classification:

- intended correctness fix;
- intended identity epoch;
- serialization-only difference with semantic equivalence;
- nondeterministic provider variation;
- defect.

A migration cannot label unexplained differences “expected.” The classification becomes a checked artifact in the change review.

22.16 Statistical evaluation without loss of custody

Deterministic retrieval metrics need no repeated provider sampling when the index and query path are deterministic. Generative or remote-reranker evaluations may require repeats. The experiment plan should state the unit of pairing, repeat count, randomization or alternation schedule, provider settings, and uncertainty method before execution.

Useful analyses include paired mean or median differences, paired bootstrap intervals over cases, sign tests for ordinal outcomes, and protected-slice bounds. These statistics supplement, rather than replace, hard gates. Missing or failed outcomes remain explicit and participate under a declared policy; they are not removed to satisfy a statistical routine.

The report should separate:

- observed paired effect;
- uncertainty under the stated sampling model;
- operational failure rate;
- deterministic contract and safety counts;
- decision policy.

A statistically uncertain improvement may still be rejected by policy, while a statistically clear style gain cannot override one unauthorized disclosure.

22.17 Performance and resource conformance

Correctness includes honoring declared resource policy. Benchmarks should measure:

- index build throughput by stage and artifact size;
- cache hit and verification cost;
- query latency by channel, authorization, rerank, context, and generation;
- peak memory for candidate pools and context assembly;
- file-descriptor and connection bounds;
- provider request concurrency and admission behavior;
- runstore append and resume cost;
- artifact verification and garbage-collection cost.

A plan's resource analysis can be checked against execution observations. Exceeding a hard budget is a typed failure. Soft estimates can emit variance warnings and inform scheduling. Worker count may alter performance but must not alter semantic outputs for deterministic operations.

22.18 Proof candidates

Machine-checked proof is most defensible for pure, stable kernels. Candidate theorems include:

- the finite-score comparator is a total order;
- lexicographic gate evaluation returns the first failing phase and cannot be changed by later phases;
- exact pairing creates a bijection between required coordinates and paired outcomes when it succeeds;
- the ledger reducer preserves terminal immutability under all accepted events;
- domain-separated identity cannot equate two values from distinct domains under the assumed hash model;
- a simple authorization plan type system prevents source-text values from reaching remote-disclosure nodes without a certificate.

Proof candidates require generated vectors that continuously test refinement to production code.

23. Operations, versioning, and governance

23.1 Version every semantic boundary

The system needs versions for more than APIs. At minimum, version:

- canonical codecs and identity schemas;
- document and corpus manifests;
- chunker and representation specifications;
- index bundle and backend formats;
- query, reranker, context, and answer plans;
- native artifact schemas;
- evaluation suites and metric schemas;
- run and ledger events;
- gate policies and reports;
- release manifests;
- authorization policy and certificate formats.

A schema version is identity-relevant when it changes interpretation. It should not be inferred from a Go module version because one module release can contain multiple persistent schemas, and old artifacts may remain readable across module upgrades.

23.2 Read compatibility versus write authority

Each component should define a read set and exactly one current write version. Reading old artifacts is a compatibility service; writing multiple authoritative versions creates ambiguity.

A migration adapter can produce a new artifact from an old one, but the new artifact must link to the old source and record the adapter identity. It is a derived migration, not the same artifact with a new label. Unknown or unsupported old versions fail with a diagnosable error rather than being partially decoded into zero values.

23.3 Cache epochs

A cache epoch changes whenever a semantic input or interpretation changes, including:

- prompt or message composition;
- provider/model/endpoint semantics;
- embedding dimensions or normalization;
- reranker document composition;
- answer contract or repair behavior;
- canonical codec;
- output schema;
- deterministic postprocessing;
- authorization policy when cached outputs contain subject-dependent data.

The cache key should identify the operation specification and exact input identity. A human-friendly epoch label can aid operations, but it does not replace structured identity. Old caches may remain on disk and become unreachable; background garbage collection is safer than key aliasing.

23.4 Release activation

Activation should be a small privileged operation:

1. resolve the proposed release manifest by exact ID;
2. verify the root and every required child artifact;
3. check target compatibility and policy approvals;
4. append an activation-intent event;
5. atomically update the target's active release pointer;
6. append an activation-observed event after health checks;
7. retain the previous pointer for rollback.

The optimizer, evaluator, and report renderer do not hold activation credentials. A report may state “eligible for review,” never “deployed.”

A release pointer can be a small signed file, database row with compare-and-swap version, or object-store key with conditional update. The adapter must state its atomicity assumptions.

23.5 Rollback

Rollback selects a prior verified release root. It does not rebuild an old index or reconstruct prompt files from source control at incident time. A rollback test should exercise all child artifacts, including product databases, synonyms, tool schemas, and authorization policy.

Data migrations require special treatment. If a runtime writes state that is incompatible with the prior release, the release manifest must declare a state-compatibility range and rollback procedure. RAG artifacts are often immutable, but chat persistence and product databases may not be.

23.6 Artifact retention and garbage collection

Content-addressed storage needs reachability rules. Roots include:

- active and retained releases;
- nonexpired experiment runs and promotion reports;
- retained build records;
- legal or incident holds;
- explicitly pinned diagnostic artifacts.

Garbage collection traverses verified manifest links from roots, computes unreachable objects, applies retention periods, and deletes only after a second verification pass or quarantine period. It should not parse arbitrary product payloads to discover links; links must be declared in typed manifests.

Provider-native artifacts and transcript data may require shorter retention than index artifacts. A manifest can reference a redacted durable summary while the sensitive payload expires. Resolution then returns an explicit expired or redacted status rather than a dangling path.

23.7 Privacy and tenancy

Content digests can reveal equality. Two tenants with the same sensitive document would receive the same global digest under a global domain, permitting equality inference to anyone who can observe IDs. Deployments with stronger confidentiality requirements can use tenant-scoped domains or keyed identities for externally visible references while retaining internal unkeyed checksums for corruption detection.

Artifact authorization is independent of identity. Knowing an ID does not grant resolution. Stores must enforce subject, tenant, purpose, and retention policy. Admin APIs should return metadata projections unless the caller has capability to resolve exact content.

Query and evaluation artifacts should classify fields such as user text, source text, model response, reasoning payload, and judge explanation. Redaction is schema-aware; string replacement over arbitrary JSON is insufficient.

23.8 Observability

Every execution should expose structured observations keyed by plan, operation, execution, release, and subject-safe correlation IDs. Core fields include:

- operation start and terminal outcome;
- cache decision and verified artifact IDs;
- resource admission and wait;
- provider, backend, and adapter identities;
- input/output counts and byte sizes;
- token and call usage;
- latency distributions;
- degradation and warning classes;
- authorization certificate and disclosed evidence counts;
- artifact publication or resolution failures.

Metrics labels must avoid high-cardinality raw IDs unless the telemetry backend is designed for them. Full identities belong in traces or artifacts; aggregate metrics use bounded schema, provider, operation kind, release channel, and failure class labels.

23.9 Service-level objectives

SLOs should be attached to stages and release behavior rather than one opaque request duration.

Examples include:

- percentage of queries with verified release resolution;
- search and reranker availability, including degraded success rate;
- unauthorized-disclosure count, with a target of zero;
- answer-contract validity rate;
- p50/p95/p99 query stage latency;
- build completion and verification time;
- runstore append durability and resume success;
- artifact resolution success for retained reports;
- rollback completion and health-check success.

A degraded result should be visible in SLOs even when the user receives an answer. Otherwise, silent fallback can become the normal path.

23.10 Reproducibility levels

Operations should label reproducibility honestly:

1. **semantic-plan reproducible:** the same versioned plan and input identities can be reconstructed;
2. **material replayable:** exact outputs and native provider artifacts are retained and resolvable;

3. **deterministically recomputable:** executing the plan under the specified environment produces the same material artifacts;
4. **statistically repeatable:** nondeterministic execution produces behavior within declared evaluation bounds.

Provider-based generation often satisfies the first, second when artifacts are retained, and fourth, but not the third. Reports should not use “reproducible” without stating the level.

23.11 Module release policy

`evidencekit`, `ragkit`, and `ragopt` should release independently. A compatibility table records supported schema versions and minimum module versions. Semantic versioning covers exported Go APIs, while persistent schema compatibility is documented separately.

A release checklist for a shared module includes:

- public API diff;
- persistent schema diff;
- canonical golden-vector diff;
- identity epoch declaration;
- dependency graph and binary-size diff;
- boundary-test result;
- law and fuzz suite result;
- downstream conformance for GEC and TTC fixtures.

A module release that changes canonical identity without an explicit epoch declaration should fail CI.

23.12 Ownership map

A clear ownership map prevents semantic drift:

Decision	Owner
Canonical identity and artifact verification laws	evidencekit maintainers
Document/chunk/representation semantics	ragkit maintainers
Retrieval and answer plan semantics	ragkit maintainers with adopter review
Candidate, pairing, comparison, and gate semantics	ragopt maintainers
GEC scopes, roles, judge, and admin behavior	GEC maintainers
TTC tool loop, product facts, connected data, and answer schema	RAG-TTC maintainers
Garden intent, choices, widgets, and calibration expectations	Garden maintainers
Activation authority and target policy	deployment owner, outside optimizer

Cross-cutting changes require approval from the semantic owner and at least one affected adopter. Ownership is about deciding meaning, not merely merging code.

23.13 Architecture decision records

Persistent decisions should be recorded as ADRs with the following minimum fields:

- context and problem;
- semantic owner;
- alternatives considered;
- decision and invariants;

- identity and schema consequences;
- migration and deletion plan;
- verification evidence;
- security and privacy impact;
- conditions that would reopen the decision.

The most important initial ADRs are listed in Appendix D. ADRs should link to executable boundary tests or law suites where possible.

23.14 The two-consumer rule

A shared package is justified when two independent consumers need the same semantics, not merely similar code. The rule applies especially to `ragbuild`, campaign search, common admin APIs, and provider abstractions.

Two implementations that both use JSON files are not necessarily the same ledger. Two systems that both transition from `active` to `completed` may still have different custody invariants. Extraction follows a comparison of event meaning, failure semantics, and operational assumptions.

The exception is a correctness kernel whose duplication itself creates unacceptable risk, as with canonical identity or total ordering. Even then, the package must remain narrow.

23.15 Change review questions

Every change to indexing, querying, or optimization should answer:

1. Which semantic identity changes?
2. Which exact artifacts change?
3. Does the change alter a trust boundary or remote payload?
4. Which release root contains the change?
5. What is the fallback, and is it observable?
6. Which native evaluation artifacts test the change?
7. Are cells paired exactly and failures retained?
8. Which hard and regression gates apply?
9. Can the old release be replayed or rolled back?
10. What code or compatibility path will be deleted?

These questions convert architecture from a diagram into routine engineering practice.

24. Worked examples

24.1 Example A: build a hybrid index

Assume a corpus snapshot contains Markdown documents and a build should produce raw and breadcrumb representations, BM25 and exact-vector indexes, and a verified bundle.

The semantic input is:

```
type BuildSpec struct {
  Corpus      artifact.Ref
  Chunker     ragkit.ChunkerSpec
  Representations []ragkit.RepresentationSpec
  Lexical     ragkit.LexicalIndexSpec
  Vector      *ragkit.VectorIndexSpec
  Embedding   *ragkit.EmbeddingSpec
  BundleSchema identity.Schema
}
```

The plan is statically composed:

```
verify corpus
-> chunk documents
-> validate source lineage
-> generate representations
-> validate representation ownership
-> [build lexical || embed representations -> build vector]
-> assemble bundle manifest
-> verify bundle
-> publish artifact
```

The build plan ID commits to the corpus reference and every semantic specification. It excludes worker counts, temporary directories, retry delays, and request batching unless batching changes provider semantics. The execution ID commits to the plan ID plus execution policy and environment description.

Suppose the embedding provider returns nondeterministic vectors for the same request. Two executions can share a plan ID but produce different vector child artifacts and therefore different bundle artifact IDs. Both are valid only if they pass dimensionality, finiteness, ownership, and bundle verification.

Evaluation and activation refer to the exact bundle artifact.

The build ledger might contain:

```
BuildRequested(plan=P, target=staging)
SnapshotVerified(corpus=C)
StageStarted(chunk)
StageCompleted(chunk, artifact=K)
StageCompleted(represent, artifact=R)
StageCompleted(lexical, artifact=L)
StageCompleted(embed, artifact=E)
StageCompleted(vector, artifact=V)
BundleAssembled(root=B, children=[C,K,R,L,E,V])
BundleVerified(root=B)
BuildAwaitingActivation(release=Q)
```

An evaluation run compares release Q to the incumbent. A passing gate produces a promotion plan. A deployment operator activates Q through an atomic pointer update. The build executor never writes the active pointer directly.

24.2 Example B: authorized GEC query with reranker failure

A finance administrator asks a question. The application resolves subject S, active release R, and server-owned scopes. The model-provided tool arguments contain only query text and a requested result count; any scope-like strings in the prompt are ignored.

The query plan executes:

1. verify release R and open its index artifact;
2. normalize the query under profile Q;
3. expand curated synonyms for the lexical channel only;
4. search lexical and vector channels with backend metadata prefilter;
5. collapse representations to source chunks;
6. fuse rankings deterministically;
7. evaluate GEC authorization and issue certificate A for the selected candidate set;
8. compose heading-prefixed reranker documents;
9. invoke the remote reranker with A;
10. observe a provider timeout and return a typed degraded outcome;
11. select the fused order under the GEC fallback policy;
12. hydrate and assemble bounded context;
13. disclose the authorized context to the answer generator;
14. validate the grounded answer contract;
15. append evidence-disclosed and evidence-cited events.

The user receives an answer. The trace still records the reranker failure. The semantic query plan ID is unchanged by the timeout; the execution outcome differs and the exact answer artifact receives its own identity.

An admin view can show:

```
Release: R
Plan: P
Authorization: A, 18 candidates admitted, 6 selected
Lexical: 20 hits; Vector: 20 hits; Fused: 28 chunks
Reranker: degraded/provider timeout; fallback=fused_order
Context: 6 evidence items, 7,840 runes
Contract: valid; 4 claims; 4 cited
Remote disclosure: reranker 18 authorized items; generator 6 authorized items
```

No raw unauthorized candidate text is present in the trace projection. A privileged resolver can inspect exact artifacts under separate authorization.

24.3 Example C: optimize a TTC search-tool description

The incumbent TTC snapshot contains orchestration prompt, answer schema, search description, tool profile, judge policy, and release. A candidate changes only the search description.

Candidate validation proves:

```
changed assets = {search_description}
locked asset digests match base snapshot
candidate asset is contained under copied candidate root
candidate snapshot has a new content ID
```

The suite has 80 cases and two repeats. The required coordinate set contains $80 \times 2 \times 2 = 320$ cells. ragopt creates an active run whose config digest commits to suite, incumbent, challenger, repeat count, metric schema, evaluator identity, and run policy.

For each cell, the TTC arm:

- materializes the exact snapshot into an isolated runtime directory;
- executes the real tool loop;
- stores the session transcript and tool trace;
- parses the answer contract;
- invokes the locked judge;

- writes a native TTC artifact;
- projects quality dimensions, contract state, abstention, provider calls, tool calls, and tokens.

One challenger cell times out after the provider has accepted the request. This is an ordinary completed outcome with `Completed=false` or a typed failure class according to the metric schema; it is not dropped. The paired comparator includes the failure against the incumbent cell at the same case and repeat.

The gate passes hard contract limits but rejects the candidate because the protected comparison slice regresses despite a small overall quality gain. The report links the exact failing pairs and native transcripts. No weighted average is allowed to hide the slice regression.

A later candidate may use those diagnostics, but it receives a new candidate ID and run. The rejected run is immutable.

24.4 Example D: optimize a Garden multi-turn interaction

A Garden calibration case is:

```
id: narrow-evergreen-followup
expected_intent: recommendation
evidence_need: [source_chunk, product_fact]
turns:
  - user: "I need a narrow evergreen for zone 6 and wet soil."
    expect:
      choices: true
      min_choices: 2
      max_answer_words: 100
      source_kinds: [source_chunk]
  - choose_index: 0
    expect:
      final_answer: true
      source_kinds: [source_chunk, product_fact]
      max_answer_words: 180
```

The incumbent and challenger differ only in the intent-routing policy. Each arm starts an isolated session under the same Garden release except for the candidate asset. The first turn produces choices. The runner verifies that index 0 exists in that exact preceding snapshot and submits its associated message with an idempotency key derived from run, case, turn, and arm coordinate.

The native artifact records both turns, normalized snapshots, admitted source evidence, structured product facts, widget publications, and expectation results. The projected outcome reports scenario pass, intent match, required-source coverage, choice validity, answer-length compliance, latency, and provider usage.

Suppose the challenger selects a better product but publishes a presentation-only fact as though it were admitted answer evidence. The provenance hard gate rejects the candidate even though a human might prefer the wording. The native artifact identifies the exact evidence-use mismatch.

24.5 Example E: inspect a disputed answer

An administrator receives an answer ID from a support ticket. The admin service resolves:

```
answer artifact -> execution -> semantic query plan -> release
                -> context artifact -> evidence references
                -> authorization certificate
                -> source or structured-fact provenance
```

The service first returns a redacted overview. A caller with source-resolution capability can request one evidence item. The resolver verifies tenant, subject, retention, and release constraints before reading the exact artifact. The administrator can determine whether the claim was unsupported, whether the source was stale, whether the wrong product entity resolved, or whether the UI presented different evidence than the generator received.

The answer is not “explained” by asking another model to narrate a trace. The trace is a typed evidence graph. A model may summarize it, but the underlying links and checks are deterministic.

24.6 Example F: resume after process failure

An evaluation process completes 117 of 320 cells, appends and synchronizes each result, then loses power. On restart, `ragopt` opens the active run and verifies:

- run schema and config digest;
- event hash chain;
- copied suite and candidate artifacts;
- expected coordinate set;
- uniqueness and validity of 117 terminal cell records;
- resolvability of each native artifact.

It schedules only the 203 missing coordinates. A cell whose native artifact exists but whose terminal event was not committed is not assumed complete; the product arm may detect an idempotent session artifact and safely reproject it, or execute again under policy. A terminal cell is never overwritten.

Completion is accepted only after all 320 coordinates have terminal outcomes. The uninterrupted and resumed runs may have different execution IDs and schedules but the same experiment semantic identity. Their exact material run artifacts differ because event timestamps and provider outputs differ.

25. Conclusions

25.1 Answers to the research questions

RQ1: smallest shared kernel. The common requirement is a verified evidence substrate, not a generic RAG pipeline. Versioned canonical identity, immutable artifacts, finite ordering, explicit outcomes, composable observations, append-only reducers, and law tests are shared honestly by `ragkit` and `ragopt`. RAG semantics remain in `ragkit`; experiment semantics remain in `ragopt`.

RQ2: compositional structures. Indexing, querying, and evaluation can be modeled as typed transformations with sequential composition and independent parallel product. Effects are interpreted explicitly. An arrow-like free plan is appropriate for the high-assurance core because future structure must be inspected for identity, budget, capability, and trust-boundary analysis. Dynamic orchestration remains available at application boundaries.

RQ3: shared versus product-owned mechanisms. Retrieval, lineage, fusion, context, grounding, and IR metrics are RAG-domain concerns. Candidate custody, exact pairing, comparison, and gates are optimizer concerns. GEC scopes and judges, TTC tool loops and product facts, and Garden choices and widgets remain product-owned. Native artifacts preserve their semantics while shared packages manage references and evidence custody.

RQ4: executable correctness. The architecture identifies small kernels for canonical identity, lineage, total ranking, artifact commit, ledger reduction, exact pairing, lexicographic gating, grounding, and authorization-before-disclosure. Unit, fuzz, property, differential, crash, model-checking, and selective proof techniques apply to distinct claims.

RQ5: migration. The first priority is a fixture-backed hard cut from the copied common RAG implementation in `rag-ttc` to `ragkit`. The next priorities are full release identities and adoption of `ragopt` custody through product-owned arms. The shared kernel can be introduced incrementally, and `ragbuild` should remain conditional on two proven adopters.

25.2 Immediate architectural decisions

The supplied system should adopt the following decisions now:

1. `ragkit` is the sole source of common RAG semantics and implementation.
2. `ragopt` remains independent of RAG and product packages.
3. a small domain-neutral kernel is introduced for identity, artifacts, outcomes, ordering, and ledgers;
4. every serving application records a full release ID rather than only an index ID;
5. product evaluators retain native artifacts and project narrow `ragopt` outcomes;
6. authorization precedes every remote source-text disclosure;
7. non-finite retrieval and metric scores are rejected at construction;
8. experiment comparison requires exact coordinate pairing and explicit failures;
9. promotion output remains human-apply-required and separate from activation;
10. compatibility paths require deletion conditions.

25.3 What should not be unified

The design explicitly rejects several forms of unification:

- one universal `Evidence` struct for source chunks, experiment proof, structured facts, and admin traces;

- one weighted scalar objective for safety, quality, and cost;
- one workflow engine that owns both stage execution and infrastructure scheduling;
- one chat package shared across GEC and Garden;
- one provider abstraction inside the correctness kernel;
- one identity for plan, output, run, and release;
- one generic evaluator that discards product-native artifacts;
- permanent dual implementations for migration safety.

Good architecture is not maximum reuse. It is singular ownership of shared meaning and explicit boundaries around meanings that differ.

25.4 The deeper result

The category-theoretical vocabulary is useful because it exposes a practical design criterion: composition is trustworthy only when the connecting objects have stable meaning. A function from “some JSON” to “some JSON” composes mechanically but proves little. A morphism from `VerifiedCorpus` to `ValidatedChunks`, or from `CompletePairedRun` to `GateDecision`, carries obligations that can be checked.

The same principle explains proof-carrying artifacts. Index builders, providers, tool loops, and evaluators may remain large and effectful. Their outputs enter the durable system only through smaller validators that establish exact identities and invariants. This is the realistic route to a high-assurance RAG platform: not proving every line, but shrinking the set of lines whose correctness the rest of the system must trust.

25.5 Final recommendation

Treat the current applied implementations as the source of architectural truth. Extract only the algebra they demonstrably share, preserve their native evidence, and delete duplicate semantic authority aggressively. The resulting system is not a monolithic RAG framework. It is a family of product systems built on a verified evidence kernel, one RAG semantics layer, one experiment-custody layer, and explicit release and trust boundaries.

Appendix A. Empirical inventory

A.1 Scope and counting method

The empirical review covers the Go source present in the supplied archive under five scopes: `ragkit`, `ragopt`, `rag-ttc`, `GEC`, and the Garden backend in `ttc-design-system/backend`. Generated dependencies and vendor trees were not included. Counts were computed from files ending in `.go`; nonblank source lines were counted without attempting to classify generated code, comments, or statements. Test functions were counted by top-level Go functions whose names begin with `Test`. Package counts represent distinct directories containing Go package declarations.

These measurements describe the supplied snapshot, not the present state of any remote repository.

Scope	Go files	Test files	Test functions	Nonblank Go lines	Package directories
<code>ragkit</code>	173	61	273	17,743	23
<code>ragopt</code>	45	9	42	5,925	12
<code>rag-ttc</code>	515	179	905	76,705	77
<code>GEC</code>	200	65	252	28,668	34
Garden backend	70	24	108	8,485	19
Total	1,003	338	1,580	137,526	165

The package counts are not additive in a semantic sense because packages have different responsibilities and sizes. The table is useful mainly to establish that the applied systems are substantial evidence sources and that `ragopt` is intentionally small relative to the product repositories.

A.2 `ragkit` package inventory

The reviewed `ragkit` tree contains the following major responsibilities:

Package area	Observed responsibility	Proposed disposition
<code>rag</code>	domain types, component interfaces, validation, ordering, evidence identity, usage	retain semantics; later split into named semantic packages
<code>rag/chunking</code>	fixed, Markdown, heading-aware chunking	retain
<code>rag/representations</code>	deterministic and generated retrieval material; injectable prompts	retain
<code>rag/embedding</code>	fan-out, caching, budgeted embedding, fixtures	retain domain adapter; move generic cache substrate to kernel where justified
<code>rag/lexical</code> and <code>rag/lexical/bleve</code>	in-memory BM25 and persistent lexical backend	retain; isolate heavy backend dependency
<code>rag/vector</code> and <code>rag/vector/sqliteexact</code>	exact vector search and persistent backend	retain; isolate heavy backend dependency
<code>rag/indexbundle</code>	immutable bundle build, open, inspect, stats, verified documents	retain and evolve to typed artifact links
<code>rag/retrieval</code>	collapse, weighted RRF, hydration, filtering, source roles	retain generic retrieval semantics; product authorization remains outside
<code>rag/reranking</code>	cached reranker decorator and overlap reranker	retain mechanisms; provider adapters outside core
<code>rag/generation</code>	caching, observation, flow adapters	retain RAG-facing generation operations; generic observation moves downward
<code>rag/answering</code>	strategy orchestration, context policy,	retain; make plans and evidence kinds

Package area	Observed responsibility	Proposed disposition
	grounded contract	explicit
rag/evaluation	precision, recall, MRR, nDCG, target validation	retain deterministic IR metrics
rag/dataset	corpus and evaluation loading with digest checks	retain or split into codec adapters
digest	JSON/file digest helpers	replace identity-critical use with versioned canonical kernel
execution	budgets, caches, maps, rates, resource plans	adopt as interpreter substrate; extract only truly common laws
flow	typed steps, policies, batch/bulk composition, reports	evolve toward static operation plans or retain as compatibility interpreter
text, vector	domain-neutral term/vector helpers	retain temporarily; move only with clear ownership
internal/fsutil, internal/jsonutil	atomic and strict utility behavior	absorb into artifact/canonical implementations, not public generic utilities

The existing boundary test is a model worth preserving: shared core code should be unable to import provider, chat, CLI, or UI frameworks accidentally.

A.3 ragopt package inventory

Package	Observed responsibility	Proposed disposition
candidate	base snapshots, one mutable asset, path containment, content digests	retain; use typed artifact refs and canonical schemas
eval	opaque cases, arms, outcomes, suites, exact run/resume	retain product boundary; use shared outcomes/artifacts
compare	build paired comparisons from complete cell evidence	retain and strengthen metric schema
gate	ordered hard/target/regression/cost policy	retain; finite thresholds and decision certificates
report	machine and human output with human-apply requirement	retain
runstore	copied inputs, append-only results, active/terminal custody	retain semantics; implement over shared ledger/artifact substrate
cmd/ragopt	CLI application	retain outside library core

The absence of retrieval, provider, judge, deployment, and search-algorithm packages is a strength. The architecture should not fill those absences merely because the surrounding program is called an optimizer.

A.4 ragkit versus copied RAG-TTC substrate

The overlap analysis compared every file under `ragkit` with the corresponding path under `rag-ttc/pkg`. For normalization, module path strings were treated as equivalent and log-area naming differences were ignored. The result is:

Classification	Files	Meaning
Exact bytes	50	direct duplicate
Equal after limited normalization	47	duplicate apart from module/logging

Classification	Files	Meaning
		names
Changed	68	shared ancestry with source drift
No corresponding path	11	extraction-only metadata, boundary/internal utilities, or new tests
Total ragkit files compared	176	

This classification deliberately does not claim that every changed file has a substantive semantic change. Generated logging identifiers, import paths, fixture locations, and extraction adjustments account for part of the drift. The important architectural result is that the two trees are sufficiently similar to be competing implementations and sufficiently different to be unsafe as interchangeable authorities.

Observed substantive differences include deterministic tie-breaking, prompt injection, grounded-contract configuration, lexical-only bundle support, bundle identity labeling, self-contained datasets, and dependency-boundary enforcement. These differences should be adopted or rejected explicitly during cutover.

A.5 Applied-system overlap inventory

Responsibility	ragkit	ragopt	GEC	RAG-TTC	Garden
Corpus/chunk lineage	primary	none	consumes	duplicate/common plus product	consumes through RAG-TTC
Representation generation	primary	none	bundle consumer	duplicate/common	indirect
Index bundle	primary	artifact-opaque	consumes/build command	duplicate/common	consumes
Hybrid retrieval/fusion	mechanisms	none	product orchestration	common plus product routes	TTC facade
Authorization filtering	generic filter only	none	primary product policy	route-dependent product policy	indirect/customer policy
Context/grounding	primary shared semantics	outcome projection only	tool/session integration	TTC-specific contract and loop	widgets and source presentation
Candidate snapshot	none	primary	sweep-like local mechanisms	ragopt adapter plus local config	calibration candidates not yet unified
Run/resume custody	flow/cache only	primary experiment custody	local eval/sweep	multiple experiment mechanisms plus ragopt	local calibration runner
Judge	adapter boundary only	deliberately absent	product judge	product judge	expectations/product judge potential
Exact pairing/gates	IR metrics only	primary	local selection logic	ragopt integration plus legacy evaluation	local pass/fail aggregation
Native artifacts	bundle/cache	generic reference	traces/judge outputs	session and judge artifact	turn records/snapshots/widgets
Activation/release	bundle publication only	deliberately absent	deployment/application	commands/configs	runtime profiles and local data

The table shows why a single all-purpose package would be misleading. The overlap lies in substrate and custody, while the domain meanings remain distinct.

Appendix B. Consolidated API blueprint

B.1 Evidence kernel

The minimum stable surface should resemble:

```
package canon

type Codec[T any] interface {
    Schema() identity.Schema
    EncodeCanonical(T) ([]byte, error)
    DecodeCanonical([]byte) (T, error)
}

package identity

type Schema struct {
    Domain string
    Name string
    Version uint32
}

type ID struct {
    Schema Schema
    Sum [32]byte
}

func Identify[T any](codec canon.Codec[T], value T) (ID, error)
func Parse(text string) (ID, error)
func (id ID) Verify(schema Schema, canonical []byte) error

package artifact

type Ref struct {
    ID identity.ID
    MediaType string
    Size int64
}

type Manifest struct {
    Schema identity.Schema
    Payload Ref
    Children map[string]Ref
}

type Store interface {
    Put(context.Context, identity.Schema, io.Reader) (Ref, error)
    Resolve(context.Context, Ref) (io.ReadCloser, error)
    Verify(context.Context, Ref) error
    CommitManifest(context.Context, Manifest) (Ref, error)
}

package ordered

type Finite float64
func NewFinite(float64) (Finite, error)
func CompareFinite(Finite, Finite) int

package outcome

type Class string
const (
    Success Class = "success"
    Abstained Class = "abstained"
    Failed Class = "failed"
    Cancelled Class = "cancelled"
)

type Failure struct {
    Kind string
    Message string
    Retryable bool
    Cause *artifact.Ref
}

type Result[T any] struct {
    Class Class
    Class Class
}
```

```

    Value      *T
    Failure    *Failure
    Observations observe.Set
}
package observe

type Usage struct {
    ProviderCalls uint64
    ToolCalls     uint64
    InputTokens   uint64
    OutputTokens  uint64
}

type Set struct {
    Usage      Usage
    Artifacts  []artifact.Ref
    Warnings   []Warning
    Events     []EventRef
}

func (s Set) Combine(other Set) Set
package ledger

type Event struct {
    Sequence uint64
    Previous identity.ID
    Schema   identity.Schema
    Payload  artifact.Ref
}

type Reducer[S any] interface {
    Initial() S
    Apply(S, Event) (S, error)
}

```

The exact wire structures require design review. The invariant is that schemas, IDs, content references, and invalid states are explicit.

B.2 Operation plans

A static plan API can remain compact:

```

package op

type Spec struct {
    Schema      identity.Schema
    Parameters  artifact.Ref
    Determinism Determinism
    Capabilities []Capability
    Resources   []ResourceRequirement
    InputPolicy DataPolicy
    OutputPolicy DataPolicy
}

type Plan[I, O any] interface {
    describe() Node
}

func Primitive[I, O any](spec Spec, impl Implementation[I, O]) Plan[I, O]
func Identity[T any]() Plan[T, T]
func Then[A, B, C any](Plan[A, B], Plan[B, C]) Plan[A, C]
func Fanout[A, B, C any](Plan[A, B], Plan[A, C]) Plan[A, Pair[B, C]]
func Parallel[A, B, C, D any](Plan[A, B], Plan[C, D]) Plan[Pair[A, C], Pair[B, D]]
func Map[A, B any](Plan[A, B]) Plan[[]A, []B]

```

An implementation registry may be supplied to an interpreter so that the semantic syntax does not serialize function pointers. The same plan can be analyzed without provider clients and executed only after adapters are bound.

```

type Analyzer interface {
    Describe(Node) Description
    SemanticID(Node) (identity.ID, error)
    RequiredCapabilities(Node) []Capability
    RequiredResources(Node) ResourceEnvelope
    CheckPolicy(Node, Policy) error
}

```



```

}

type Interpreter interface {
    Run(context.Context, Node, any, ExecutionPolicy) (any, observe.Set, error)
}

```

Dynamic Bind is intentionally absent from the high-assurance core. An application can choose a later plan after inspecting a prior result, but each chosen branch receives its own plan identity and trace.

B.3 ragkit semantic surface

```

package corpus

type DocumentRevision struct {
    ID          identity.ID
    Logical     string
    MediaType   string
    Bytes       artifact.Ref
    Metadata    artifact.Ref
}

type Chunk struct {
    ID          identity.ID
    Document    identity.ID
    Start       uint64
    End         uint64
    Bytes       artifact.Ref
}

func ValidateChunk(DocumentRevision, Chunk, artifact.Store) error

package representation

type Item struct {
    ID          identity.ID
    Chunk       identity.ID
    Kind        string
    Content     artifact.Ref
}

type Spec struct {
    Kind        string
    Generator   *op.Spec
    Prompt      *artifact.Ref
    Parameters  artifact.Ref
}

package retrieve

type Query struct {
    ID          identity.ID
    Text        string
}

type Hit struct {
    Representation identity.ID
    Chunk          identity.ID
    Document       identity.ID
    Score          ordered.Finite
    Rank           uint32
}

type Ranking struct {
    Channel string
    Hits    []Hit
}

type Trace struct {
    Channels    []Ranking
    Collapsed   []Ranking
    Fusion      artifact.Ref
    Authorization *artifact.Ref
    Rerank      *artifact.Ref
}

package answer

type SourceEvidence struct {
    ID          identity.ID

```

```

    Document identity.ID
    Chunk    identity.ID
    Text     artifact.Ref
    Metadata artifact.Ref
}

type Context struct {
    ID        identity.ID
    Evidence []SourceEvidence
    Policy    identity.ID
}

type Contract interface {
    Validate(context.Context, Context, artifact.Ref) (Validation, error)
    Identity() identity.ID
}

```

Backends implement narrow capabilities and produce or consume these values. They do not redefine identity or evidence semantics.

B.4 ragopt semantic surface

```

package candidate

type Snapshot struct {
    ID        identity.ID
    Assets    map[string]artifact.Ref
}

type Candidate struct {
    ID            identity.ID
    Parent        identity.ID
    MutableAsset  string
    Replacement   artifact.Ref
    Locked        map[string]artifact.Ref
}

package eval

type Coordinate struct {
    Case   string
    Repeat uint32
    Arm    string
}

type Case struct {
    ID        string
    Payload   artifact.Ref
    Tags      []string
}

type Arm interface {
    Name() string
    Run(context.Context, Request) (Outcome, error)
}

type Outcome struct {
    Class          outcome.Class
    ContractValid  bool
    Abstained      bool
    Metrics        map[string]ordered.Finite
    Usage          observe.Usage
    NativeArtifact artifact.Ref
    Failure        *outcome.Failure
}

package compare

type Metric struct {
    Name        string
    Direction   Direction
    Unit        string
    Missing     MissingPolicy
}

type Pair struct {
    Coordinate eval.Coordinate
    Incumbent  eval.Outcome
}

```

```

    Challenger eval.Outcome
}

type Result struct {
    ID          identity.ID
    Pairing     artifact.Ref
    Metrics     map[string]MetricResult
    Slices      map[string]SliceResult
}

package gate

type Phase struct {
    Name        string
    Predicates  []Predicate
}

type Policy struct {
    ID          identity.ID
    Phases      []Phase
}

type Decision struct {
    ID          identity.ID
    Eligible    bool
    DecisivePhase string
    Reasons     []Reason
    Comparison  artifact.Ref
    HumanApplyOnly bool
}

```

These APIs do not expose documents, chunks, prompts, judges, or chat sessions. Such values appear only inside opaque case and native artifact references.

B.5 Release and build surface

A provisional application-local API is:

```

type ReleaseManifest struct {
    Schema identity.Schema
    Product string
    Children map[string]artifact.Ref
    Policy artifact.Ref
}

type BuildRecord struct {
    ID identity.ID
    Plan identity.ID
    State BuildState
    Events artifact.Ref
    Release *artifact.Ref
}

type Activator interface {
    Current(context.Context, Target) (artifact.Ref, error)
    CompareAndSwap(context.Context, Target, artifact.Ref, artifact.Ref) error
}

```

This API belongs in `ragbuild` only after GEC and TTC prove equivalent state and target semantics.

Appendix C. Laws, obligations, and conformance suites

C.1 Algebraic laws

Structure	Identity	Operation	Required laws
Observation usage	zero counts	componentwise addition	associativity, identity, no overflow under declared bounds
Warning set	empty	deterministic union/concatenation policy	associativity, identity; stable order if serialized
Artifact link graph	leaf	named manifest composition	deterministic names, acyclic verification policy, root commits to children
Plan sequencing	identity plan	Then	left/right identity and associativity up to plan normalization
Independent product	unit pair	Parallel	associativity and symmetry up to canonical structural isomorphism
Ranking order	none	comparator	totality, transitivity, antisymmetry
Ledger state	initial state	event application	prefix determinism and invalid-transition rejection
Gate phases	no predicates/pass	ordered conjunction	first-failure determinism; later phases cannot reverse earlier failure

“Up to structural isomorphism” must not become an excuse for unstable identity. The plan codec chooses one canonical association and field order so semantically equivalent constructed plans normalize before identification.

C.2 Kernel proof obligations

K0 Canonical identity

- the codec is deterministic;
- the schema is explicit and domain-separated;
- unsupported values fail rather than receive implementation-dependent bytes;
- verification recomputes from canonical bytes.

K1 Source lineage

- every source evidence item resolves to one document revision and valid span;
- the bytes equal the source slice;
- retrieval representations cannot substitute for evidence.

K2 Total ranking

- scores are finite;
- the identity tie-break is total;
- sort and fusion outputs are invariant to container iteration.

K3 Artifact commit

- visibility implies complete verification;
- an existing mismatched object is an error;
- publication cannot overwrite a distinct object under the same ID.

K4 Ledger reducer

- accepted events preserve state invariants;

- terminal states are immutable;
- resume from any committed prefix is equivalent to continuous reduction.

K5 Exact pairing

- success yields one pair for every required coordinate and no extras;
- missing, duplicate, or mismatched coordinates fail;
- failed outcomes remain values in pairs.

K6 Lexicographic gate

- decision equals the first failing ordered phase;
- later phases cannot compensate for earlier failure;
- invalid metric values cannot enter predicates.

K7 Grounding

- every accepted citation resolves to admitted evidence in the same context generation;
- each evidence kind satisfies its own provenance rule;
- unsupported claim or malformed output follows explicit contract policy.

K8 Authorization boundary

- every source-text disclosure event is covered by a matching authorization certificate;
- model input cannot expand the subject's authority;
- release and subject mismatch invalidate the certificate.

C.3 Required conformance suites

A backend or adapter should pass a suite appropriate to its role.

Searcher conformance

- valid finite scores and positive ranks;
- deterministic output for deterministic backend class;
- stable identity tie-break;
- no unknown representation IDs;
- capability declaration for prefiltering and exhaustiveness;
- cancellation and limit semantics.

Embedder conformance

- exact declared dimensions;
- finite vector components;
- one result per requested item or explicit per-item failure;
- stable association independent of batch ordering;
- identity includes provider, model, dimensions, normalization, and request composition.

Reranker conformance

- output is a permutation/subset according to declared policy;
- no fabricated evidence IDs;
- finite scores where supplied;
- request identity commits to exact candidate text composition;
- failure classification is explicit.

Artifact-store conformance

- put, resolve, verify, duplicate put, corruption, partial write, symlink, concurrency, and crash simulations;
- authorization on resolution where supported;

- immutable object semantics.

ragopt arm conformance

- stable arm name and evaluator identity;
- exact use of request coordinate and candidate view;
- ordinary product failures returned as outcomes;
- custody/cancellation errors returned as errors;
- native artifact contained and verified;
- finite declared metrics only;
- idempotent or explicitly non-idempotent replay semantics.

Release activator conformance

- compare-and-swap correctness;
- verified target only;
- atomic current-pointer observation;
- rollback to retained verified release;
- audit event emission.

C.4 CI layers

A practical continuous-integration order is:

1. formatting, static analysis, boundary imports;
2. unit and golden tests;
3. property and fuzz smoke corpus;
4. canonical cross-module vectors;
5. deterministic differential fixtures for GEC and TTC;
6. runstore/build model checks on bounded state spaces;
7. backend integration tests;
8. optional provider contract tests under controlled credentials;
9. artifact and binary compatibility report;
10. nightly extended fuzzing, fault injection, and product calibration.

Provider-dependent tests should not determine whether pure kernel changes are correct. They run as a separate evidence layer and store exact native artifacts.

Appendix D. Architecture decisions and rejected alternatives

D.1 ADR-001: one common RAG implementation

Decision. `ragkit` owns the common RAG implementation. RAG-TTC deletes copied common packages after fixture-backed cutover.

Rejected alternative. Continue cherry-picking changes between trees.

Reason. Synchronization procedure cannot guarantee singular semantic custody, and divergence already affects correctness and capabilities.

D.2 ADR-002: domain-neutral evidence kernel

Decision. Shared identity, artifact, ordering, outcome, observation, and ledger laws live in a small module imported by both `ragkit` and `ragopt`.

Rejected alternative. Make `ragopt` import `ragkit` for helpers.

Reason. It violates the optimizer's domain-neutral dependency direction and makes RAG semantics appear foundational to all experiments.

Rejected alternative. Duplicate the helpers because they are small.

Reason. Small correctness kernels are precisely the code that should have one law suite and one owner.

D.3 ADR-003: static core plans, dynamic application edges

Decision. Expensive auditable core paths use statically inspectable typed plans. Applications may choose branches dynamically, with each branch receiving an explicit plan identity.

Rejected alternative. Unrestricted monadic bind as the only composition primitive.

Reason. It hides future operations needed for resource, cache, capability, and disclosure analysis.

Rejected alternative. Encode the whole application as a workflow graph.

Reason. Conversation, UI, and product control flow often depends legitimately on dynamic state and would make the graph an inaccurate second programming language.

D.4 ADR-004: native artifacts are authoritative

Decision. Product evaluators write native artifacts; `ragopt` stores verified references and narrow comparable projections.

Rejected alternative. Standardize one universal outcome document containing every diagnostic.

Reason. It either loses product semantics or grows without a coherent law. Opaque verified artifacts permit extensibility without semantic pretense.

D.5 ADR-005: ordered gates, not weighted utility

Decision. Promotion policy is an ordered sequence of predicates: hard safety, coverage, target, regression, and optional cost tie-break.

Rejected alternative. One weighted score across quality, safety, and cost.

Reason. Compensability is wrong for hard constraints and obscures the decisive reason.

D.6 ADR-006: four identity strata

Decision. Plan, material artifact, execution, and release identities are distinct typed values.

Rejected alternative. Use build or run ID as the universal identifier.

Reason. Nondeterministic producers and operational retries break one-to-one correspondence. Release behavior also includes multiple artifacts beyond a build output.

D.7 ADR-007: authorization before remote disclosure

Decision. Source evidence is authorized before any remote reranker or generator receives text, and the disclosure adapter verifies a matching certificate.

Rejected alternative. Retrieve globally, remotely rerank, then filter before returning results.

Reason. Unauthorized content has already crossed the trust boundary.

Rejected alternative. Fixed over-fetch as the permanent authorization strategy.

Reason. It can starve authorized results and cannot prove authorized-subset completeness.

D.8 ADR-008: build scheduling remains external

Decision. The inner plan interpreter handles stage execution; an external scheduler launches or resumes coarse build jobs. A shared build registry is conditional.

Rejected alternative. Make the RAG library a cluster scheduler.

Reason. Scheduling, tenancy, quotas, and infrastructure placement are operational platform concerns with different failure semantics.

D.9 ADR-009: finite scores only

Decision. Retrieval scores, metric values, weights, and gate thresholds are finite validated values.

Rejected alternative. Define an arbitrary order over NaN and infinities.

Reason. Non-finite values usually indicate a producer defect and contaminate aggregates. Rejection localizes the fault.

D.10 ADR-010: full release roots

Decision. Serving records a root manifest covering index, prompts, profiles, product data, contracts, and policy relevant to behavior.

Rejected alternative. Treat index bundle ID or container image tag as release identity.

Reason. Both omit mutable or independently deployed behavior inputs.

D.11 ADR-011: human promotion boundary

Decision. `ragopt` reports eligibility and emits a reviewable promotion plan with `human_apply_required`; activation is separate.

Rejected alternative. Auto-deploy the highest-scoring candidate.

Reason. Evaluator bugs, suite gaps, security policy, and operational readiness require an independent authority boundary.

D.12 ADR-012: two-consumer extraction rule

Decision. New shared operational modules require two adopters with matching semantics, except for narrow correctness kernels.

Rejected alternative. Extract every repeated shape immediately.

Reason. Similar code is not sufficient evidence of shared meaning, and premature generic packages impede deletion.

Appendix E. Glossary

Abstention. An explicit successful decision not to make a substantive answer under the applicable contract. It is not the same as provider failure or empty output.

Arm. A named product executor used by `ragopt` to evaluate one snapshot at one case and repeat coordinate.

Artifact. Immutable bytes or a typed manifest identified by canonical content and verified before acceptance.

Authorization certificate. A local typed proof that a subject and policy admitted a specific evidence set under a specific release or epoch.

Build plan ID. Identity of the semantic intent to produce an artifact, independent of execution scheduling.

Candidate. An immutable proposed snapshot that changes exactly one declared asset relative to a parent while locking all other assets.

Canonical encoding. A versioned deterministic byte representation used for identity, signing, or exact comparison.

Capability. A declared effect or facility required by an operation, such as prefiltered search, provider embedding, artifact write, or remote source disclosure.

Cell. One experiment coordinate: case $\times$ repeat $\times$ arm.

Collapse. Mapping ranked retrieval representations to their owning source chunks, usually retaining the best representative rank under a declared policy.

Context. The exact admitted evidence and formatting policy supplied to a generator for one answer operation.

Custody. The rules by which inputs, events, outcomes, and terminal state are durably recorded and resumed without silent loss or mutation.

Degraded outcome. A serving success produced through an explicit fallback after a stage failure. The warning remains part of observations.

Determinism class. A declaration of whether an operation is pure deterministic, deterministic given retained external artifacts, provider-nondeterministic, time-dependent, or otherwise effect-sensitive.

Evidence. In the RAG domain, verified source material admitted to context. In optimization, the complete paired run and linked native artifacts supporting a decision. The unqualified term should be avoided in APIs where these meanings could collide.

Execution ID. Identity of an attempt or run policy around a semantic plan, including operational details needed for audit but excluded from plan identity.

Grounding contract. A versioned validator relating answer claims and citations to the admitted context and evidence-kind provenance rules.

Hydration. Resolving ranked chunk identities into verified source evidence.

Identity epoch. An explicit break in persistent identity or cache compatibility caused by changed schema, codec, prompt composition, or semantics.

Ledger. An append-only ordered event sequence whose current state is produced by a pure reducer.

Material artifact ID. Identity of exact produced bytes and manifest links.

Monoid. A set with an associative combination operation and identity element. Usage and observation aggregation are practical examples.

Native artifact. A product-owned diagnostic object whose full semantics are not standardized by the experiment framework.

Operation specification. A versioned semantic description of one transformation, including parameters, determinism, capabilities, and data policy.

Pairing certificate. An immutable record of the exact incumbent and challenger outcomes paired at every required coordinate.

Plan. A statically inspectable typed composition of operation specifications.

Proof-carrying artifact. An artifact accompanied by references or certificates that a small validator can check before acceptance; not necessarily a theorem-prover proof.

Release ID. Identity of the complete activated manifest of artifacts and policies that determine product behavior.

Representation. Retrieval material derived from a source chunk, such as raw text, breadcrumbs, a summary, or generated questions. It is not source evidence.

Resume equivalence. The property that continuing from a valid committed prefix yields the same semantic terminal state as uninterrupted processing of the same event sequence.

Semantic plan ID. Identity of what computation is intended, excluding execution scheduling and incidental environment details.

Source evidence. A verified document revision and exact chunk span admitted for use by an answer or presentation policy.

Structured fact. Evidence derived from a declared structured query over an exact database or dataset snapshot, with entity, field, value, and query provenance.

Trusted kernel. The small set of validators, comparators, codecs, and reducers whose correctness is relied upon by the larger system.

Appendix F. Selected bibliography

The bibliography emphasizes primary sources relevant to the architecture. The repository snapshot itself is the primary empirical source for Chapters 2 through 6 and 18 through 21.

- Bormann, Carsten, and Paul Hoffman. 2020. *Concise Binary Object Representation (CBOR)*. RFC 8949, STD 94. DOI: 10.17487/RFC8949.
- Claessen, Koen, and John Hughes. 2000. “QuickCheck: A Lightweight Tool for Random Testing of Haskell Programs.” In *Proceedings of the Fifth ACM SIGPLAN International Conference on Functional Programming*, 268–279. DOI: 10.1145/351240.351266.
- Cormack, Gordon V., Charles L. A. Clarke, and Stefan Büttcher. 2009. “Reciprocal Rank Fusion Outperforms Condorcet and Individual Rank Learning Methods.” In *Proceedings of the 32nd International ACM SIGIR Conference on Research and Development in Information Retrieval*, 758–759. DOI: 10.1145/1571941.1572114.
- Green, Todd J., Grigoris Karvounarakis, and Val Tannen. 2007. “Provenance Semirings.” In *Proceedings of the Twenty-Sixth ACM SIGMOD-SIGACT-SIGART Symposium on Principles of Database Systems*, 31–40. DOI: 10.1145/1265530.1265535.
- Hughes, John. 2000. “Generalising Monads to Arrows.” *Science of Computer Programming* 37 (1–3): 67–111. DOI: 10.1016/S0167-6423(99)00023-4.
- Järvelin, Kalervo, and Jaana Kekäläinen. 2002. “Cumulated Gain-Based Evaluation of IR Techniques.” *ACM Transactions on Information Systems* 20 (4): 422–446. DOI: 10.1145/582415.582418.
- Karpukhin, Vladimir, Barlas Oğuz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. 2020. “Dense Passage Retrieval for Open-Domain Question Answering.” In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing*, 6769–6781. DOI: 10.18653/v1/2020.emnlp-main.550. arXiv:2004.04906.
- Lamport, Leslie. 1994. “The Temporal Logic of Actions.” *ACM Transactions on Programming Languages and Systems* 16 (3): 872–923. DOI: 10.1145/177492.177726.
- Lamport, Leslie. 2002. *Specifying Systems: The TLA+ Language and Tools for Hardware and Software Engineers*. Boston: Addison-Wesley.
- Lewis, Patrick, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. 2020. “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks.” In *Advances in Neural Information Processing Systems* 33, 9459–9474. arXiv:2005.11401.
- McBride, Conor, and Ross Paterson. 2008. “Applicative Programming with Effects.” *Journal of Functional Programming* 18 (1): 1–13. DOI: 10.1017/S0956796807006326.
- Moggi, Eugenio. 1991. “Notions of Computation and Monads.” *Information and Computation* 93 (1): 55–92. DOI: 10.1016/0890-5401(91)90052-4.
- Necula, George C. 1997. “Proof-Carrying Code.” In *Proceedings of the 24th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, 106–119. DOI: 10.1145/263699.263712.
- Paterson, Ross. 2001. “A New Notation for Arrows.” In *Proceedings of the Sixth ACM SIGPLAN International Conference on Functional Programming*, 229–240. DOI: 10.1145/507635.507664.
- Plotkin, Gordon, and John Power. 2003. “Algebraic Operations and Generic Effects.” *Applied Categorical Structures* 11: 69–94. DOI: 10.1023/A:1023064908962.
- Robertson, Stephen, and Hugo Zaragoza. 2009. “The Probabilistic Relevance Framework: BM25 and Beyond.” *Foundations and Trends in Information Retrieval* 3 (4): 333–389. DOI: 10.1561/15000000019.
- Rundgren, Anders, Benjamin Jordan, and Samuel Erdtman. 2020. *JSON Canonicalization Scheme (JCS)*. RFC 8785. DOI: 10.17487/RFC8785.
- Scott, Dana, and Christopher Strachey. 1971. “Toward a Mathematical Semantics for Computer Languages.” Technical Monograph PRG-6, Oxford University Computing Laboratory.

The supplied repository snapshot. August 2026. ragkit, ragopt, rag-ttc, GEC, and ttc-design-system/backend, analyzed as local source without relying on inaccessible worktree history.